



数据挖掘

Data Mining

主讲: 张仲楠 教授



廈門大學
XIAMEN UNIVERSITY



分类方法

目 录

01

基本概念

02

决策树分类器

03

最近邻分类器

04

朴素贝叶斯分类器

05

支持向量机

目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

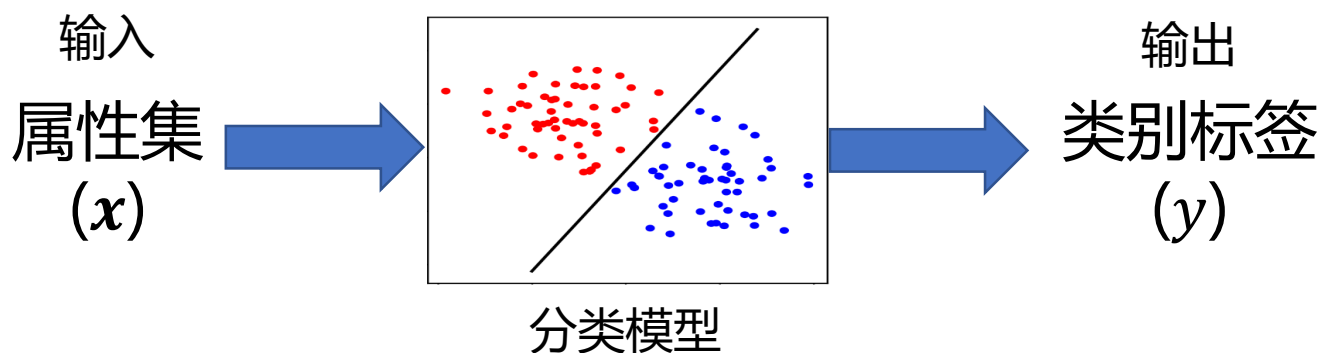
10

类不均衡问题

1. 基本概念

1. 总体思路

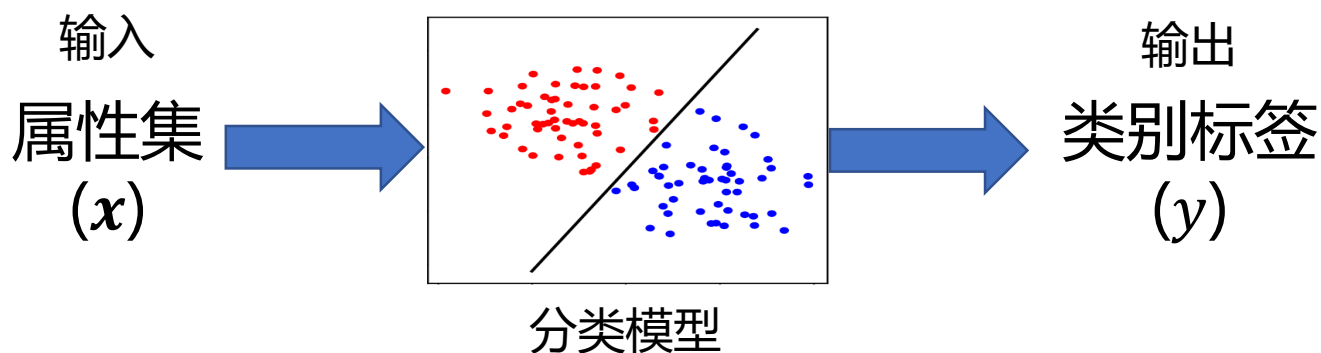
- 分类任务的数据由一组实例(记录)组成
- 每个这样的实例都表示为元组 (x, y) ，其中 x 是描述实例的属性值集合， y 是实例的类别标签
- 属性集 x 可以包含任何类型的属性，而类别标签 y 必须是可分类的



1. 基本概念

1. 总体思路

- 分类模型(classification model)是属性集和类别标签之间关系的抽象表示
- 在数学表达上把它作为一个目标函数 f ，它将属性集 x 作为输入并产生一个对应于预测类别标签的输出
- 如果 $f(x) = y$ ，则该模型可正确地对实例 (x, y) 进行分类



1. 基本概念

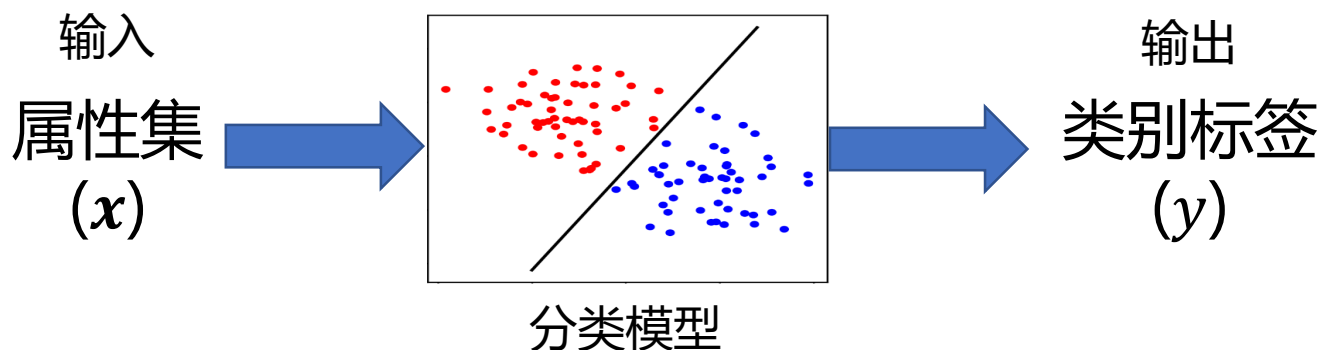
1. 总体思路

■ 分类模型在数据挖掘中担当两个重要角色：

■ **预测模型**(predictive model)：对先前未标记的实例进行分类

■ 一个好的分类模型必须以快速的响应时间提供准确的预测

■ **描述性模型**(descriptive model)：识别区分不同类别实例的特征



1. 基本概念

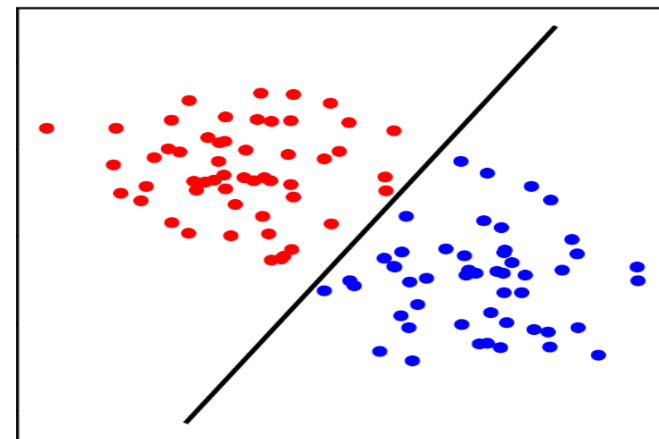
1. 总体思路

- 类别标签必须是标称类型。这将分类与其他预测建模任务(如回归)区分开来。
- 可能并非所有属性都与分类任务相关。寻找区分不同类别实例的最佳属性组合是构建最优分类模型的关键挑战。
- “分类器” 和 “模型” 通常被认为是同义词。

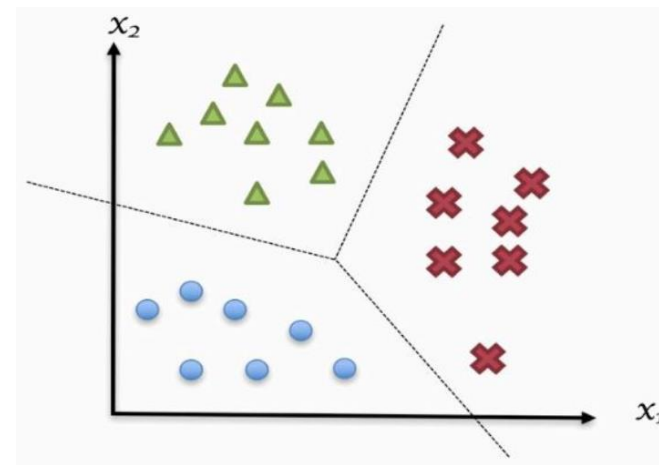
1. 基本概念

2. 分类器的种类

- **二分类器**将每个数据实例分配给**两种可能的标签**中的一种，通常表示为+1(**正类**)和-1(**负类**)
- 与负类相比(例如，电子邮件分类中的垃圾邮件类别)，**正类**通常指的是我们对**正确预测更感兴趣**的类别
- **多分类器**：有**两个以上的可用标签**



二分类器

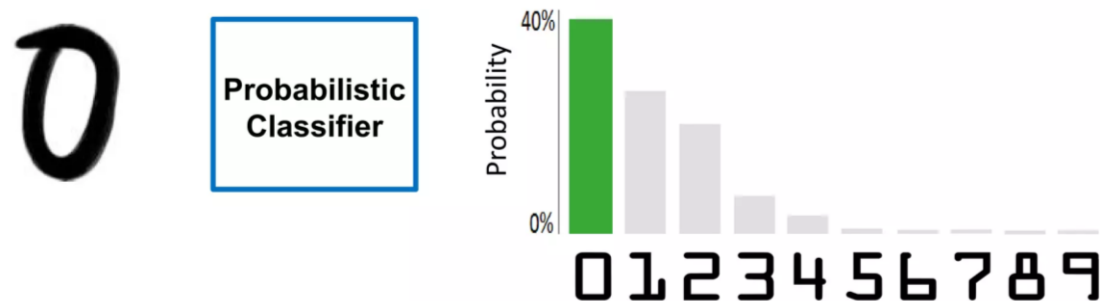


多分类器

1. 基本概念

2. 分类器的种类

- **确定性**分类器：对每个它分类的数据实例产生一个**离散值标签**
- **概率性**分类器：分配一个在**0到1之间的连续的分**数来表示特定标签的**可能性**，各个分类下的概率分数总和为1
 - 提供了更多关于将实例分配到类时的置信度信息
 - 数据实例通常分配给**概率得分最高的类**

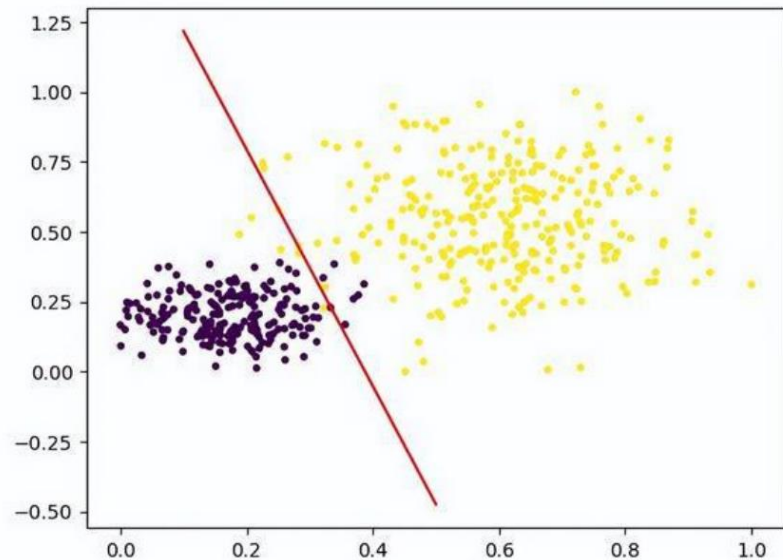


1. 基本概念

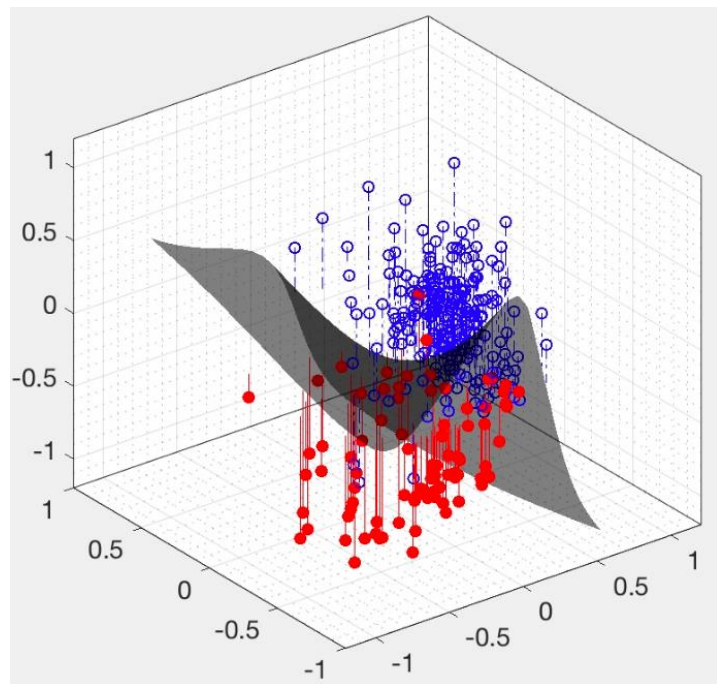
2. 分类器的种类

- 线性分类器：用一个线性分离超平面来区分不同类别中的实例
- 非线性分类器：能够构造更复杂的、非线性的决策表面
- 与非线性分类器相比，线性分类器在拟合复杂数据方面不太灵活，但不易受模型过拟合的影响

线性分类器



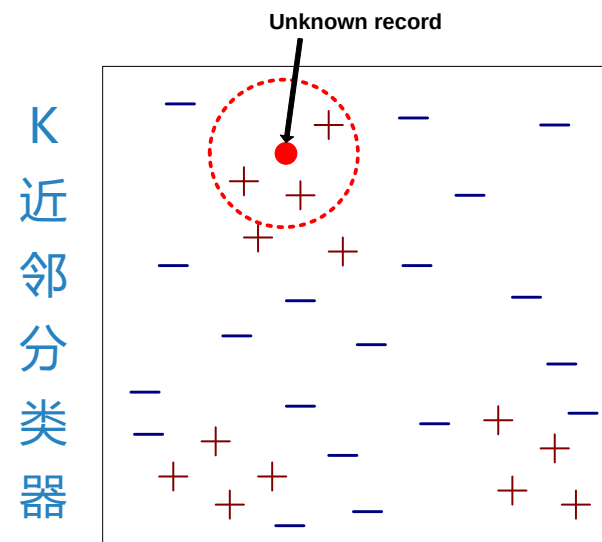
非线性分类器



1. 基本概念

2. 分类器的种类

- 全局分类器：用单个模型拟合整个数据集
- 局部分类器：将输入空间划分为更小的区域，并将不同的模型拟合于每个区域的训练实例
 - K近邻分类器是一个经典例子
- 局部分类器在拟合复杂决策边界时更具灵活性，但它们也更容易受到模型过拟合问题的影响



1. 基本概念

2. 分类器的种类

- 除了预测类别标签之外，我们也可能对生成实例所属标签类别的底层机制感兴趣
- 生成分类器：在预测类别标签的过程中，学习每个类的生成模型
 - 比如：朴素贝叶斯分类器和贝叶斯网络
- 判别分类器：在没有明确描述每个类别分布的情况下，直接预测类别标签
 - 它们不用对数据实例生成机制进行深入研究
 - 比如：决策树、基于规则的分类器、最近邻分类器、人工神经网络和支持向量机等

1. 基本概念

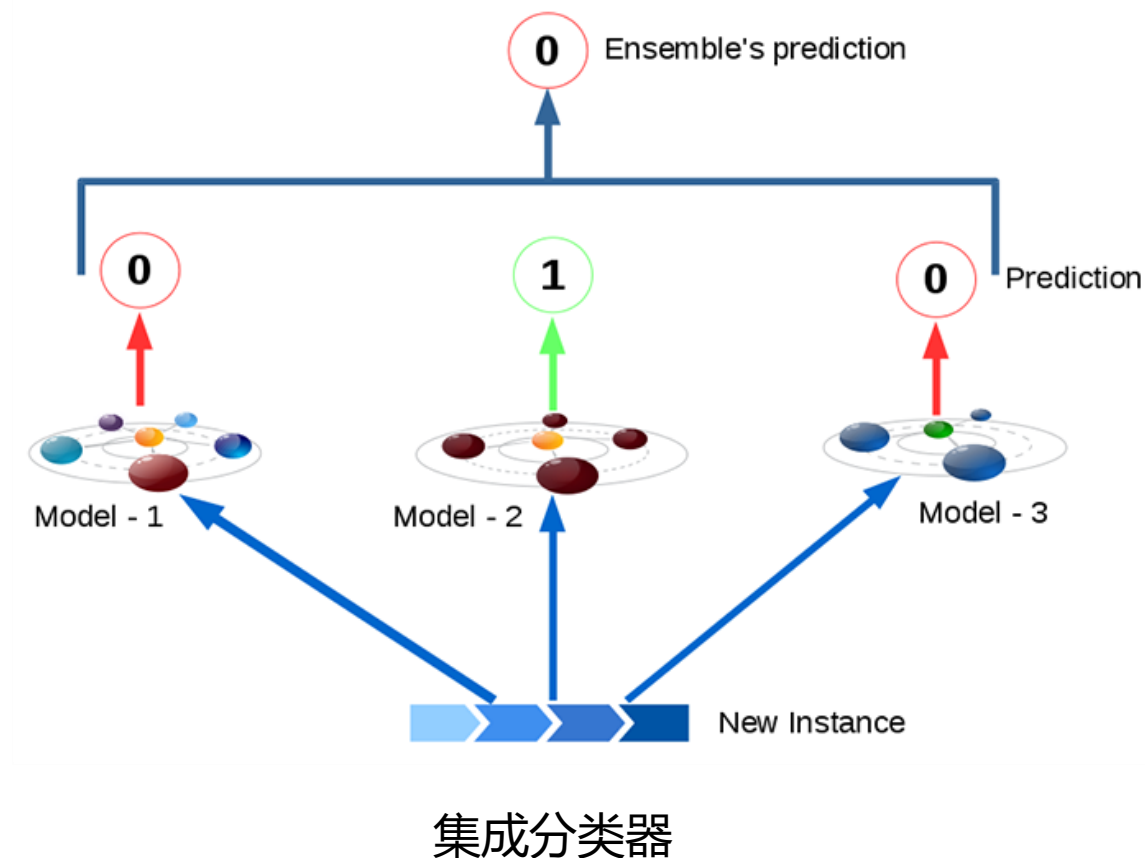
2. 分类器的种类

■ 基分类器 (Base Classifiers)

- 基于决策树的方法
- 基于规则的方法
- 最近邻方法
- 朴素贝叶斯和贝叶斯信念网络
- 支持向量机
- 神经网络, 深度神经网络

■ 集成分类器 (Ensemble Classifiers)

- 提升、装袋、随机森林

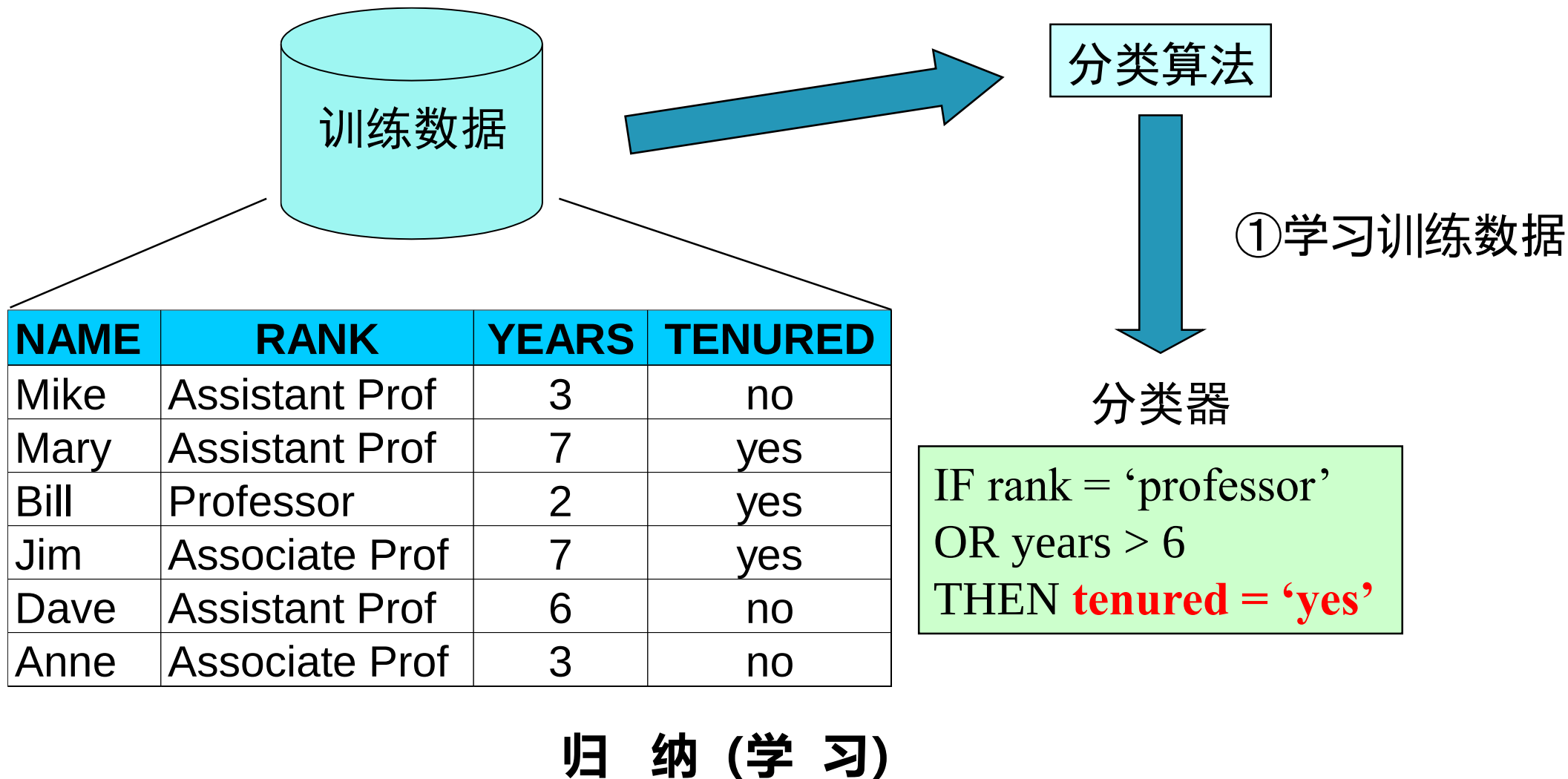


1. 基本概念

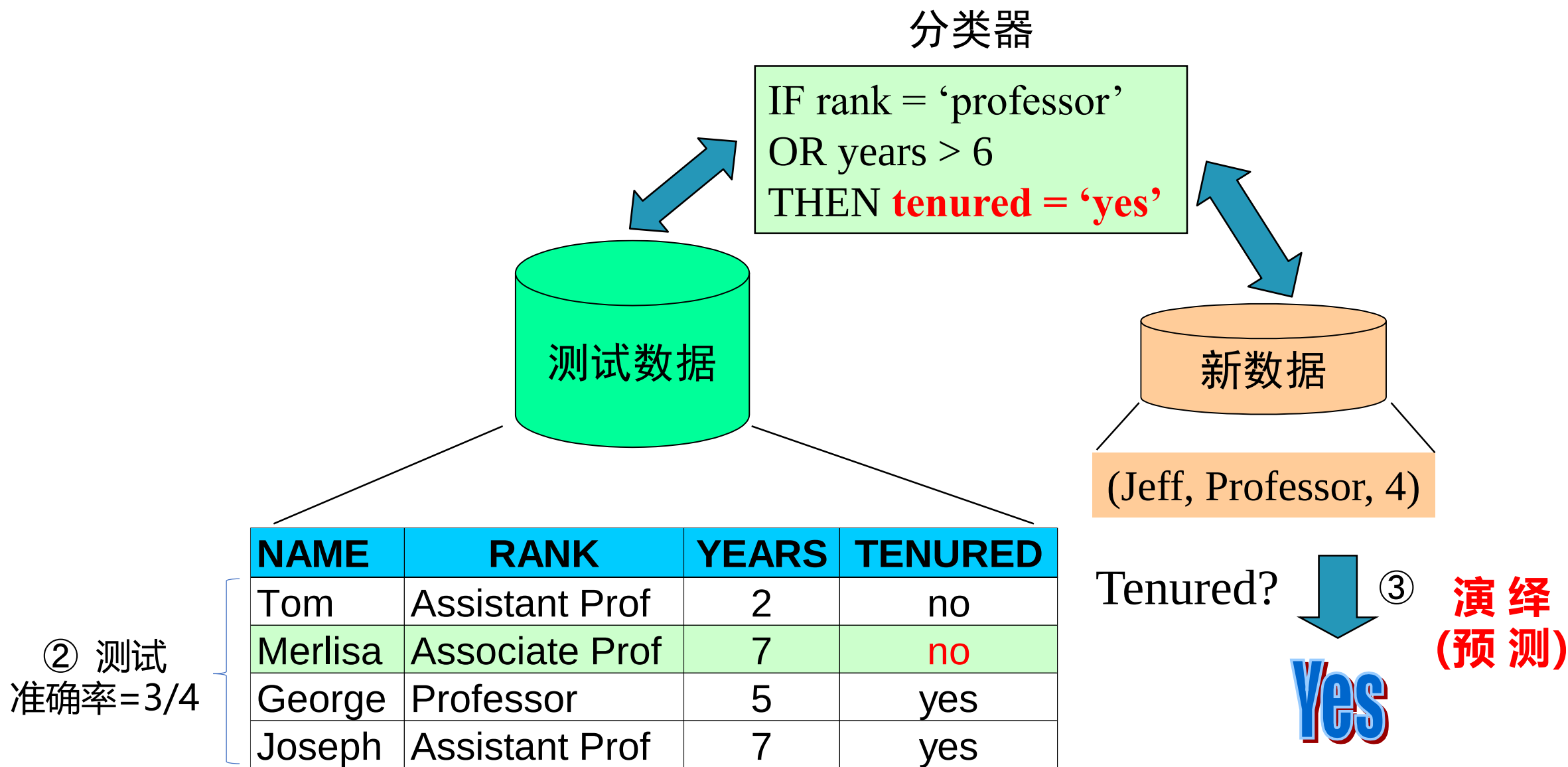
3. 一般的分类框架

- 分类模型是使用给定的一组实例创建的，这组实例称为训练集(training set)，其中包含每个实例的属性值以及类别标签
- 学习算法(learning algorithm)：用于学习给定训练集的分类模型的系统方法
- 归纳(induction)：使用学习算法从训练数据建立分类模型的过程
- 演绎(deduction)：在未知的测试实例上应用分类模型来预测它们的类别标签的过程

1. 基本概念



1. 基本概念



1. 基本概念

3. 一般的分类框架

- 训练集和测试集应该是相互独立的，以确保归纳模型能够准确预测以前从未遇到过的实例的类别标签 --- 具有良好的泛化性能(generalization performance)
- 模型(分类器)的性能可以通过比较实例的预测标签和真实标签来评估。这些信息可以在混淆矩阵(confusion matrix)中总结出来

二分类混淆矩阵

		预测类别	
		类=1	类=0
实际类别	类=1	f_{11}	f_{10}
	类=0	f_{01}	f_{00}

红色：预测正确的实例数

蓝色：预测错误的实例数

1. 基本概念

二分类混淆矩阵

		预测类别	
		类=1	类=0
实际类别	类=1	f_{11}	f_{10}
	类=0	f_{01}	f_{00}

3. 一般的分类框架

■ 评估度量(evaluation metric): 准确率, 错误率

■ 对于二分类问题, 准确率 = $\frac{\text{正确预测的数量}}{\text{总预测数量}} = \frac{f_{00} + f_{11}}{f_{00} + f_{01} + f_{10} + f_{11}}$

■ 对于二分类问题, 错误率 = $\frac{\text{错误预测的数量}}{\text{总预测数量}} = \frac{f_{01} + f_{10}}{f_{00} + f_{01} + f_{10} + f_{11}}$

■ 大多数分类技术的学习算法旨在学习在应用于测试集时达到最高准确度或等效最低错误率的模型

目 录

01

基本概念

02

决策树分类器

03

最近邻分类器

04

朴素贝叶斯分类器

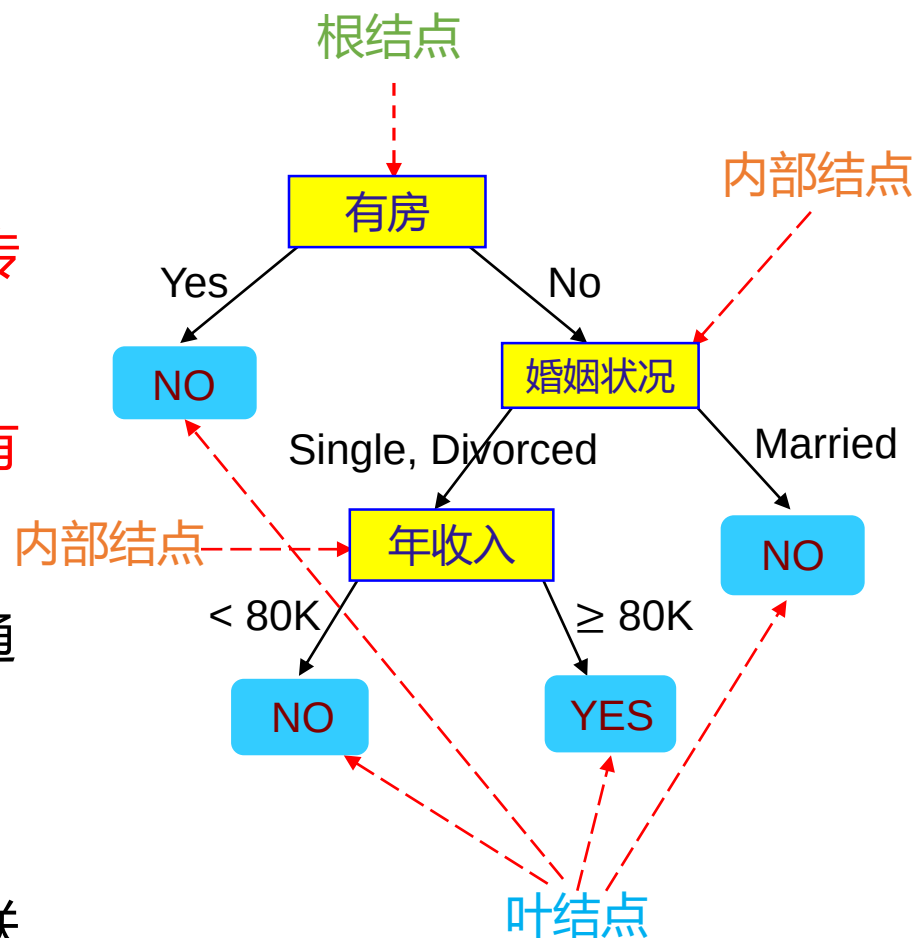
05

支持向量机

2. 决策树分类器

1. 基本概念

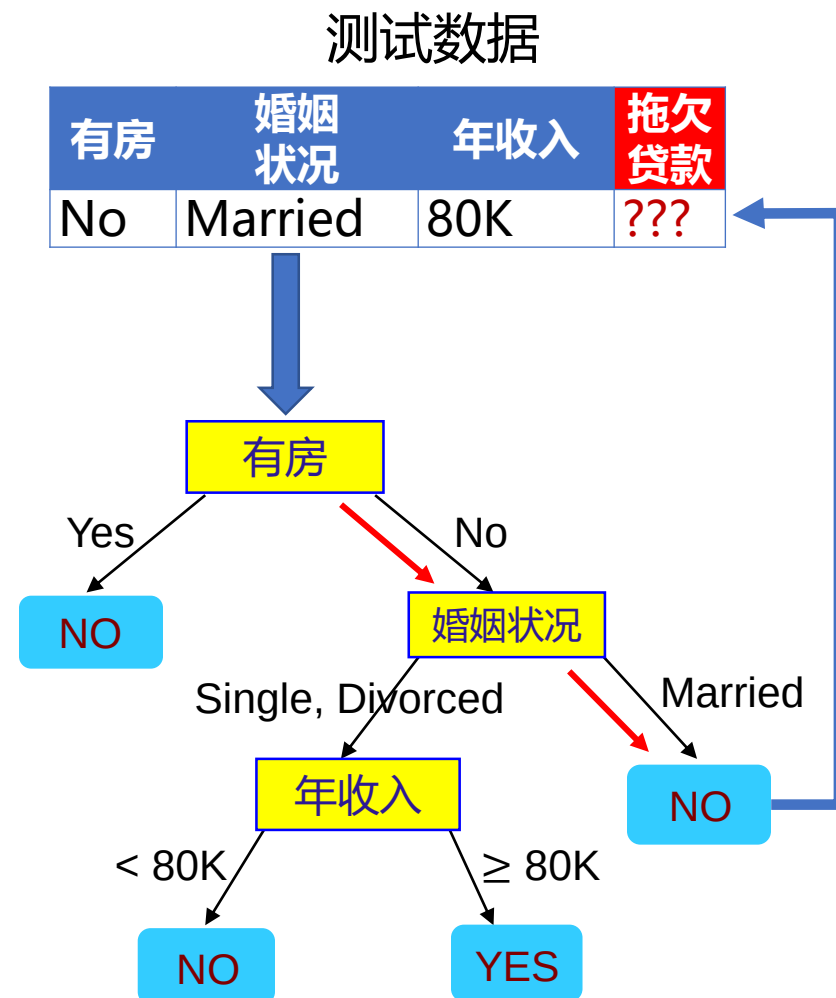
- 根结点：没有传入连接和零个或多个传出连接
- 内部结点：每个结点只有一个传入连接和两个或更多的传出连接
- 叶结点或终端结点：每个结点只有一个传入连接并且没有传出连接，每个叶结点都与一个类别标签相关联
- 非终端(non-terminal)节点：包括根节点和内部节点，通常使用单个属性定义的属性测试条件(attribute test condition)
- 测试条件的每个可能结果都与该结点的一个子结点相关联



2. 决策树分类器

1. 基本概念

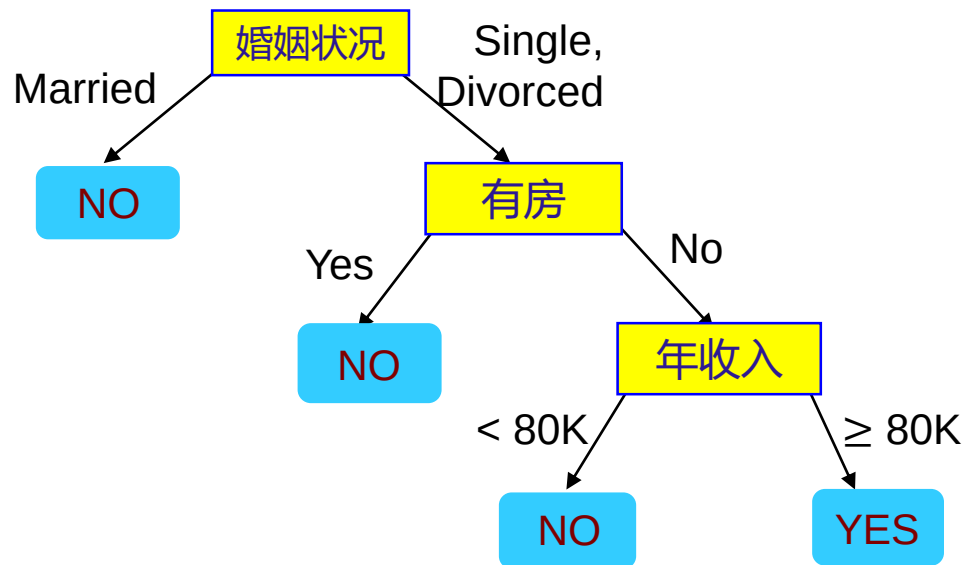
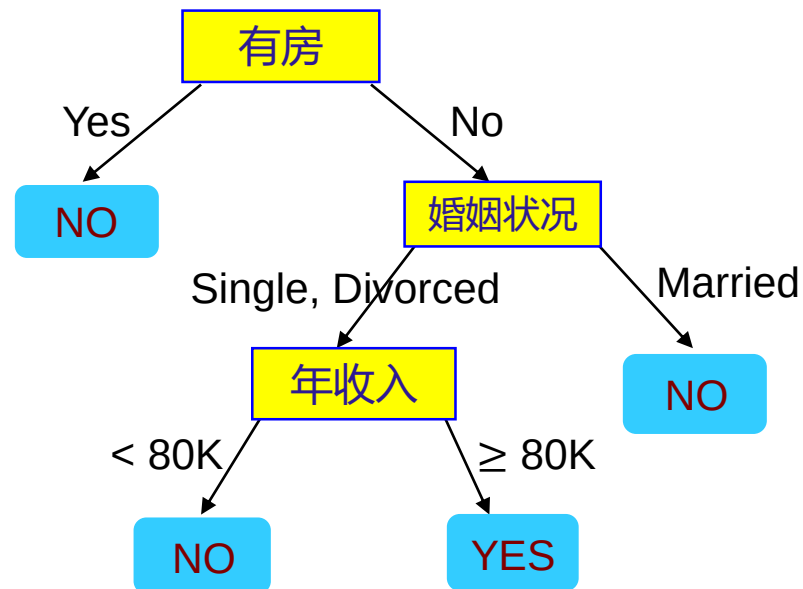
- 给定一个决策树，可以很简单地对测试实例进行分类。
- 从根结点开始，应用其属性测试条件，并根据测试结果按照适当的分支进行操作。
- 这将导致转移到另一个内部结点或叶结点，该结点应用了新属性测试条件。
- 一旦到达叶结点，将与该结点关联的类别标签分配给测试实例。



2. 决策树分类器

2. 构建决策树的基本算法 (Hunt)

- 搜索空间随数据集规模指数级上升，寻找最佳树的代价是昂贵的
- 需要在合理的时间归纳出具有合理准确率的决策树，尽管不是最优的
- 贪心策略：以自顶向下的方式生成决策树，对划分训练数据时要使用的属性进行一系列局部最优决策



2. 决策树分类器

2. 构建决策树的基本算法 (Hunt)

- 决策树是以递归方式生长的
- 该树最初包含一个与所有训练实例关联的根结点
- 若某节点关联的实例来自多个类别，则通过分裂准则(splitting criterion)确定的属性测试条件对其进行扩展
- 为属性测试条件的每个结果创建一个子结点，并根据测试结果将与父结点关联的实例分发给子结点
- 只要子节点仍包含多个类别的标签，就可递归应用此节点扩展步骤
- 若某叶节点关联的所有实例都具有相同类别标签，则终止该节点的扩展。每个叶节点会被分配其关联训练实例中出现频率最高的类别标签

2. 决策树分类器

拖欠贷款

(N:7,Y:3)

(a)

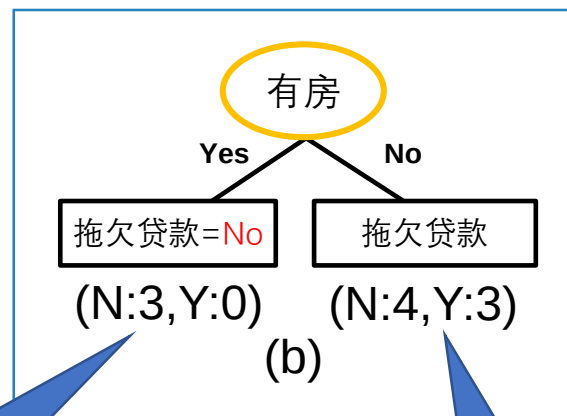
若某节点关联的实例来自多个类别，则通过分裂准则(splitting criterion)确定的属性测试条件对其进行扩展

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器

拖欠贷款
(N:7,Y:3)

(a)



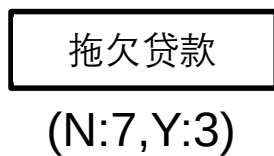
(b)

若某叶节点关联的**所有实例**都具有**相同类别**标签，则终止该节点的扩展。每个叶节点会被分配其关联训练实例中出现**频率最高的类别标签**

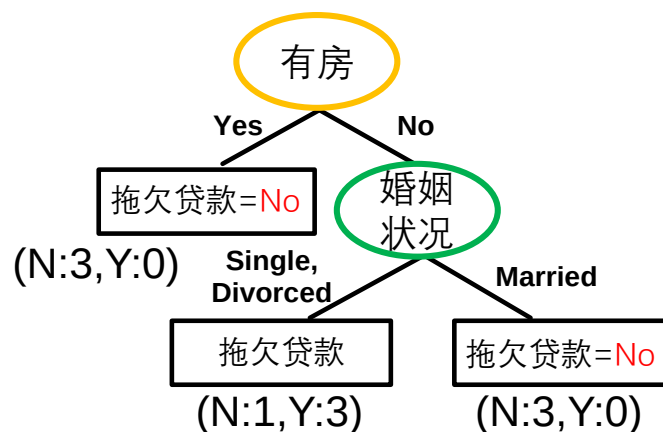
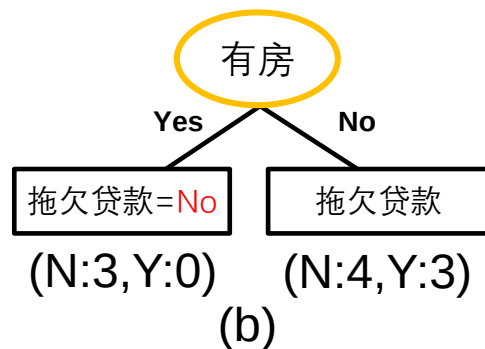
只要子节点仍**包含多个类别**的标签，就可**递归**应用此节点扩展步骤

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器



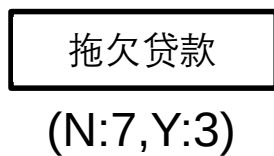
(a)



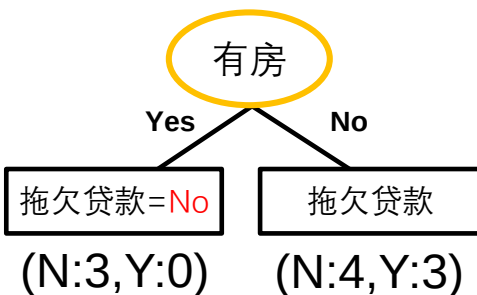
(c)

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

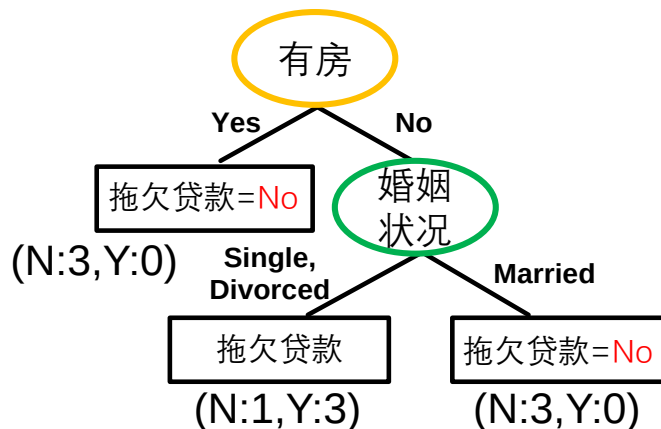
2. 决策树分类器



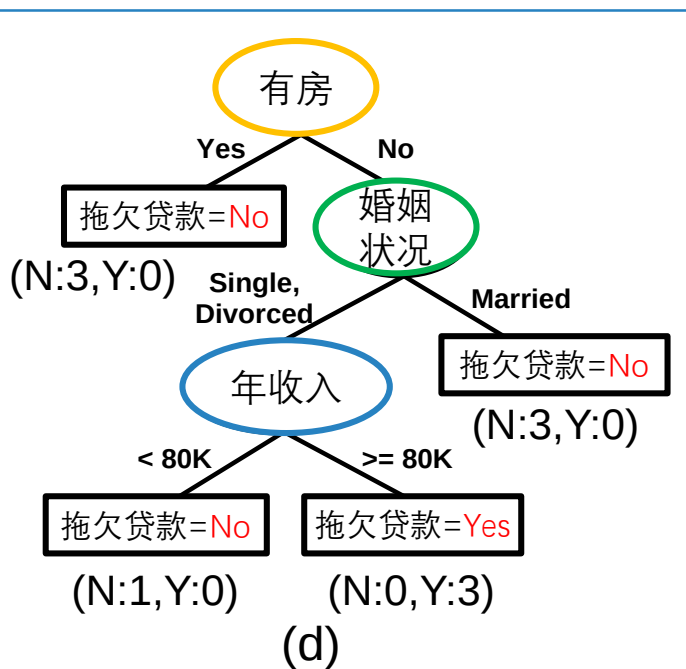
(a)



(b)



(c)



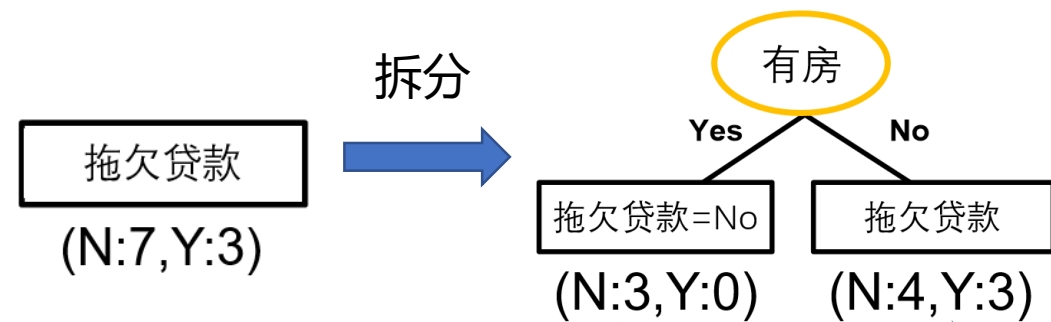
(d)

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器

2. 构建决策树的基本算法 (Hunt)

- 两个关键问题： 拆分标准、终止标准
- 在每次递归中， 必须选择一个属性， 将与结点关联的训练实例划分为与其子结点关联的较小子集。
- 拆分标准： 决定选择哪个属性作为测试条件以及如何将训练实例分配给子结点



2. 决策树分类器

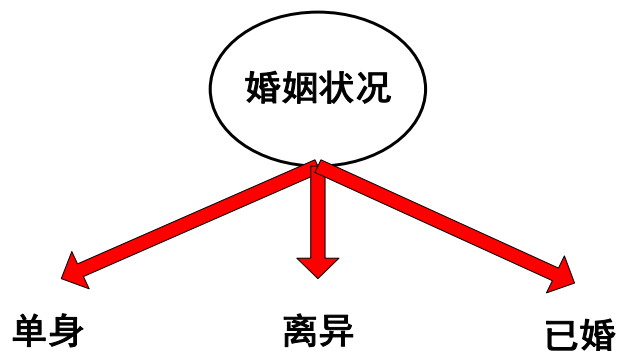
2. 构建决策树的基本算法 (Hunt)

- 两个关键问题：拆分标准、终止标准
- 只有当与结点相关的所有训练实例具有相同的类别标签或具有相同的属性值时，基本算法才会停止扩展结点。
- 如果与一个结点相关的所有训练实例具有相同的属性值，但具有不同的类别标签，为其分配与该结点关联的在训练实例中出现最频繁的类别标签。
- 提前终止：即使叶结点包含来自多个类的训练实例，也有理由更早地终止扩展结点。
- 终止标准：判定是否停止节点扩展的条件

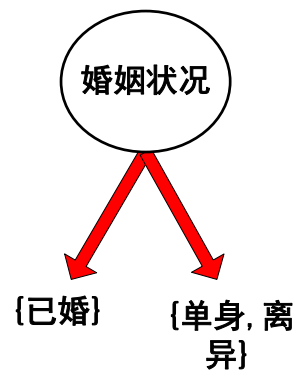
2. 决策树分类器

3. 表示属性测试条件的方法

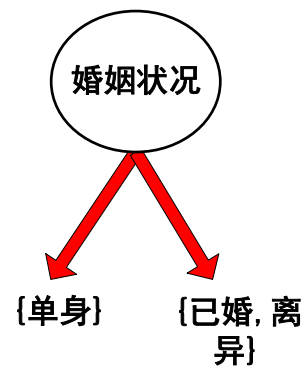
- 二元属性的测试条件产生两个可能的输出
- 标称属性由于标称属性可以有多个属性值，因此它的测试条件可以用两种方式表示：多路划分和二元划分($2^{k-1} - 1$ 种)



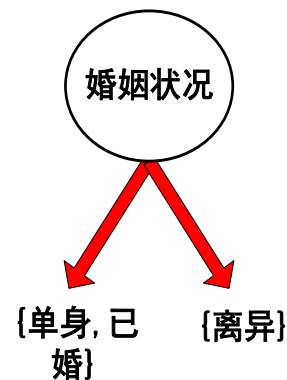
多路划分



OR



OR

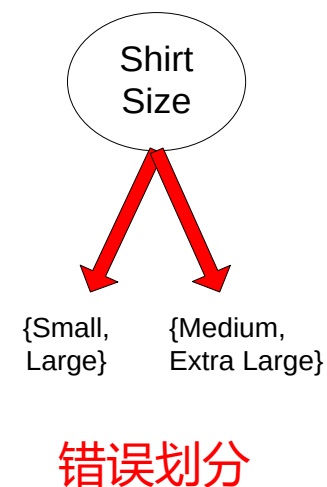
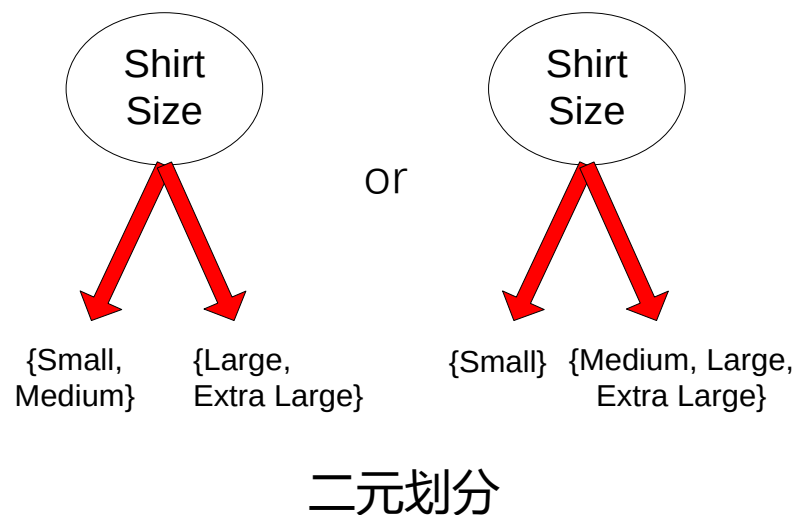
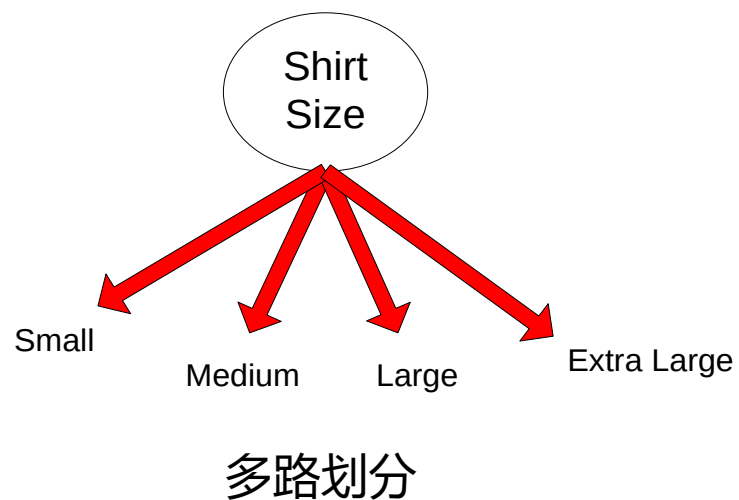


二元划分

2. 决策树分类器

3. 表示属性测试条件的方法

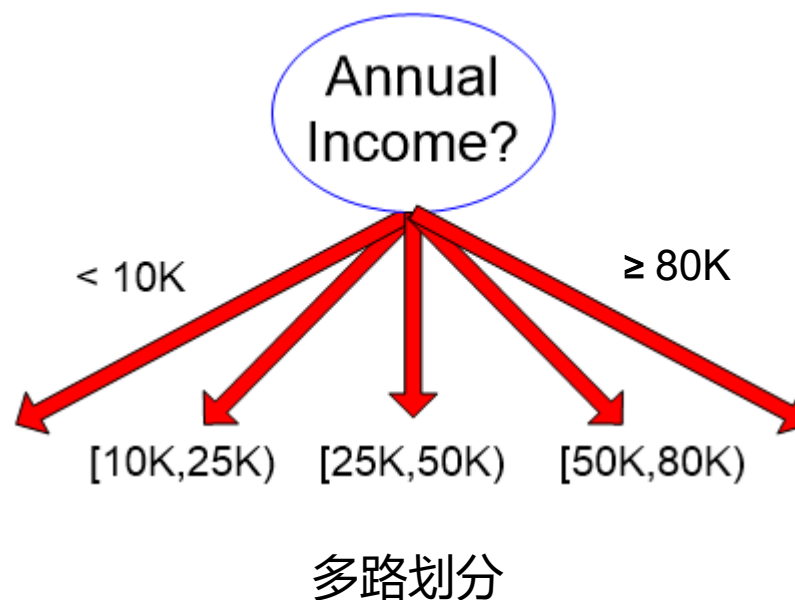
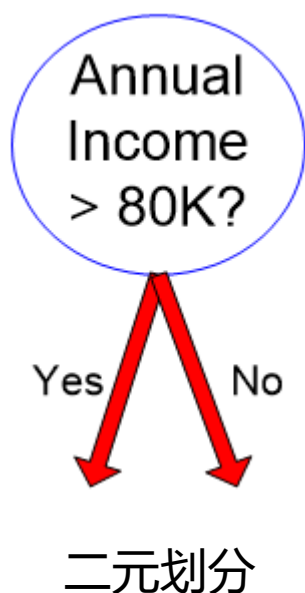
- **序数属性**也可以产生二元或多路划分，只要分组**不违反**属性值的**有序性**就可以对序数属性值进行分组



2. 决策树分类器

3. 表示属性测试条件的方法

- **连续属性**：测试条件可以表示为产生二元划分的比较测试(例如 $A < v$)或表示为用于产生多路划分的形如 $v_i \leq A < v_{i+1} (i = 1, \dots, k)$ 的范围查询



2. 决策树分类器

4. 如何确定最佳划分

■ **贪心方法**：优先考虑将训练实例划分为子节点中更纯净的子集的属性测试条件

■ 结点的**不纯度度量**：度量共有结点的数据实例的类别标签的差异程度

■ 基尼指数：
$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

■ 熵：
$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

■ 分类误差：
$$Classification\ error = 1 - \max[p_i(t)]$$

$p_i(t)$ ：结点 t 属于类 i 的
训练实例的**相对频率**
 c ：类的总个数

2. 决策树分类器

不纯度: $N_1 < N_2 < N_3$

结点 N_1	计数
类=0	0
类=1	6

结点 N_2	计数
类=0	1
类=1	5

结点 N_3	计数
类=0	3
类=1	3

$$Gini\ Index = 1 - \left(\frac{0}{6}\right)^2 - \left(\frac{6}{6}\right)^2 = 0$$

$$Entropy = -\left(\frac{0}{6}\right) \log_2\left(\frac{0}{6}\right) - \left(\frac{6}{6}\right) \log_2\left(\frac{6}{6}\right) = 0$$

$$Classification\ error = 1 - \max\left[\frac{0}{6}, \frac{6}{6}\right] = 0$$

$$Gini\ Index = 1 - \left(\frac{1}{6}\right)^2 - \left(\frac{5}{6}\right)^2 = 0.278$$

$$Entropy = -\left(\frac{1}{6}\right) \log_2\left(\frac{1}{6}\right) - \left(\frac{5}{6}\right) \log_2\left(\frac{5}{6}\right) = 0.650$$

$$Classification\ error = 1 - \max\left[\frac{1}{6}, \frac{5}{6}\right] = 0.167$$

$$Gini\ Index = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 0.5$$

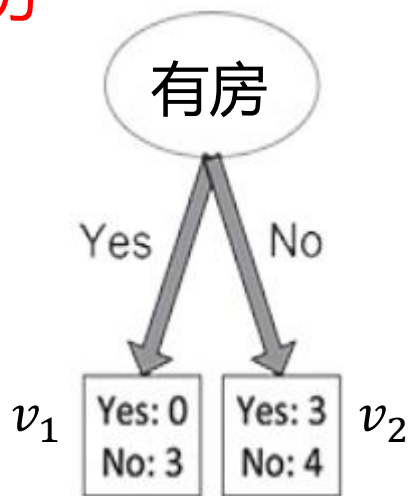
$$Entropy = -\left(\frac{3}{6}\right) \log_2\left(\frac{3}{6}\right) - \left(\frac{3}{6}\right) \log_2\left(\frac{3}{6}\right) = 1$$

$$Classification\ error = 1 - \max\left[\frac{3}{6}, \frac{3}{6}\right] = 0.5$$

2. 决策树分类器

4. 如何确定最佳划分

- 设想一个属性测试条件，该条件将包含 N 个训练实例的结点拆分为 k 个子结点 $\{v_1, v_2, \dots, v_k\}$
- 每个子结点表示由属性测试的 k 个结果之一产生的数据划分



训练实例: 10

属性测试条件: 是否有房

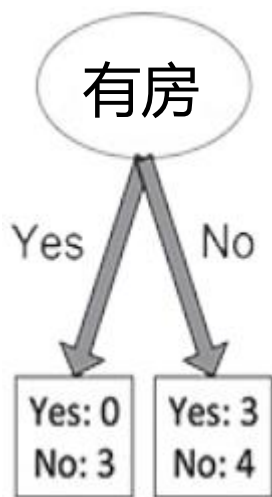
拆分出子节点: v_1, v_2

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器

4. 如何确定最佳划分

- 通过对子结点的不纯度进行加权求和来计算子结点的总不纯度



用属性“有房”进行划分

$$I(\text{子结点}) = \sum_{j=1}^k \frac{N(v_j)}{N} I(v_j)$$

$$I(\text{有房} = \text{Yes}) = -\left(\frac{0}{3}\right) \log_2\left(\frac{0}{3}\right) - \left(\frac{3}{3}\right) \log_2\left(\frac{3}{3}\right) = 0$$

$$I(\text{有房} = \text{No}) = -\left(\frac{3}{7}\right) \log_2\left(\frac{3}{7}\right) - \left(\frac{4}{7}\right) \log_2\left(\frac{4}{7}\right) = 0.985$$

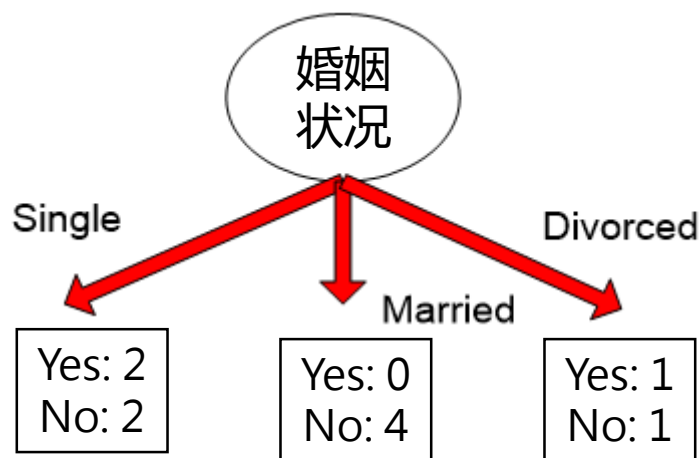
$$I(\text{有房}) = \left(\frac{3}{10}\right) * 0 + \left(\frac{7}{10}\right) * 0.985 = \mathbf{0.690}$$

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器

4. 如何确定最佳划分

- 通过对子结点的不纯度进行加权求和来计算子结点的总不纯度



用属性“婚姻状况”进行划分

$$I(\text{子结点}) = \sum_{j=1}^k \frac{N(v_j)}{N} I(v_j)$$

$$I(\text{婚姻} = \text{Single}) = -\left(\frac{2}{4}\right) \log_2\left(\frac{2}{4}\right) - \left(\frac{2}{4}\right) \log_2\left(\frac{2}{4}\right) = 1$$

$$I(\text{婚姻} = \text{Married}) = -\left(\frac{0}{4}\right) \log_2\left(\frac{0}{4}\right) - \left(\frac{4}{4}\right) \log_2\left(\frac{4}{4}\right) = 0$$

$$I(\text{婚姻} = \text{Divorced}) = -\left(\frac{1}{2}\right) \log_2\left(\frac{1}{2}\right) - \left(\frac{1}{2}\right) \log_2\left(\frac{1}{2}\right) = 1$$

$$I(\text{婚姻}) = \left(\frac{4}{10}\right) * 1 + \left(\frac{4}{10}\right) * 0 + \left(\frac{2}{10}\right) * 1 = \mathbf{0.6}$$

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. 决策树分类器

$I(\text{有房}) > I(\text{婚姻})$, 用“**婚姻状况**”来划分较好

4. 如何确定最佳划分

- 为了评估属性测试条件的优劣, 我们需要比较**父节点** (分裂前) 的**不纯度** 与**子节点** (分裂后) 的**加权不纯度**
- 它们的**差异越大**, 测试条件**越好**
- 这种差异(Δ)也称为属性测试条件的**纯度增益(gain)**

$$\Delta = I(\text{父结点}) - I(\text{子结点})$$

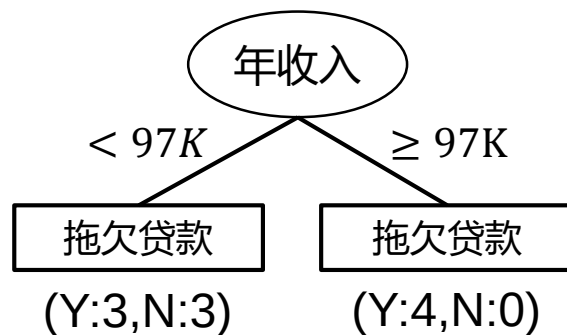
- 在给定结点处最大化增益相当于**最小化其子项的加权不纯度度量**, 因为对于所有候选属性测试条件, **$I(\text{父结点})$ 是相同的**
- 当**熵**用作不纯度度量时, 通常称为**信息增益(information gain)** --- Δ_{info}

2. 决策树分类器

4. 如何确定最佳划分

- 例如：考虑定量属性年收入的确定最佳二元划分 “年收入 $< \tau$ ” 的问题
- 尽管 τ 可以在训练集中年收入的最小值和最大值之间取任意值，但只需将训练集中观察到的年收入值视为候选划分位置就足够了。
- 对于每个候选，训练集被扫描一次以计算年收入小于或大于 τ 的借款人数量及其类别比例。
- 然后，计算每个候选划分位置的不纯度度量，并选择产生最小值的划分位置。
- 在每个候选划分位置计算不纯度度量需要 $O(N)$ 次操作，其中 N 是训练实例的数量。由于最多有 N 个候选，所以该暴力法的计算复杂度为 $O(N^2)$ 。

2. 决策树分类器



基尼指数 $Gini Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$

$$g_1 = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = \frac{1}{2}$$

$$g_2 = 1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 = 0$$

$$\left(\frac{6}{10}\right) * \frac{1}{2} + \left(\frac{4}{10}\right) * 0 = 0.3$$

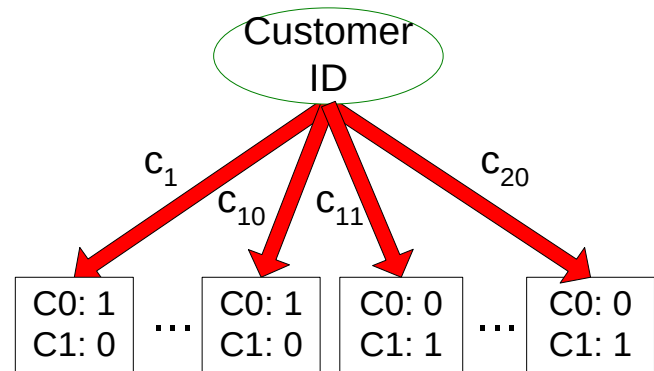
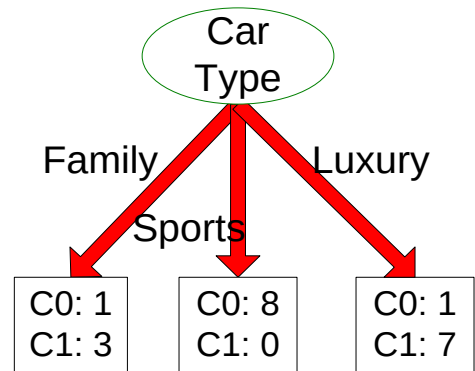
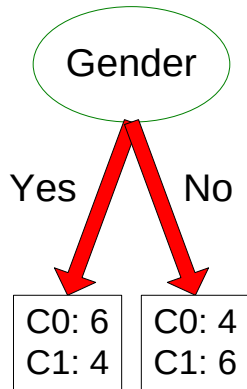
4. 如何确定最佳划分

- 可以将此问题的计算复杂度降至 $O(N \log N)$
- 首先根据年收入将训练实例排序，所需时间为 $O(N \log N)$ 。
- 从两个相邻的排过序的属性值中选择中间值作为候选划分点。
- 对于每一个候选，计算不纯度度量， $O(1)$ 。最后，找到最佳划分位置， $O(N)$ 。

类	Annual Income																			
	排序后的值																			
	划分点																			
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1
Gini	0.420	0.400	0.375	0.343	0.417	0.400	0.300	0.343	0.375	0.400	0.420	0.420	0.420	0.420	0.420	0.420	0.420	0.420	0.420	0.420

2. 决策树分类器

5. 增益率



- 熵和基尼指数等不纯度度量存在一个**潜在的局限**，即它们更容易选择具有**大量不同值的定性属性**
- **单一的低不纯度结点**不足以找到良好的属性测试条件
- **子结点越多，决策树越复杂**，更容易出现**过拟合**
- 在决定最佳属性测试条件时，**考虑**划分属性产生的**子结点数量**
 - 一种方法是**仅生成二元决策树**，从而**避免使用不同数量的划分**来处理属性(比如CART)
 - 另一种方法是修改划分标准以**考虑属性生成的划分数**量(比如C4.5)

CustomerID具有最高的信息增益，因为**所有子项的熵都为零**

2. 决策树分类器

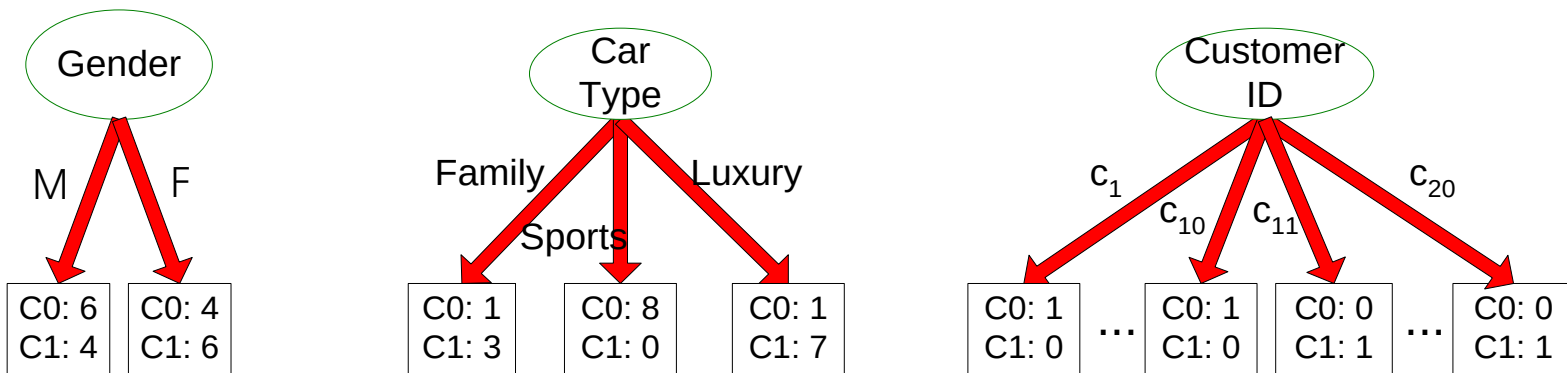
5. 增益率

$$\text{Gain Ratio} = \frac{\Delta_{info}}{\text{Split Info}} \quad \text{Split Info} = - \sum_{i=1}^k \frac{n_i}{n} \log_2 \frac{n_i}{n}$$

- 分裂信息(split information)通过计算节点分裂为子节点时的熵，来评估该拆分是否倾向于生成更多大小相近的子节点
- 这个公式和熵的公式一模一样，只不过这里的“类别”变成了子节点本身，而“概率”变成了每个子节点所含样本的比例
 - **分裂信息值大**：当所有子节点的大小都差不多时（ $\frac{n_i}{n}$ 都相近），计算得到的分裂信息值会很大。这意味着这次拆分非常“平衡”，将数据均匀地分散到了各个分支。
 - **分裂信息值小**：当拆分产生一个特别大的子节点和几个极小的子节点时（即数据分布极不平衡），分裂信息值会很小。
- 一个特征如果产生很多分支，其分裂信息通常也会很大，从而降低了增益率(Gain Ratio)。

2. 决策树分类器

$$\text{Gain Ratio} = \frac{I(\text{父结点}) - I(\text{子结点})}{\text{Split Info}}$$



Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

$$I(\text{父结点}) = -\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20} = 1$$

性别:  $I(\text{子结点}) = \frac{10}{20}(-\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10}) * 2 = 0.971$

$\text{Split Info} = -\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20} = 1$

$\text{Gain Ratio} = \frac{1-0.971}{1} = \mathbf{0.029}$

车型:  $I(\text{子结点})$

$= \frac{4}{20}(-\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4}) + \frac{8}{20} * 0 + \frac{8}{20}(-\frac{1}{8} \log_2 \frac{1}{8} - \frac{7}{8} \log_2 \frac{7}{8}) = 0.380$

$\text{Split Info} = -\frac{4}{20} \log_2 \frac{4}{20} - \frac{8}{20} \log_2 \frac{8}{20} - \frac{8}{20} \log_2 \frac{8}{20} = 1.52$

$\text{Gain Ratio} = \frac{1-0.38}{1.52} = \mathbf{0.41}$

顾客ID:  $I(\text{子结点}) = \frac{1}{20}(-\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1}) * 20 = 0$

$\text{Split Info} = (-\frac{1}{20} \log_2 \frac{1}{20}) * 20 = 4.32$

$\text{Gain Ratio} = \frac{1-0}{4.32} = \mathbf{0.23}$

2. 决策树分类器

6. 决策树归纳算法

- 算法的输入是一组训练实例集 E 和属性集 F
- `createNode()`函数通过创建一个新结点来扩展决策树
- `Classify()`函数确定要分配给叶结点的类别标签
- `find_best_split()`函数确定应当选择哪个属性作为划分训练实例的测试条件
- `stopping_cond()`函数通过检查所有实例是否具有相同的类别标签, 或相同的属性值来决定是否终止决策树的生长

TreeGrowth(E, F)

```
1:  if stopping_cond( $E, F$ ) = true then
2:       $leaf$  = createNode()
3:       $leaf.label$  = Classify( $E$ )
4:      return  $leaf$ 
5:  else
6:       $root$  = createNode()
7:       $root.test\_cond$  = find_best_split( $E, F$ )
8:      let  $V = \{ v \mid v \text{ is a possible outcome of } root.test\_cond \}$ 
9:      for each  $v \in V$  do
10:          $E_v = \{ e \mid root.test\_cond(e) = v \text{ and } e \in E \}$ 
11:          $child$  = TreeGrowth( $E_v, F$ )
12:         add  $child$  as descendent of  $root$  and label the
            edge ( $root \rightarrow child$ ) as  $v$ 
13:      end for
14:  end if
15:  return  $root$ 
```

2. 决策树分类器

7. 决策树分类器的特点 --- 适用性

- 决策树是构建分类模型的非参数化方法。
- 不需要任何关于数据的类别和属性概率分布的先验假设，因此适用于各种各样的数据集。
- 适用于连续可分类数据，而不需要通过二值化、标准化或规范化将属性转换为通用表示。
- 可以处理多分类问题，不需要将它们分解为多个二分类任务
- 小型树易于理解
- 对许多简单的数据集，树的准确率也与其他分类技术相当

2. 决策树分类器

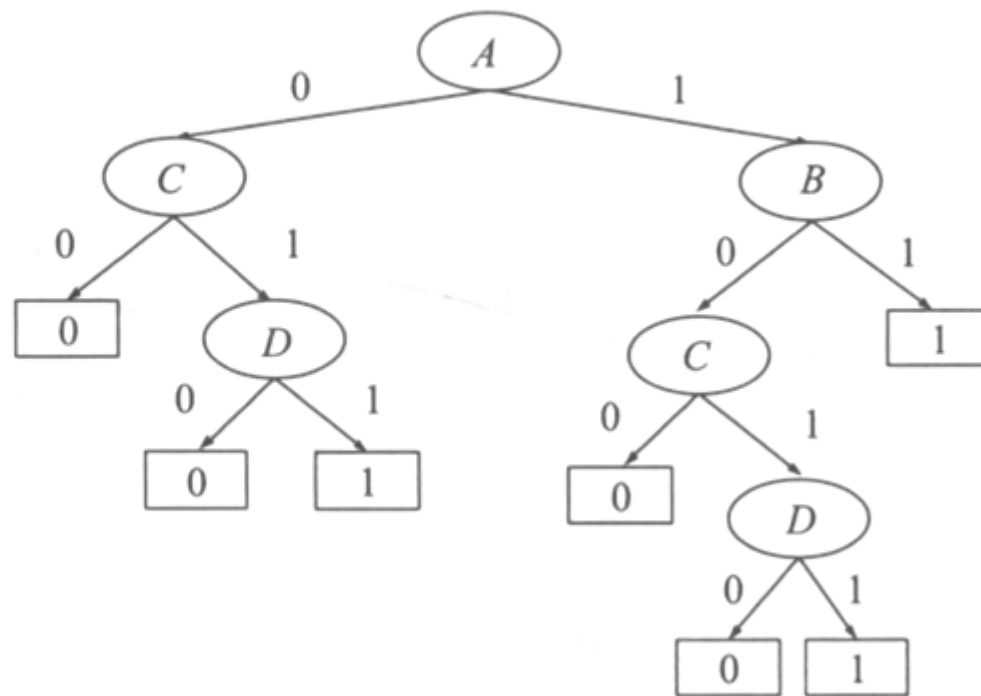
7. 决策树分类器的特点 --- 表达能力

- 决策树为离散值函数提供了一个通用表示，它可以编码任何离散值属性的函数
- 每个离散值函数都可以表示为一个赋值表，其中每个离散属性的唯一组合都被赋予一个类别标签
- 由于每个属性组合可以表示为决策树中的一个叶结点，我们总能找到一个决策树，其叶结点处的标签分配与原始函数的分配表相匹配

2. 决策树分类器

7. 决策树分类器的特点 --- 表达能力

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	类
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1



布尔函数 $(A \wedge B) \vee (C \wedge D)$ 的决策树

2. 决策树分类器

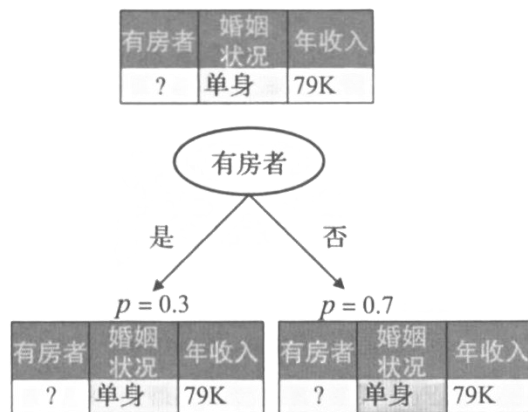
7. 决策树分类器的特点 --- 计算效率

- 由于可能的决策树数量可能非常庞大，因此许多决策树算法采用基于启发式的方法来指导它们在广阔的假设空间中进行搜索
 - 使用自顶向下的贪心递归划分策略
- 一旦构建了决策树，可迅速对测试记录进行分类，最坏情况下的复杂度为 $O(\omega)$ ，其中 ω 是树的最大深度

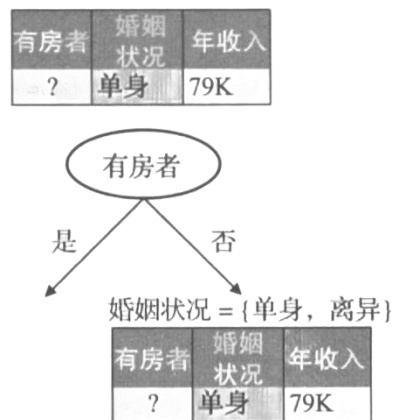
2. 决策树分类器

7. 决策树分类器的特点 --- 处理缺失值

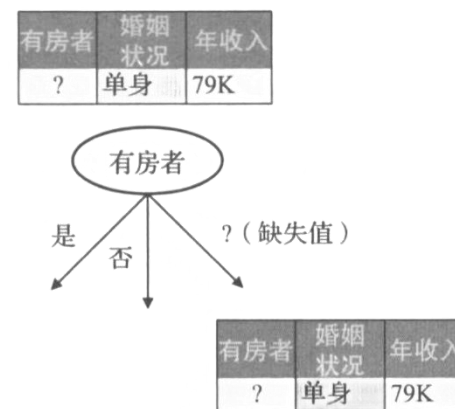
- 当测试集有缺失值时，如果给定的测试实例缺少划分结点属性的值，则分类器必须决定遵循哪个分支
- C4.5决策树分类器采用概率划分法 --- 根据缺失属性具有特定值的概率
- CART算法使用替代划分法 --- 根据另一个未缺失的替代属性值将其分配到某个子节点
- CHAID算法使用独立类法，将缺失值视为与划分属性的其他值不同的单独分类值



概率划分法



替代划分法

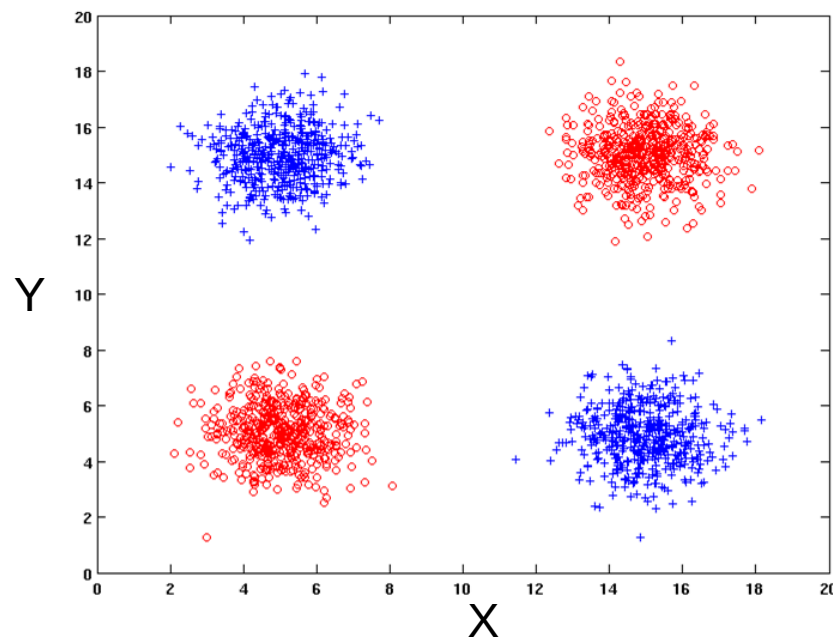
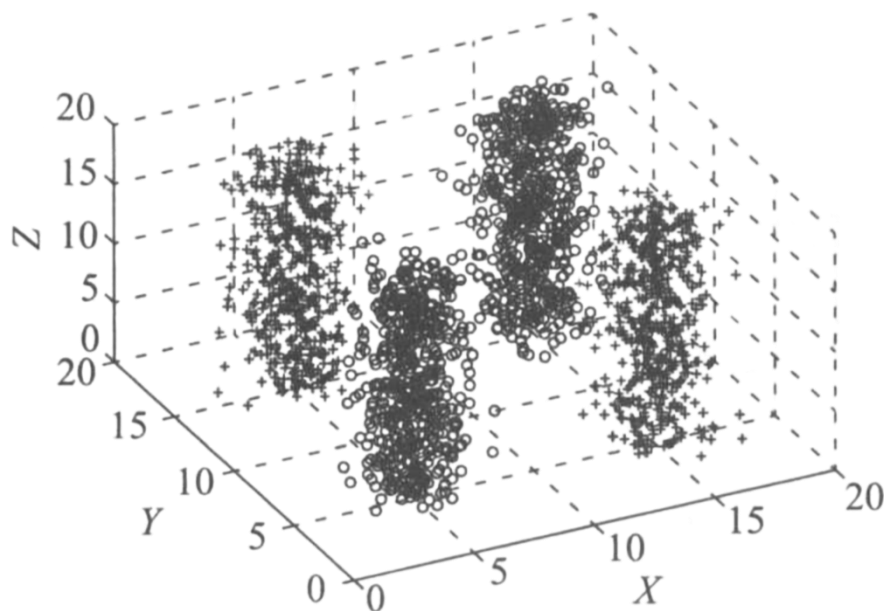


独立类法

2. 决策树分类器

7. 决策树分类器的特点 --- 属性之间的相互作用

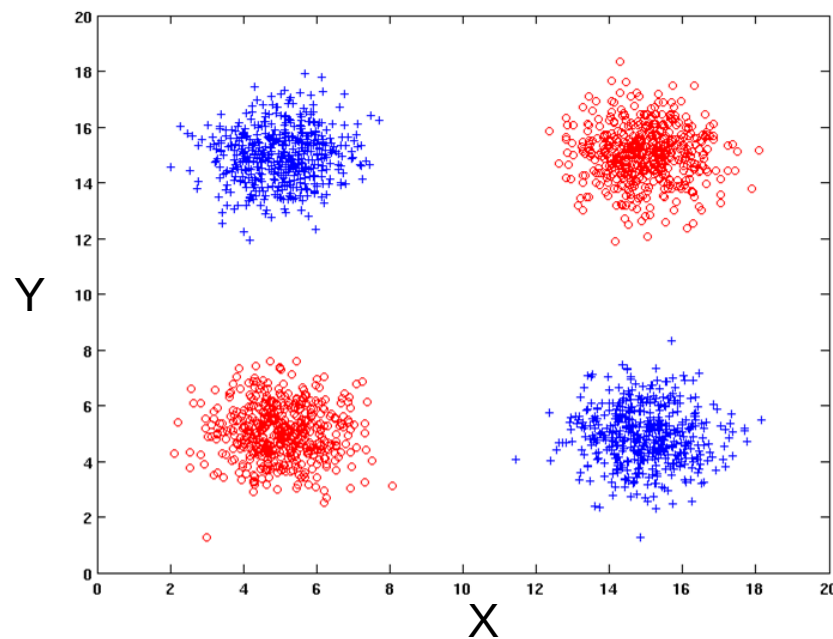
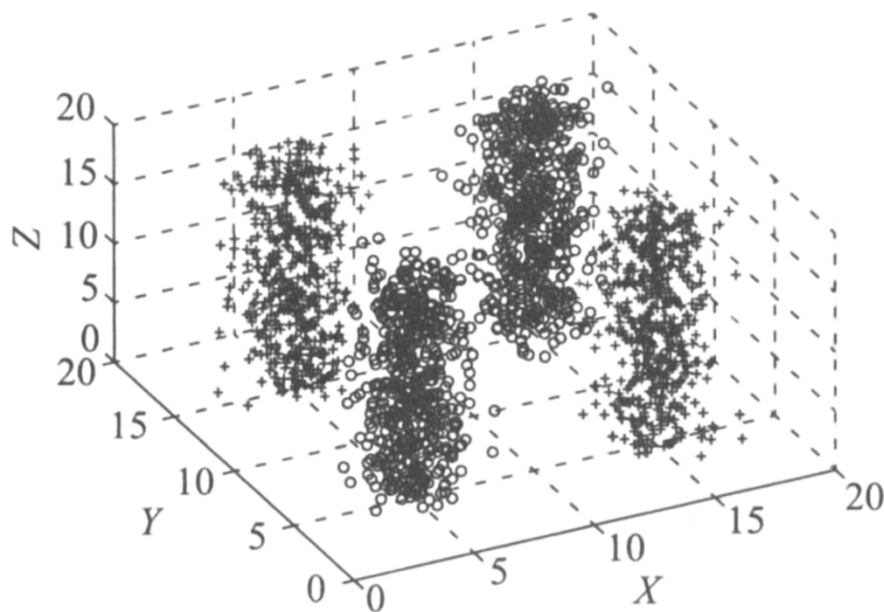
- 属性被认为是相互作用的：一起使用时能够区分类别，但是它们单独使用只能提供很少或不提供信息
- 当属性之间存在相互作用时，决策树可能表现不佳，比如属性 X 和 Y



2. 决策树分类器

7. 决策树分类器的特点 --- 处理不相关的属性

- 如果属性对分类任务无用，则属性无关紧要，比如属性 Z
- 特征选择技术可以通过预处理过程消除不相关的属性来帮助提高决策树的准确性



2. 决策树分类器

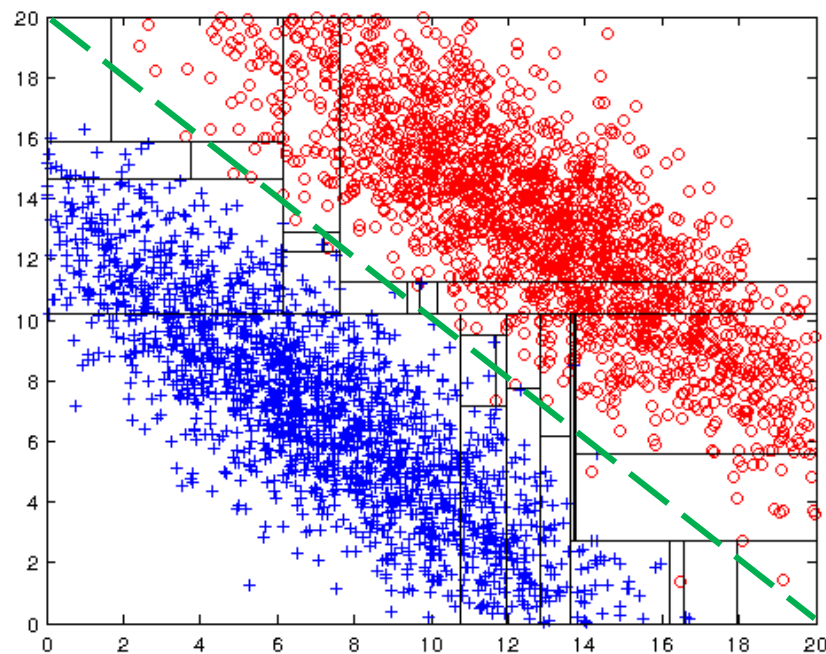
7. 决策树分类器的特点 --- 处理冗余属性

- 如果属性与数据中的另一个属性**强相关**，则该属性是**冗余的**
- 冗余属性在被选择用于划分时显示出**相似的纯度增益**，只有其中**一个属性被选为属性测试条件**
- 决策树可以处理冗余属性的存在

2. 决策树分类器

7. 决策树分类器的特点 --- 使用直线分割

- 不同类别的两个相邻区域之间的边界称为**决策边界**(decision boundary)
- 测试条件一次**只涉及单个属性**，因此决策边界是直线，即**平行于坐标轴**
- 限制了决策树在表示具有**连续属性**数据集的决策边界时的**表达能力**
- **倾斜决策树**(oblique decision tree)可以通过允许**使用多个属性**指定测试条件来克服这个限制



— — — $x + y = 20$

2. 决策树分类器

8. 三种常见决策树算法

算法	支持模型	树结构	划分特征选择	连续值处理	缺失值处理	剪枝	特征属性多次使用
ID3	分类	多叉树	信息增益	不支持	不支持	不支持	不支持
C4.5	分类	多叉树	信息增益率	支持	支持	支持	不支持
CART	分类、回归	二叉树	基尼指数、均方差	支持	支持	支持	支持

目 录

01

基本概念

02

决策树分类器

03

最近邻分类器

04

朴素贝叶斯分类器

05

支持向量机

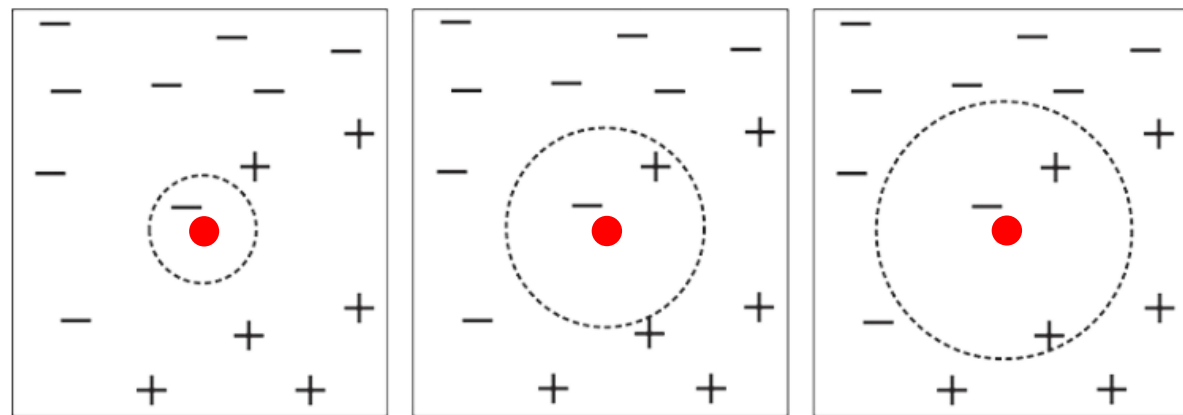
3. 最近邻分类器

1. 基本概念

- 分类框架涉及两个步骤: (1)用于从数据构造分类模型的归纳步骤;
(2)将该模型应用于测试实例的演绎步骤
- 决策树分类器是渴望学习器(eager learner)的例子
 - 在训练数据可用后立即学习一个将输入属性映射到类标签的模型
- 懒惰学习器(lazy learner): 延迟对训练数据建模的过程, 直到需要对测试实例进行分类

3. 最近邻分类器

1. 基本概念

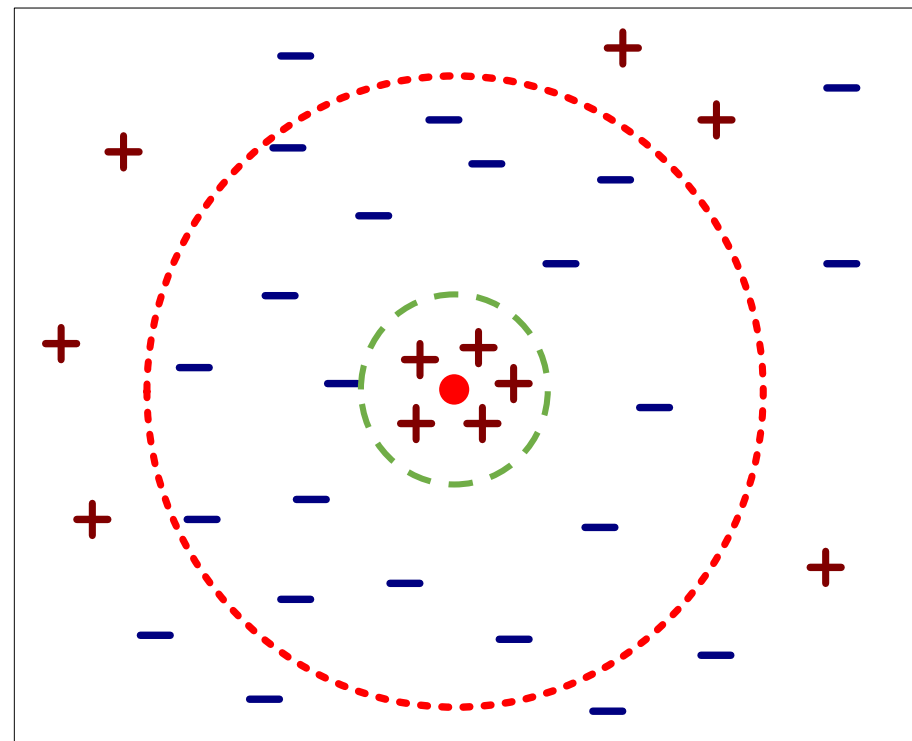


- 最近邻(nearest neighbor): 找到与测试实例的属性相对相似的所有训练实例, 用来确定测试实例的类别标签
- 最近邻分类器将每个样本表示为 d 维空间中的数据点, 其中 d 是属性数目
- 给定一个测试实例, 根据某种邻近度度量方法计算它与训练实例的接近程度
- 给定测试实例 z 的 k 个最近邻 (k -nearest neighbors), 指的是训练样本中与 z 最接近的 k 个样本
- 该实例的类别由其邻近样本的类别标签决定。若邻近样本存在多个类别标签, 则将该测试实例归类为其最近邻样本中的多数类别
- 若出现类别得票数相同的情况, 则可随机选择其中一个类别作为该数据点的最终分类

3. 最近邻分类器

1. 基本概念

- 如果 k 太小，那么最近邻分类器容易因训练数据中的噪声（如错误标注样本）而产生过拟合
- 如果 k 太大，则可能因近邻列表中包含过多远离测试实例的训练样本而导致分类错误



3. 最近邻分类器

2. 算法

- 在多数表决(majority voting)方法中, 每个邻居对分类具有相同的影响
- 为了减少 k 的影响的方法之一是, 根据距离权衡每个最近邻的影响
- 使用距离加权 $w_i = \frac{1}{d(\mathbf{x}', \mathbf{x}_i)^2}$ 投票方案, 距离 z 较远的训练样本对分类的影响要小于距离 z 较近的

v 是类别标签, y_i 是最近邻样本之一的类别标签

k -最近邻分类算法

- 1: 令 k 是最近邻数目, D 是训练样本的集合
- 2: **for** 每个测试实例 $z = (\mathbf{x}', y')$ **do**
- 3: 计算 z 和每个样本 $(\mathbf{x}, y) \in D$ 之间的距离 $d(\mathbf{x}', \mathbf{x})$
- 4: 选择离 z 最近的 k 个训练样本的集合 $D_z \subseteq D$
- 5: $y' = \underset{v}{\operatorname{argmax}} \sum_{(x_i, y_i) \in D_z} w_i \times I(v = y_i)$ /* $I(\cdot)$ 是一个指示符函数*/
- 6: **end for**

3. 最近邻分类器

3. 最近邻分类器的特点

- 最近邻分类器是以**实例学习**而闻名的一种通用技术
- 需要用**邻近度量**来确定**实例之间**的相似性或距离，以及基于测试实例与其他实例的**邻近度**返回其预测类别的分类函数
- 懒惰学习器，**无须建模**；需要**逐一计算**测试样本与训练样本之间的**邻近度**，其分类过程可能**消耗较大**计算成本
- 最近邻分类器**基于局部信息**进行预测(这相当于为每个测试实例建立**局部模型**)；**决策树分类器**试图找到适合整个输入空间的**全局模型**；最近邻分类器(具有较小的 k 值)**对噪声非常敏感**

3. 最近邻分类器

3. 最近邻分类器的特点

- 最近邻分类器可以产生任意形状的决策边界；决策树分类器通常被约束到直线决策边界
- 最近邻分类器难以处理训练集和测试集中的缺失值，因为邻近度计算通常需要所有属性存在
- 不相关属性的存在会扭曲常用的邻近度量
 - 如果存在大量彼此高度相关的冗余属性，那么邻近度量可能过度偏向于这些属性，从而导致不正确的距离估计
- 除非采取适当的邻近度量和数据预处理步骤（比如标准化），否则最近邻分类器可能产生错误的预测

目 录

01

基本概念

02

决策树分类器

03

最近邻分类器

04

朴素贝叶斯分类器

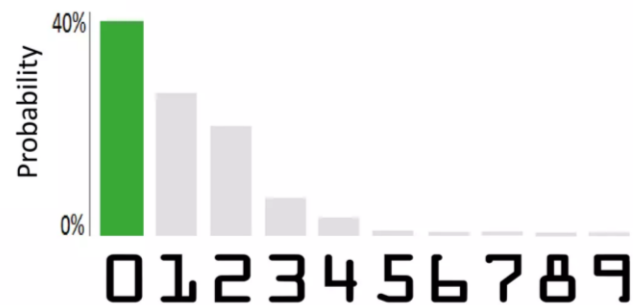
05

支持向量机

4. 朴素贝叶斯分类器

0

Probabilistic
Classifier



0. 基本概念

- 许多分类问题涉及不确定性
- 在不确定性的存在下，不仅需要对类别标签进行预测，还需要提供与每个预测相关联的置信度的度量
- 概率论为量化和处理数据中的不确定性提供了一种系统的方法
- 利用概率论来表示属性和类别标签之间的关系 的分类模型称为 概率分类模型 (probabilistic classification model)

4. 朴素贝叶斯分类器

1. 概率论基础

- 考虑变量 X ，它可以从集合 $\{x_1, \dots, x_k\}$ 取任一离散值
- 假设对于 n_i 个数据对象， X 的值均为 x_i ， $X = x_i$ 的相对频率为 $\frac{n_i}{N}$ ，其中 N 表示总的出现次数（ $N = \sum_{i=1}^k n_i$ ）
- 一个事件 e 的概率(probability)为 $P(X = x_i)$ ，代表事件 e 发生的可能性
- 在任何情况下，**概率总是介于0和1之间**
- **所有可能事件的概率值的总和等于1**
- 具有与每个可能结果(值)相关联的概率的变量称为**随机变量**(random variable)

4. 朴素贝叶斯分类器

1. 概率论基础

- 两个随机变量 X 和 Y ，每个都可以取 k 个离散值
- n_{ij} 是 $X = x_i, Y = y_j$ 被观察到次数，联合概率(joint probability)可以被估计为 $P(X = x_i, Y = y_j) = \frac{n_{ij}}{N}$ ，并且 $\sum_{j=1}^k P(X = x_i, Y = y_j) = P(X = x_i)$
- 边缘化 (marginalization)：对随机变量 Y 的联合概率求和，我们得到观察剩余变量 X 的概率
- 通过边缘化 Y 得到的概率值 $P(X = x_i)$ 有时称为 X 的边缘概率 (marginal probability)

4. 朴素贝叶斯分类器

$$P(X, Y) = P(Y|X) \times P(X) = P(X|Y) \times P(Y)$$

1. 概率论基础 --- 贝叶斯定理

- $P(Y|X)$ 表示当随机变量 X 取一个特定值时观察随机变量 Y 的**条件概率** (conditional probability)。
- $P(Y|X)$ 常被理解为以 **X 的结果为条件**的观察 Y 的**概率**。
- X 和 Y 的**条件概率**与它们的**联合概率**相关: $P(Y|X) = \frac{P(X, Y)}{P(X)} = \frac{P(X|Y)P(Y)}{P(X)}$
- $P(X) = \sum_{i=1}^k P(X, y_i) = \sum_{i=1}^k P(X|y_i) \times P(y_i)$ --- **全概率公式**
- $P(Y)$ 是**类先验概率**

贝叶斯
定理

4. 朴素贝叶斯分类器

1. 概率论基础 --- 贝叶斯定理的分类应用

- 对于分类，我们感兴趣的是计算从给定属性值 x 的数据实例中观察到类别标签 y 的概率 $P(y|x)$ ，它被称为目标类的后验概率(posterior probability)。

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

- $P(x|y)$ 被称为给定类别标签的属性的类条件(class-conditional)概率，测量从属于 y 类的实例分布中观察到 x 的可能性
- 如果 x 确实属于 y 类，那么应该期望 $P(x|y)$ 较大
- 涉及计算类条件概率的概率分类模型称为生成式模型(generative model)
 - 需要显式地学习类条件概率 $P(x|y)$ 和先验概率 $P(y)$

4. 朴素贝叶斯分类器

1. 概率论基础 --- 贝叶斯定理的分类应用

- 对于分类，我们感兴趣的是计算从给定属性值 x 的数据实例中观察到类别标签 y 的概率 $P(y|x)$ ，它被称为目标类的后验概率(posterior probability)。

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

- 先验概率 $P(y)$ 独立于观察到的属性值，捕获了关于类别标签分布的先验知识
- 先验概率既可以利用专家知识获得，也可以从类别标签的历史分布中推断出来

4. 朴素贝叶斯分类器

1. 概率论基础 --- 贝叶斯定理的分类应用

- 对于分类，我们感兴趣的是计算从给定属性值 $\mathbf{x}$ 的数据实例中观察到类别标签 y 的概率 $P(y|\mathbf{x})$ ，它被称为目标类的**后验概率**(posterior probability)。

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$

- 概率 $P(\mathbf{x})$ **不依赖于类别标签**，在后验概率计算中可以被**视为归一化常数**

$$P(\mathbf{x}) = \sum_i P(\mathbf{x}|y_i)P(y_i)$$

4. 朴素贝叶斯分类器

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

1. 概率论基础 --- 贝叶斯定理的分类应用

- 在训练阶段，我们需要学习 $P(y)$ 和 $P(x|y)$ 的参数
- 通过计算属于每个类的训练实例的占比，可以很容易地从训练集估计先验概率 $P(y)$
- 为了计算类条件概率 $P(x|y)$ ，一种方法是针对每个可能的属性值组合来考虑给定类的训练实例的占比：比如 $P(X_1 = c_i, X_2 = c_j | Y = 0) = \frac{n_{ij}^0}{n^0}$
- 由于属性值组合的数量呈指数增长，该方法很容易随着属性数量的增加而变得难以计算： d 个属性，每个属性可以取 k 个值，组合等于 k^d
- 大量的属性值组合也会导致类条件概率的估计较差，因为当训练集较小时，每个组合将具有较少的训练实例，甚至没有对应的实例

4. 朴素贝叶斯分类器

2. 朴素贝叶斯假设 --- 条件独立

- 每个数据实例 x 由 d 个属性组成 $\{x_1, x_2, \dots, x_d\}$
- 朴素贝叶斯分类器假设所有属性 x 的类条件概率 $P(x|y)$ 可以被分解为类条件概率的乘积： $P(x|y) = \prod_{i=1}^d P(x_i|y)$
- 上述式子背后的基本假设是：给定的类别标签 y ，属性值 x_i 是相互条件独立(conditionally independent)的，这意味着只有目标类可以影响属性
- 设 X_1, X_2 和 Y 表示三组随机变量。给定 Y ，如果下列条件成立，则称 X_1 条件独立于 X_2 ： $P(X_1|X_2, Y) = P(X_1|Y)$ 或 $P(X_1, X_2|Y) = P(X_1|Y) \times P(X_2|Y)$
- 在 Y 条件下， X_1 的分布不受 X_2 的结果影响

4. 朴素贝叶斯分类器

2. 朴素贝叶斯假设 --- 工作机制

- 如果 n_i^0 和 n_j^0 分别表示属于类0且满足 $X_1 = c_i$ 和 $X_2 = c_j$ 的训练实例的数目, 则类条件概率可以被估计为:

$$P(X_1 = c_i, X_2 = c_j | Y = 0) = P(X_1 = c_i | Y = 0) \times P(X_2 = c_j | Y = 0) = \frac{n_i^0}{n^0} \times \frac{n_j^0}{n^0}$$

- 只需要计算属性 X 的 k 个值中的每一个的训练实例数, 而不考虑其他属性的值, 学习类条件概率所需的参数数量从 k^d 减少到 $d \times k$
- 朴素贝叶斯分类器利用以下方程计算检验实例 $\mathbf{x}$ 的后验概率

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^d P(x_i|y)}{P(\mathbf{x})}$$

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^d P(x_i|y)}{P(\mathbf{x})}$$

2. 朴素贝叶斯假设 --- 工作机制

- 由于 $P(\mathbf{x})$ 对于每个 y 的值都是固定的，则类条件概率可以被估计为：

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

- 因此，选择能最大化 $P(y) \prod_{i=1}^d P(x_i|y)$ 的类就足够了
- 朴素贝叶斯分类器可以自然地处理测试实例中的缺失值
- 在没有观察到属性的极端情况下，仍然可以使用先验概率 $P(y)$ 作为后验概率的估计
- 当观察更多的属性时，可以不断细化后验概率，以更好地反映观察数据实例的可能性

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^d P(x_i|y)}{P(\mathbf{x})}$$

2. 朴素贝叶斯假设 --- 分类属性的条件概率估计

一元变量

分类变量

连续变量

分类变量

- 给定一个分类属性 X_i ，类条件概率 $P(X_i = c|y)$ 是根据 y 类别中 X_i 取特定分类值 c 的训练实例的比例

来估计的:

$$P(X_i = c|y) = \frac{n_c}{n}$$

- n 为训练实例属于 y 类的数目，其中 n_c 个实例具有 $X_i = c$
- 7个样本的类别标签为(拖欠贷款 = 否)，当中3个是(有房 = 是)。结果， $P(\text{有房}=\text{是}|\text{拖欠贷款}=\text{否})$ 的条件概率等于 $\frac{3}{7}$ 。

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^d P(x_i|y)}{P(\mathbf{x})}$$

2. 朴素贝叶斯假设 --- 连续属性的条件概率估计

- 1) 可以把每一个连续的属性离散化，然后用相应的离散区间替换连续属性值，把连续属性转换成序数属性，并且可以采用先前描述的用于计算分类属性的条件概率的简单方法
 - 通过计算类 y 的训练实例中落入 X_i 对应区间的比例来估计条件概率 $P(X_i|Y = y)$
- 2) 可以假设连续变量服从某种概率分布，然后使用训练数据估计分布的参数
 - 比如高斯分布： $P(X_i = x_i|Y = y_j) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} \exp\left[-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}\right]$
 - 可以用属于类 y_j 的所有训练实例关于 X_i 的样本均值 $\bar{x}$ 来估计期望值 μ_{ij}
 - 可以用这些训练实例的样本方差 S^2 来估计方差 σ_{ij}^2

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

2. 朴素贝叶斯假设

$X = (\text{Owner} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

预测 $Y = \text{Yes}$

方法：
连续属性离散化

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$$\begin{aligned} P(\text{Yes} | X) &\propto (P(\text{Owner} = \text{No} | \text{Yes}) \times \\ &\quad P(\text{Divorced} | \text{Yes}) \times \\ &\quad P(\text{Income} \leq 120\text{K} | \text{Yes})) \times \\ &\quad P(\text{Yes}) \\ &= (1 \times \frac{1}{3} \times 1) \times \frac{3}{10} = \frac{1}{10} \end{aligned}$$

$$\begin{aligned} P(\text{No} | X) &\propto (P(\text{Owner} = \text{No} | \text{No}) \times \\ &\quad P(\text{Divorced} | \text{No}) \times \\ &\quad P(\text{Income} \leq 120\text{K} | \text{No})) \times \\ &\quad P(\text{No}) \\ &= (\frac{4}{7} \times \frac{1}{7} \times \frac{5}{7}) \times \frac{7}{10} = \frac{2}{49} \end{aligned}$$

4. 朴素贝叶斯分类器

2. 朴素贝叶斯假设

Y=Yes	$\bar{x} = \frac{95+85+90}{3}=90$
	$S^2 = \frac{(95-90)^2+(85-90)^2+(90-90)^2}{3-1}=25$
Y=No	$\bar{x} = \frac{125+100+70+120+60+220+75}{7}=110$
	$S^2 = \frac{(125-110)^2+(100-110)^2+\dots+(220-110)^2+(75-110)^2}{7-1}=2975$

$X = (Owner = No, Divorced, Income = 120K)$

$$P(Income = 120K|Yes) = 1.2 \times 10^{-9}$$

$$P(Income = 120K|No) = 0.0072$$

预测 $Y = No$

$$\begin{aligned} P(Yes|X) &\propto (P(Owner = No|Yes) \times \\ &\quad P(Divorced|Yes) \times \\ &\quad P(Income = 120K|Yes)) \times \\ &\quad P(Yes) \\ &= \left(1 \times \frac{1}{3} \times 1.2 \times 10^{-9}\right) \times \frac{3}{10} = 1.2 \times 10^{-10} \end{aligned}$$

$$\begin{aligned} P(No|X) &\propto (P(Owner = No|No) \times \\ &\quad P(Divorced|No) \times \\ &\quad P(Income = 120K|No)) \times \\ &\quad P(No) \\ &= \left(\frac{4}{7} \times \frac{1}{7} \times 0.0072\right) \times \frac{7}{10} = 4.1 \times 10^{-4} \end{aligned}$$

方法：
假设服从概率分布

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

2. 朴素贝叶斯假设 --- 零条件概率

- 如果任何属性的条件概率为零，则类条件概率的整个表达式变为零
- 当训练实例的数目小且属性的可能值的数目大时，会出现零条件概率
- 可能会出现属性值和类别标签的组合从未被观察到，从而导致零条件概率
- 在更极端的情况下，如果训练实例不包含属性值和类别标签的一些组合，那么可能甚至无法对一些测试实例进行分类

4. 朴素贝叶斯分类器

2. 朴素贝叶斯假设

$X = (\text{Owner} = \text{Yes}, \text{Divorced}, \text{Income} = 120\text{K})$

ID	有房	婚姻状况	年收入	拖欠贷款
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

假设删除
这条记录

Y=Yes	$\bar{x} = \frac{95+85+90}{3}=90$
	$S^2 = \frac{(95-90)^2+(85-90)^2+(90-90)^2}{3-1}=25$
Y=No	$\bar{x} = \frac{125+100+70+120+60+75}{6}=91$
	$S^2 = \frac{(125-91)^2+(100-91)^2+\dots+(60-91)^2+(75-91)^2}{6-1}=685$

$$P(\text{Income} = 120\text{K} | \text{Yes}) = 1.2 \times 10^{-9}$$

$$P(\text{Income} = 120\text{K} | \text{No}) = 0.0083$$

$$\begin{aligned} P(\text{Yes} | X) &\propto (P(\text{Owner} = \text{Yes} | \text{Yes}) \times \\ &\quad P(\text{Divorced} | \text{Yes}) \times \\ &\quad P(\text{Income} = 120\text{K} | \text{Yes})) \times \\ &\quad P(\text{Yes}) \end{aligned}$$

$$= \left(0 \times \frac{1}{3} \times 1.2 \times 10^{-9} \right) \times \frac{3}{9} = 0$$

两个概率一样
无法分类!

$$\begin{aligned} P(\text{No} | X) &\propto (P(\text{Owner} = \text{Yes} | \text{No}) \times \\ &\quad P(\text{Divorced} | \text{No}) \times \\ &\quad P(\text{Income} = 120\text{K} | \text{No})) \times \\ &\quad P(\text{No}) \end{aligned}$$

$$= \left(\frac{2}{6} \times 0 \times 0.0083 \right) \times \frac{6}{9} = 0$$

4. 朴素贝叶斯分类器

2. 朴素贝叶斯假设 --- 零条件概率

- 由于两类条件概率均为 0，朴素贝叶斯分类器将无法对实例进行分类
- 重要的是调整条件概率估计，使得它们不像使用训练实例的比例那样敏感

原始估计: $P(X_i = c|y) = \frac{n_c}{n}$

拉普拉斯估计: $P(X_i = c|y) = \frac{n_c + 1}{n + v}$

m - 估计: $P(X_i = c|y) = \frac{n_c + mp}{n + m}$

n : 属于类 y 的训练实例数

n_c : $X_i = c$ 且 $Y = y$ 的实例数

v : 属性 X_i 所能取的值的总数

p : 先验已知的 $P(X_i = c|y)$ 初步估计

m : 对 p 的置信度的超参数

4. 朴素贝叶斯分类器

$$\text{原始估计: } P(X_i = c|y) = \frac{n_c}{n}$$

$$\text{拉普拉斯估计: } P(X_i = c|y) = \frac{n_c + 1}{n + v}$$

$$m - \text{估计: } P(X_i = c|y) = \frac{n_c + mp}{n + m}$$

2. 朴素贝叶斯假设 --- 零条件概率

■ 例子：假设一个硬币投掷10次，结果：7次正面（H）、3次反面（T）

■ 原始估计： $P(H) = \frac{7}{10} = 0.7$, $P(T) = \frac{3}{10} = 0.3$

■ 拉普拉斯估计： $P(H) = \frac{7+1}{10+2} = \frac{8}{12} \approx 0.666$

■ 类别数 $v=2$ $P(T) = \frac{3+1}{10+2} = \frac{4}{12} \approx 0.333$

■ m -估计：

■ 先验概率 $p=0.5$ (认为H和T理论上等概率) $P(H) = \frac{7+2 \times 0.5}{10+2} = \frac{8}{12} \approx 0.666$

■ 超参数 $m=2$ (相当于增加2次伪试验) $P(T) = \frac{3+2 \times 0.5}{10+2} = \frac{4}{12} \approx 0.333$

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

2. 朴素贝叶斯假设 --- 分类器特征

- 1) 朴素贝叶斯分类器是能够通过提供**后验概率估计**来量化预测中的**不确定性的**概率分类模型，也是**生成分类模型**，捕捉生成属于每个类的**数据实例背后的基础机制**，有助于获得**预测性和描述性**见解
- 2) 即使在高维情况中，只要给定类标签，属性在条件上**相互独立**，可以很容易地计算类条件概率，是一种简单有效的分类技术
- 3) 朴素贝叶斯分类器对**孤立噪声点**具有**鲁棒性**，因为这些点**不能对条件概率估计产生显著的影响**

4. 朴素贝叶斯分类器

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

2. 朴素贝叶斯假设 --- 分类器特征

- 4) 朴素贝叶斯分类器能够通过通过在计算条件概率估计时忽略各属性的缺失值，从而处理训练集中的缺失值问题。如果某一属性值缺失的频率依赖于类别标签，那么这种方法将无法准确估计后验概率。
- 5) 朴素贝叶斯分类器对不相关属性具有鲁棒性。如果 X_i 是一个无关的属性，那么 $P(X_i|Y)$ 对于每个类 y 几乎是均匀分布的。
- 6) 相关属性会降低朴素贝叶斯分类器的性能，因为对于这些属性，条件独立的朴素贝叶斯假设不再适用。

目 录

01

基本概念

02

决策树分类器

03

最近邻分类器

04

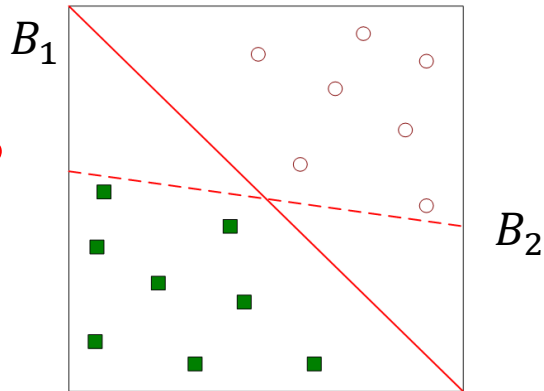
朴素贝叶斯分类器

05

支持向量机

5. 支持向量机

哪个决策边界更好?
 B_1 或者 B_2 ?



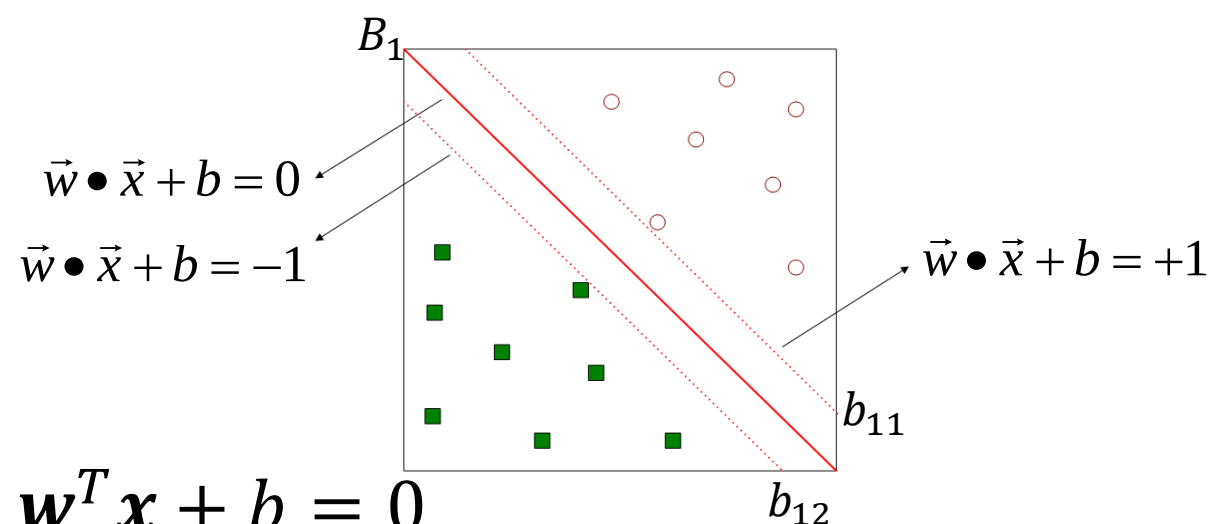
1. 基本概念

- 支持向量机(Support Vector Machine, SVM)是一种**二分类**模型, 用于学习属性空间中的**线性或非线性决策边界**来实现类别划分
- 线性决策边界: 在二维空间中该决策边界是一条直线, 在三维空间中该决策边界是一个平面, 而在更高维空间中该决策边界是一个超平面
- SVM仅使用训练实例**最难分类**的一个**子集**来表示**决策边界**, 该子集称作**支持向量**(support vector)。
- SVM是一个**判别(discriminative)**模型, 仅受两类训练样本中**边界邻近样本**的影响, 而不是学习每个类的**生成分布**。

5. 支持向量机

2. 分离超平面的间隔

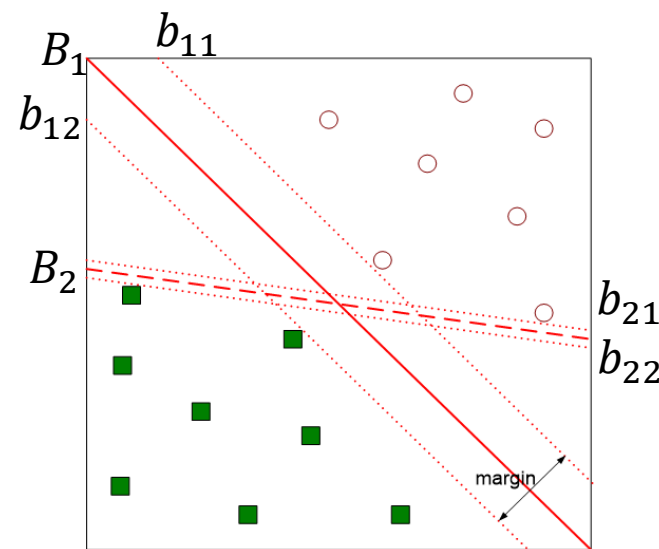
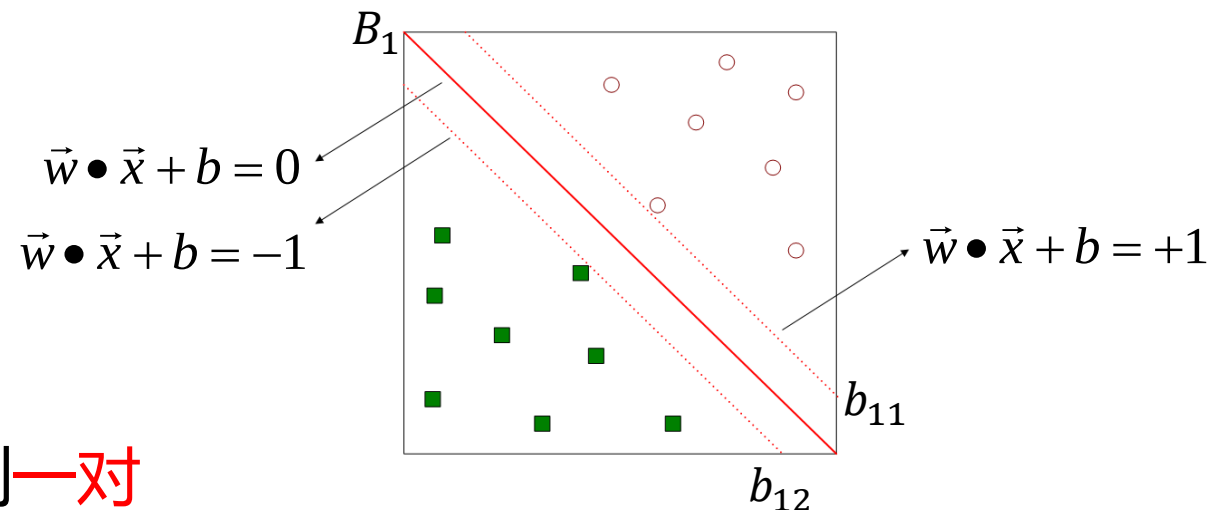
- 分离超平面的通用方程可写为： $\mathbf{w}^T \mathbf{x} + b = 0$
- $\mathbf{x}$ 代表属性， $(\mathbf{w}, b)$ 代表超平面的参数。数据实例 $\mathbf{x}_i$ 可以属于超平面的任一侧，取决于 $(\mathbf{w}^T \mathbf{x}_i + b)$ 的符号。
- 找到一个超平面，它将两个类的实例放置在超平面的相对侧，从而导致两个类的分离。
- 如果存在能够完全分离数据集中的类的超平面，那么数据集是线性可分(linearly separable)



5. 支持向量机

2. 分离超平面的间隔

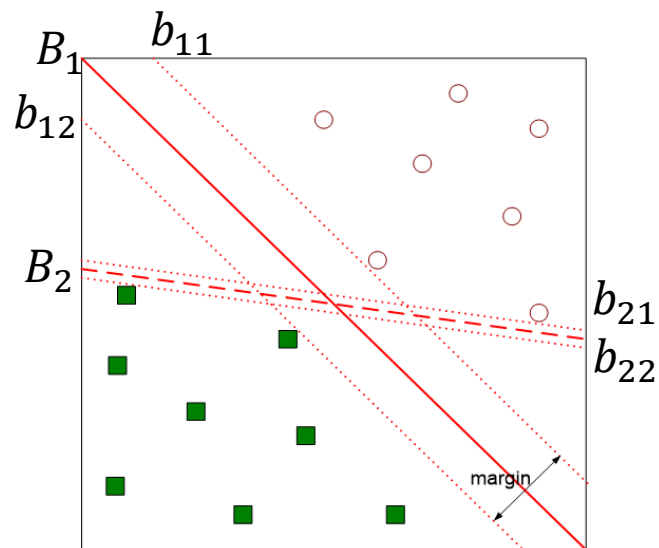
- 对于每个分离超平面 B_i ，可以对应到一对平行超平面 b_{i1} 和 b_{i2} ，使它们分别触到两个类最近的实例
- b_{i1} 和 b_{i2} 被称为 B_i 的间隔超平面 (margin hyperplane)，它们之间的间距被称为分离超平面 B_i 的间隔 (margin)。
- 右图中， B_1 就是具有最大间隔的分离超平面，称为最大间隔超平面 (maximum margin hyperplane)
- 支持向量：距离超平面最近的几个样本点



5. 支持向量机

2. 分离超平面的间隔

- 具有较大间隔的超平面往往比那些具有较小间隔的超平面具有更好的泛化性能
- 如果间隔比较小，那么超平面或边界附近的训练实例的任何轻微扰动都可能对分类性能产生显著的影响
- 具有较小间隔的超平面更容易过拟合，因为它们几乎不能将位于非常窄的区域里的类分离开，因而不能允许扰动的出现
- 同时远离属于两个类的训练实例的超平面具有充分的余地来对数据中的微小修改产生鲁棒性，并因此显示出优越的泛化性能



5. 支持向量机

3. 线性SVM

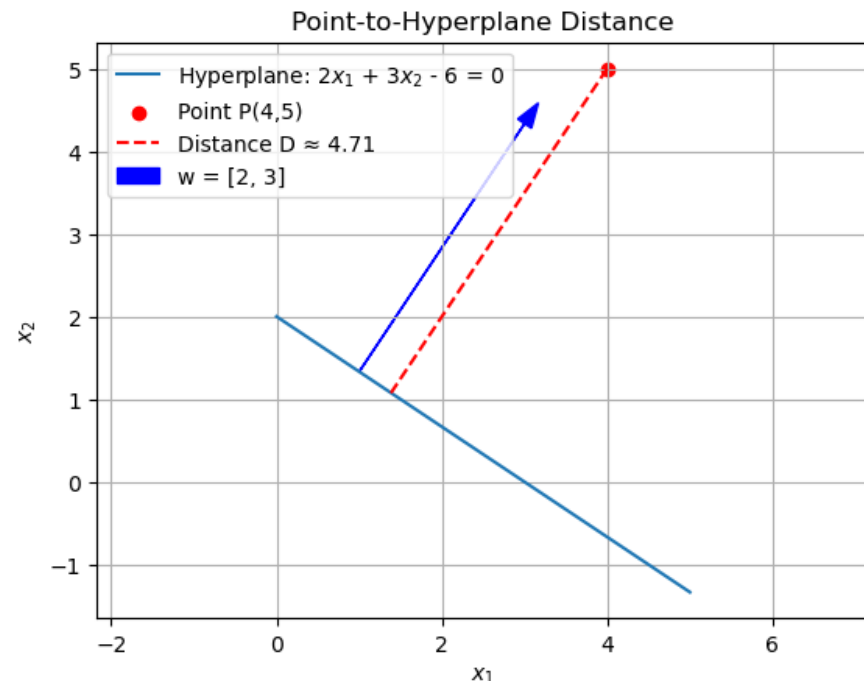
- 线性SVM是寻找具有最大间隔的分离超平面的分类器，因此它也经常被称为最大间隔分类器(maximal margin classifier)
- 考虑一个包含 n 个训练实例的二分类问题，其中每个训练实例 x_i 与二元标签 $y_i \in \{-1, +1\}$ 相关联
- 令 $w^T x + b = 0$ 是一个分离超平面的方程，它通过将两个类放置在超平面相反的两侧来分离这两个类

$$\begin{cases} w^T x_i + b > 0, & \text{如果 } y_i = +1 \\ w^T x_i + b < 0, & \text{如果 } y_i = -1 \end{cases}$$

5. 支持向量机

3. 线性SVM

- 任意点 x 到超平面的距离 $D(x) = \frac{|w^T x + b|}{\|w\|}$
- w 是超平面的法向量（决定超平面的方向）
- $|w^T x + b|$: 表示点 x 偏离超平面的程度（函数间隔的绝对值）
- $\|w\|$ 是法向量 w 的L2范数，用于归一化距离
- 例：设超平面为 $2x_1 + 3x_2 - 6 = 0$ (即 $w = [2, 3]$, $b = -6$)，计算点 $(4, 5)$ 到它的距离
 - 计算函数间隔： $w \cdot x + b = 2 \times 4 + 3 \times 5 - 6 = 8 + 15 - 6 = 17$
 - 计算法向量的L2范数： $\|w\| = \sqrt{2 \times 2 + 3 \times 3} = \sqrt{13}$
 - 代入距离公式： $D(x) = 17 / \sqrt{13} \approx 4.71$



5. 支持向量机

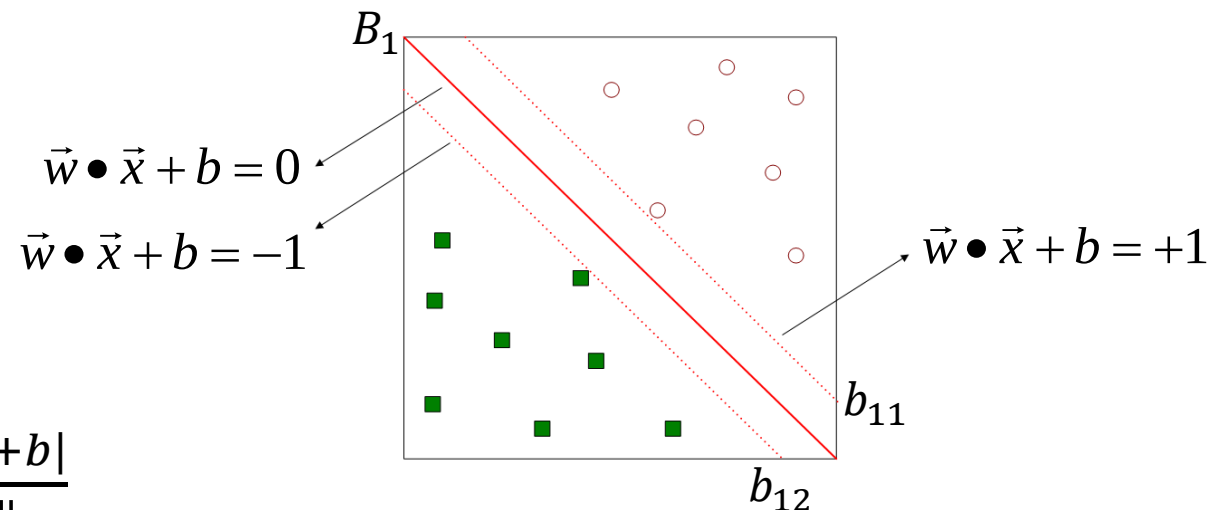
3. 线性SVM

- 任意点 x 到超平面的距离 $D(x) = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$
- 令超平面与 $y = +1$ 的最近点的距离为 $k_+ > 0$, 令 $k_- > 0$ 表示超平面与负类 ($y = -1$) 最近点的距离, 可以使用以下约束来表示:

$$\begin{cases} \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|} \geq k_+, & y_i = +1 \\ \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|} \leq -k_-, & y_i = -1 \end{cases}$$

$$\begin{cases} \mathbf{w}^T \mathbf{x}_i + b > 0, & \text{如果 } y_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b < 0, & \text{如果 } y_i = -1 \end{cases}$$

- 通过使用 y_i 和 $(\mathbf{w}^T \mathbf{x}_i + b)$ 的乘积来简洁地表示: $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq M \|\mathbf{w}\|$
- M 是与超平面的间隔有关的参数, 如果 $k_+ = k_- = M$, 则间隔 $= k_+ + k_- = 2M$



5. 支持向量机

3. 线性SVM

- 为了找到符合先前约束的最大间隔超平面，可以考虑以下优化问题：

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & M \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq M \|\mathbf{w}\| \end{aligned}$$

- 选择 $\|\mathbf{w}\| = 1/M$ 来简化不等式的右边，最大化 M 相当于最小化 $\|\mathbf{w}\|^2$ ，SVM的优化问题通常以下面的形式表示：

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{\|\mathbf{w}\|^2}{2} \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \end{aligned}$$

5. 支持向量机

3. 线性SVM

- 为什么最大化 $1/\|\mathbf{w}\|$ 等价于最小化 $\|\mathbf{w}\|^2$ --- 单调性论证
- 函数 $f(x)=1/x$ 和 $g(x)=x^2$ 的单调性:
 - 函数 $f(x)=1/x$ 在定义域 $x>0$ 上是一个单调递减函数, 最大化 $f(x)=1/x$ 等价于最小化 x
 - $\|\mathbf{w}\|$ 是非负的, 所以最大化 $1/\|\mathbf{w}\|$ 等价于最小化 $\|\mathbf{w}\|$
 - 函数 $g(x)=x^2$ 在 $x \geq 0$ 上是一个单调递增函数, x 的值越小, x^2 的值也越小
 - $\|\mathbf{w}\|$ 是非负的, 最小化 $\|\mathbf{w}\|^2$ 等价于最小化 $\|\mathbf{w}\|$
- 根据以上两点, 可以建立起等价链:

$$\text{Maximize } \frac{1}{\|\mathbf{w}\|} \quad \Leftrightarrow \quad \text{Minimize } \|\mathbf{w}\| \quad \Leftrightarrow \quad \text{Minimize } \|\mathbf{w}\|^2$$

5. 支持向量机

3. 线性SVM

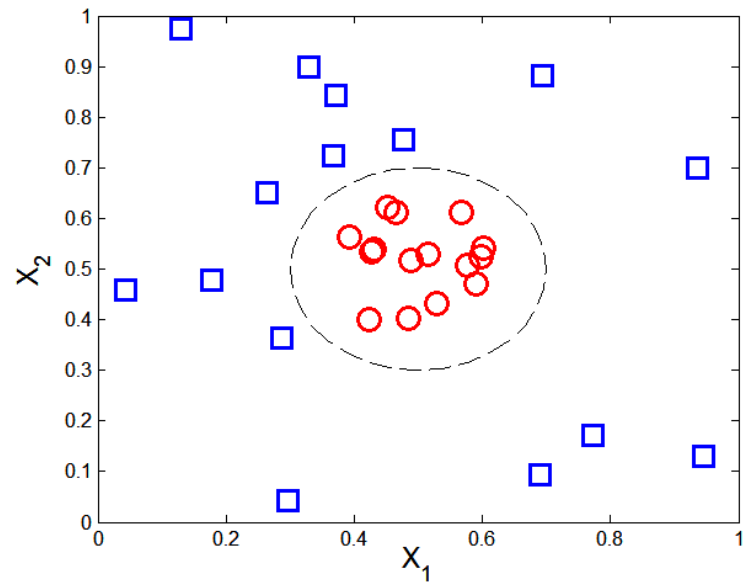
- 将目标函数从 $\|\mathbf{w}\|$ 转换为 $\|\mathbf{w}\|^2$ （并加上1/2的系数），带来了三大核心优势
- 1. 光滑性（Smoothness）：消除了在原点不可微的点，使得求导运算在任
何地方都可以进行
- 2. 强凸性（Strong Convexity）：保证了优化问题的解是唯一的，并且可
以使用高效的凸优化器
- 3. 推导便利性（Derivation Friendly）：特别是在拉格朗日对偶变换中，
求导后能得到简洁、线性、解耦的结果 $(\mathbf{w} = \sum_{i=1}^n \lambda_i y_i \mathbf{x}_i)$

↑
训练样本的线性组合

5. 支持向量机

4. 非线性SVM

- 上一节描述的SVM公式构建了一个线性的决策边界，从而把训练实例划分到它们各自的类中，如果数据集的决策边界是非线性的？
- 将数据从原先的属性空间 x 中变换到一个新的空间 $\varphi(x)$ 中，从而可以在变换后的空间中使用由SVM方法得到的线性超平面来划分实例
- 然后将学习到的超平面投影回原始属性空间，从而产生非线性决策边界



5. 支持向量机

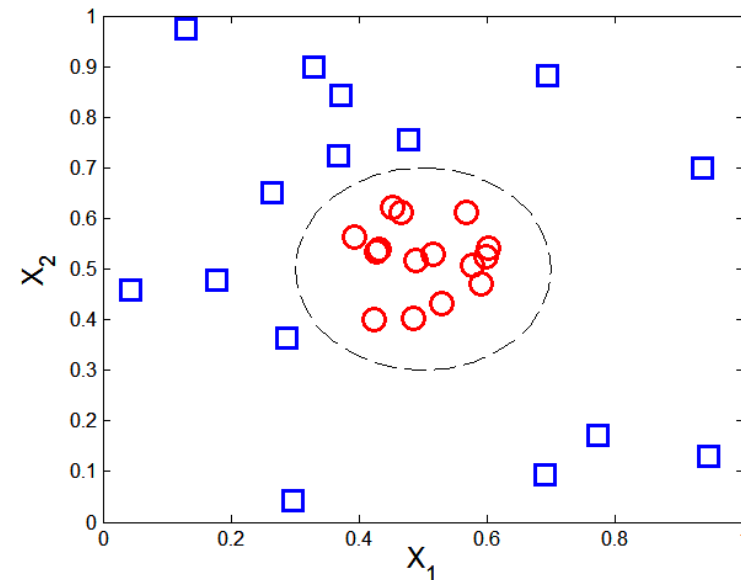
4. 非线性SVM --- 属性变换

- 一个二维数据集，包含两类样本（□和○，类标为 $y = +1$ 和 $y = -1$ ）
- 可以使用下面的公式分类数据集中的实例

$$y(x_1, x_2) = \begin{cases} +1 & \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} > 0.2 \\ -1 & \text{其他} \end{cases}$$

- 数据集的决策边界可以表示如下：

$$\sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} = 0.2$$



5. 支持向量机

4. 非线性SVM --- 属性变换

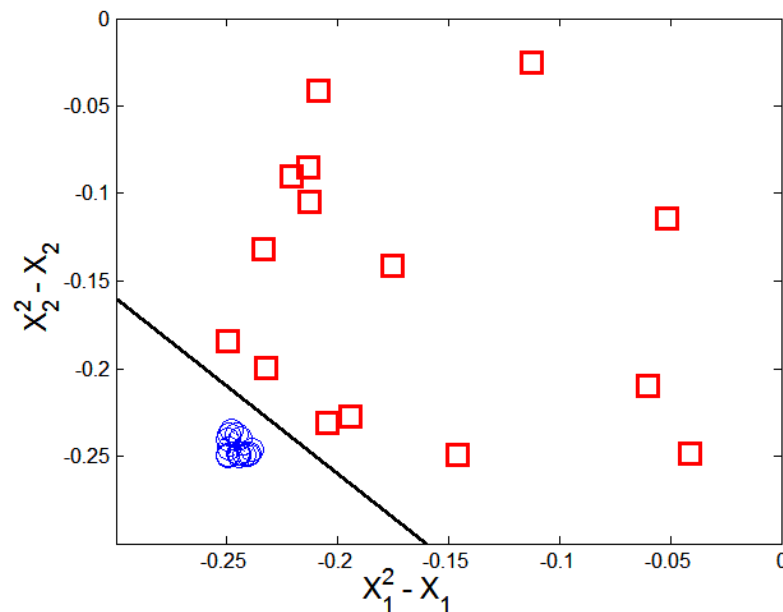
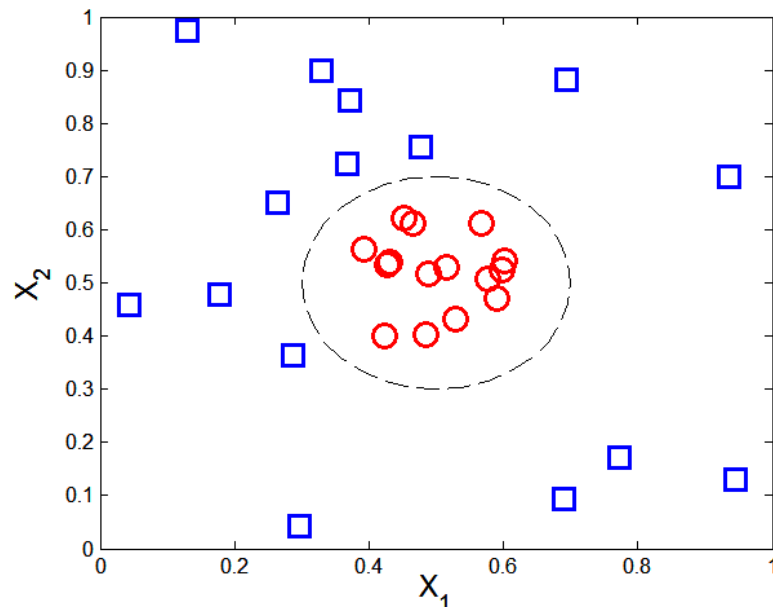
- 可以进一步简化为下面的二次方程

$$(x_1^2 - x_1) + (x_2^2 - x_2) = -0.46$$

- 需要一个非线性变换 φ ，将数据从其原始属性空间映射到一个新的空间，以便线性超平面可以分离这些类。可以通过使用以下简单变换来实现：

$$\varphi: (x_1, x_2) \rightarrow (x_1^2 - x_1, x_2^2 - x_2)$$

- 可以在变换后的空间中构建一个参数为 w 和 b 的线性超平面，从而把实例划分到各自所属的类中



5. 支持向量机

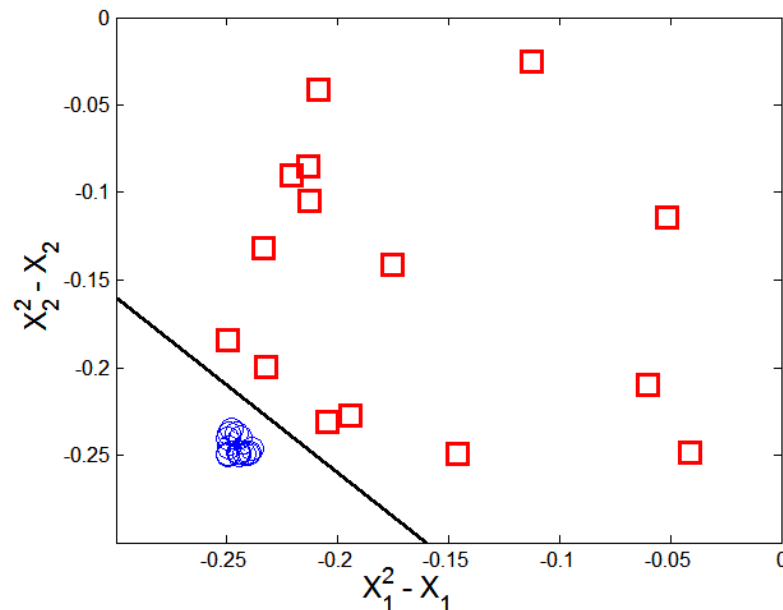
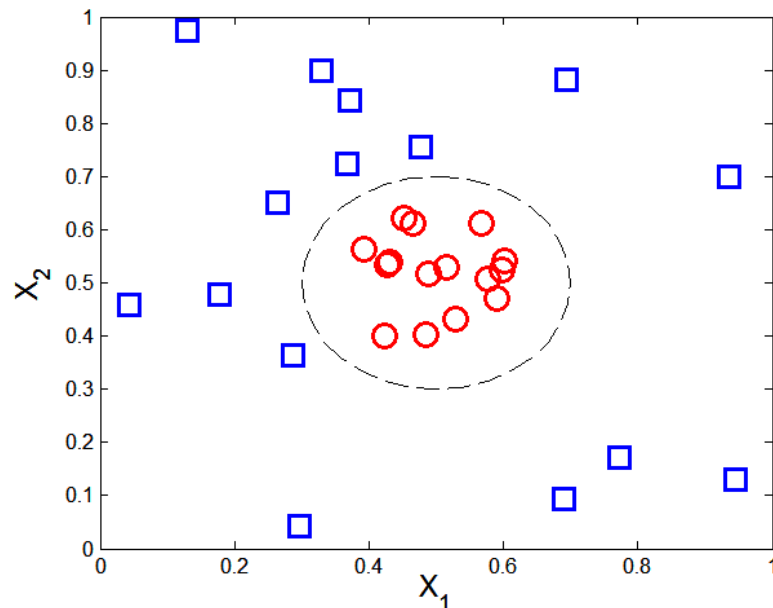
4. 非线性SVM

- 变换空间中的线性超平面可以表示为

$$\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + b = 0$$

- 为了学习最优分离超平面，可以用 $\boldsymbol{\varphi}(\mathbf{x})$ 代替SVM公式中的 $\mathbf{x}$ 来获得以下优化问题实现：

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{\|\mathbf{w}\|^2}{2} \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + b) \geq 1 \end{aligned}$$



5. 支持向量机

特征	凸二次规划	非凸二次规划
目标函数	凸函数	非凸函数
海森矩阵	半正定	不定或负定
最优解	全局最优	可能多个局部最优
求解难度	多项式时间	NP-Hard

5. 学习模型参数

- SVM的优化问题表示为：

$$\min_{\mathbf{w}, b} \frac{\|\mathbf{w}\|^2}{2}$$
$$s.t. \quad y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

- 根据拉格朗日乘子 λ 获得以下优化问题：

$$\max_{\lambda} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$
$$s.t. \quad \sum_{i=1}^n \lambda_i y_i = 0, \lambda_i \geq 0$$

- 上面的优化问题称为对偶优化问题(dual optimization problem)
- λ_i ：拉格朗日乘子，只有支持向量的 $\lambda_i > 0$

5. 支持向量机

5. 学习模型参数

- 对偶问题的解仅受最接近SVM的决策边界的支持向量影响，不依赖于远离边界的实例的分布
- 对偶问题的目标只涉及形如 $\mathbf{x}_i^T \mathbf{x}_j$ 的项，它们基本上是属性空间中的内积
- 最大间隔超平面可以表示为：

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \left(\sum_{i=1}^n \lambda_i y_i \mathbf{x}_i^T \mathbf{x} \right) + b = 0$$

- 使用这个分离超平面，可以使用 $f(\mathbf{x})$ 的符号为测试实例 $\mathbf{x}$ 分配类别标签

5. 支持向量机

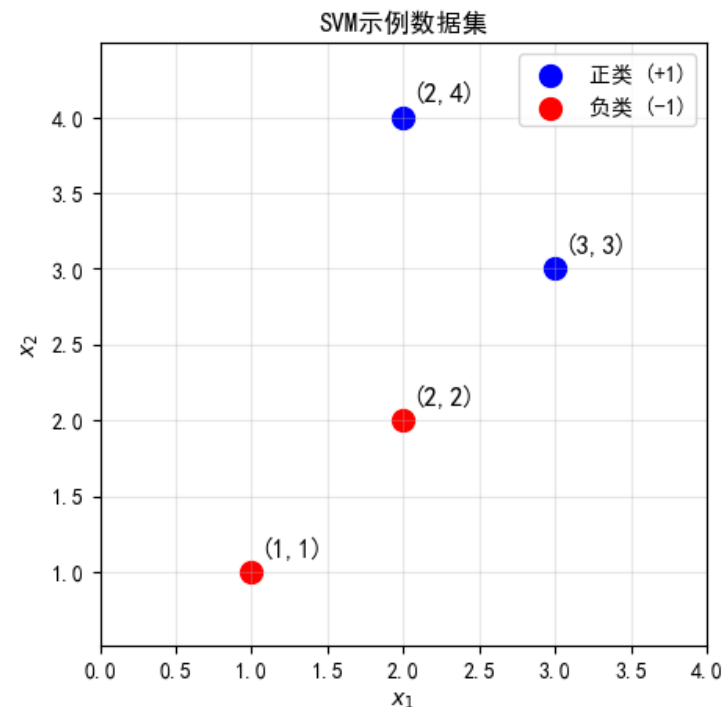
5. 学习模型参数

- 正类 ($y = +1$): (2, 4), (3, 3), 负类 ($y = -1$): (1, 1), (2, 2)
- 令 $x_1 = (2, 4)$, $x_2 = (3, 3)$, $x_3 = (1, 1)$, $x_4 = (2, 2)$
- 计算向量两两的内积 $x_i^T x_j$:

$$\begin{bmatrix} 20. & 18. & 6. & 12. \\ 18. & 18. & 6. & 12. \\ 6. & 6. & 2. & 4. \\ 12. & 12. & 4. & 8. \end{bmatrix}$$

- 对偶问题约束条件:

$$\begin{aligned} \lambda_1 + \lambda_2 - \lambda_3 - \lambda_4 &= 0 \\ \lambda_i &\geq 0 \end{aligned}$$



优化目标:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i$

拉格朗日对偶问题:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad & \sum_{i=1}^n \lambda_i y_i = 0, \lambda_i \geq 0 \end{aligned}$$

5. 支持向量机

5. 学习模型参数

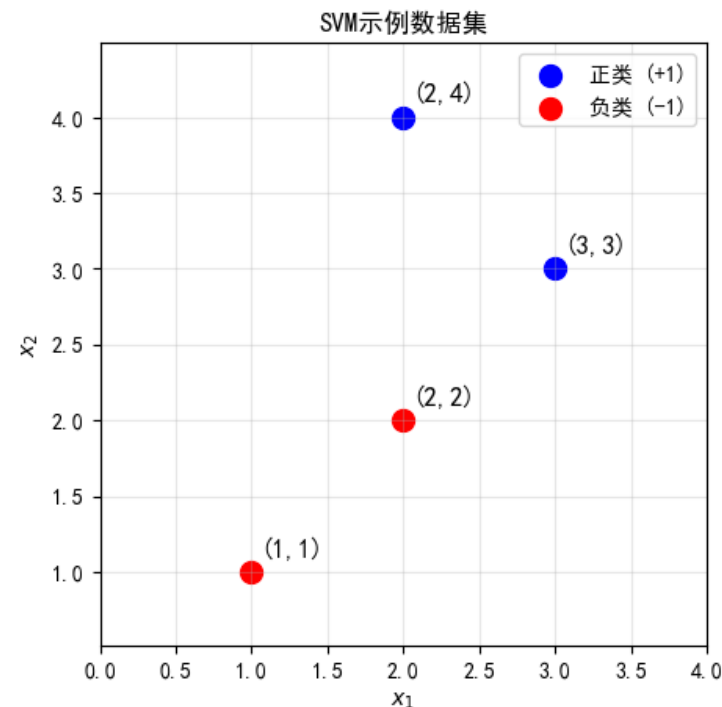
- 令 $x_1 = (2, 4)$, $x_2 = (3, 3)$, $x_3 = (1, 1)$, $x_4 = (2, 2)$
- 通过观察, 可能的支持向量是 x_2 和 x_4 , 只有支持向量的 $\lambda_i > 0$
- $\lambda_2 > 0$, $\lambda_4 > 0$, $\lambda_1 = 0$, $\lambda_3 = 0$, 所以 $\lambda_2 = \lambda_4 = a$
- 优化目标函数:

$$2a - \frac{1}{2}[a^2 * 18 + a^2 * 8 - a^2 * 12 - a^2 * 12] = 2a - 2a^2$$

- 对目标函数求导: $2 - 4a = 0$, $a = 1/2$ 时得到最大值
- 计算权重向量

$$w = \sum_{i=1}^4 \lambda_i y_i x_i = \frac{1}{2} * 1 * \begin{bmatrix} 3 \\ 3 \end{bmatrix} + \frac{1}{2} * (-1) * \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

$\lambda_i \lambda_j y_i y_j$	$x_i^T x_j$
$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 20. & 18. & 6. & 12. \end{bmatrix}$
$\begin{bmatrix} 0 & a^2 & 0 & -a^2 \end{bmatrix}$	$\begin{bmatrix} 18. & 18. & 6. & 12. \end{bmatrix}$
$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 6. & 6. & 2. & 4. \end{bmatrix}$
$\begin{bmatrix} 0 & -a^2 & 0 & a \end{bmatrix}$	$\begin{bmatrix} 12. & 12. & 4. & 8. \end{bmatrix}$



优化目标: $\min_{w, b} \frac{1}{2} \|w\|^2$

subject to $y_i(w^T x_i + b) \geq 1, \quad \forall i$

拉格朗日对偶问题:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & \sum_{i=1}^n \lambda_i y_i = 0, \lambda_i \geq 0 \end{aligned}$$

5. 支持向量机

$$\mathbf{x}_i^T \mathbf{x}_j \begin{bmatrix} 20. & 18. & 6. & 12. \\ 18. & \mathbf{18.} & 6. & \mathbf{12.} \\ 6. & 6. & 2. & 4. \\ 12. & \mathbf{12.} & 4. & \mathbf{8.} \end{bmatrix}$$

5. 学习模型参数

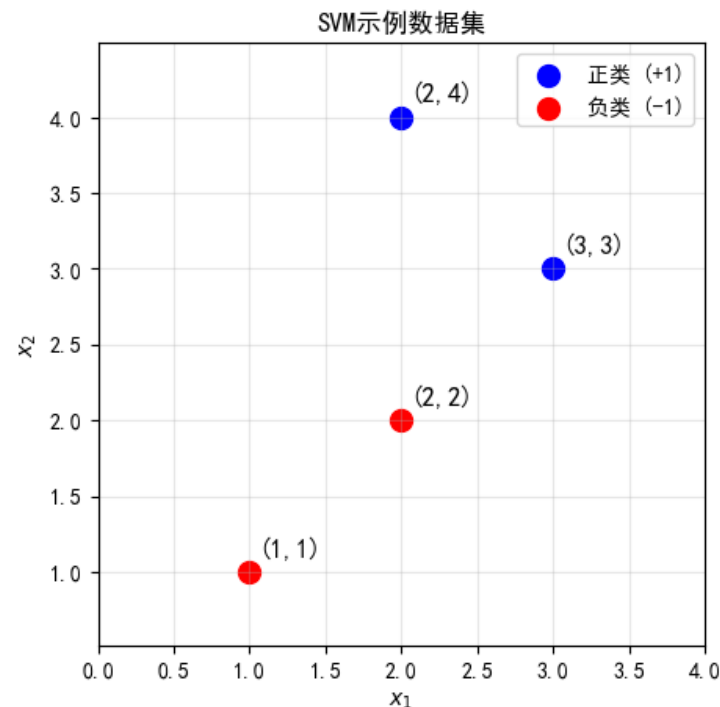
- 令 $\mathbf{x}_1 = (2, 4)$, $\mathbf{x}_2 = (3, 3)$, $\mathbf{x}_3 = (1, 1)$, $\mathbf{x}_4 = (2, 2)$
- 对于任何支持向量 $\mathbf{x}_s \in \mathcal{S}$, 其类标签为 y_s , 都有 $y_s f(\mathbf{x}_s) = 1$, 即

$$y_s \left(\sum_{i=1}^n \lambda_i y_i \mathbf{x}_i^T \mathbf{x}_s + b \right) = 1$$

- 利用支持向量 $\mathbf{x}_2$ 、 $\mathbf{x}_4$, 求解 b

$$b = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (y_s - \sum_{i \in \mathcal{S}} \lambda_i y_i \mathbf{x}_i^T \mathbf{x}_s)$$

- $$\begin{aligned} b &= \frac{1}{2} [(1 - 0.5 \mathbf{x}_2^T \mathbf{x}_2 + 0.5 \mathbf{x}_4^T \mathbf{x}_2) + (-1 - 0.5 \mathbf{x}_2^T \mathbf{x}_4 + 0.5 \mathbf{x}_4^T \mathbf{x}_4)] \\ &= \frac{1}{2} [(1 - 0.5 * \mathbf{18} + 0.5 * \mathbf{12}) + (-1 - 0.5 * \mathbf{12} + 0.5 * \mathbf{8})] \\ &= -2.5 \end{aligned}$$



优化目标:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i$

拉格朗日对偶问题:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad & \sum_{i=1}^n \lambda_i y_i = 0, \lambda_i \geq 0 \end{aligned}$$

5. 支持向量机

5. 学习模型参数

■ 令 $x_1 = (2, 4)$, $x_2 = (3, 3)$, $x_3 = (1, 1)$, $x_4 = (2, 2)$

■ $w = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, $b = -0.25$

■ 间隔超平面可以表示为

$$f(x) = w^T x + b = 0.5x_1 + 0.5x_2 - 2.5 = 0$$

■ 还要满足 $y_i (w^T x_i + b) \geq 1$

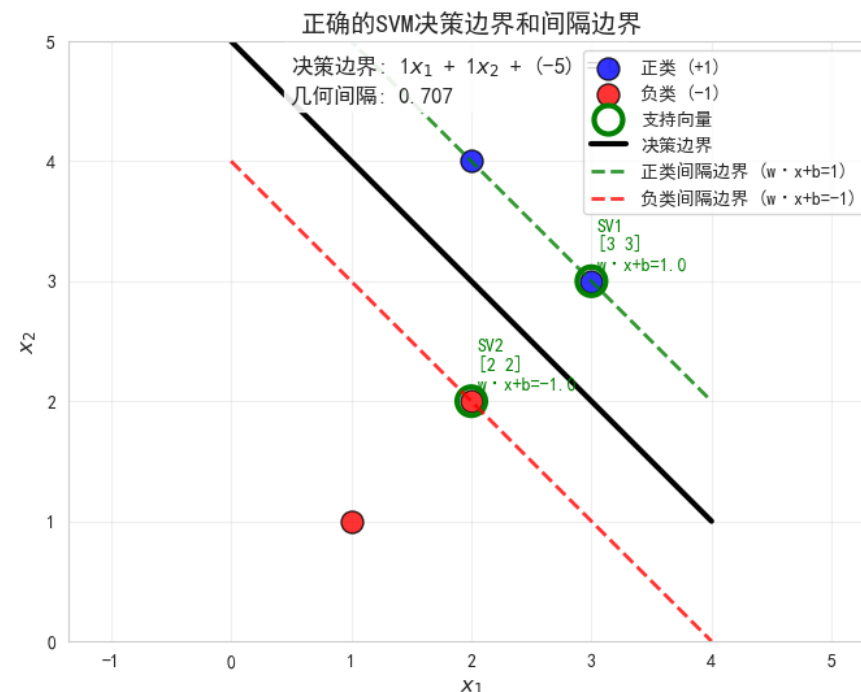
■ 对于 x_2 , $1 \cdot (0.5 \cdot 3 + 0.5 \cdot 3 - 2.5) = 0.5 < 1$

■ 对于 x_4 , $-1 \cdot (0.5 \cdot 2 + 0.5 \cdot 2 - 2.5) = 0.5 < 1$

■ 需要调整函数值的尺度

$$f(x) = x_1 + x_2 - 5 = 0$$

■ 间隔 $M = 1/\|w\| \approx 0.707$



优化目标: $\min_{w,b} \frac{1}{2} \|w\|^2$
subject to $y_i (w^T x_i + b) \geq 1, \forall i$

拉格朗日对偶问题:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & \sum_{i=1}^n \lambda_i y_i = 0, \lambda_i \geq 0 \end{aligned}$$

目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

10

类不均衡问题

6. 组合方法

1. 基本概念

- 组合(ensemble)或分类器组合(classifier combination): 通过聚集多个分类器的预测来提高分类准确率的技术
- 训练数据构建一组基分类器(base classifier), 并通过对每个基分类器的预测进行投票来进行分类

6. 组合方法

1. 基本概念

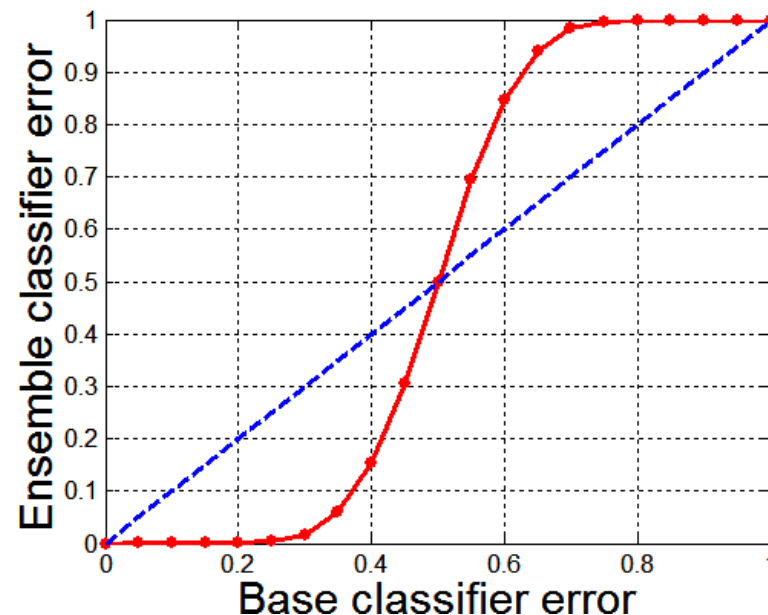
- 考虑25个二分类器的组合，每个分类器的错误率 ϵ 为0.35
- 如果所有的基分类器都相同，组合分类器的错误率仍然是 0.35
- 如果所有的基分类器相互独立，仅当超过一半的基分类器都预测错误时，组合分类器才会做出错误的预测。在这种情况下，组合分类器的错误率是：

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06 \ll 0.35$$

6. 组合方法

1. 基本概念

■ 当 ϵ 大于0.5时，组合分类器的性能比不上基分类器



■ 组合分类器的性能优于单个分类器必须满足两个必要条件:

■ (1) 基分类器应该相互独立;

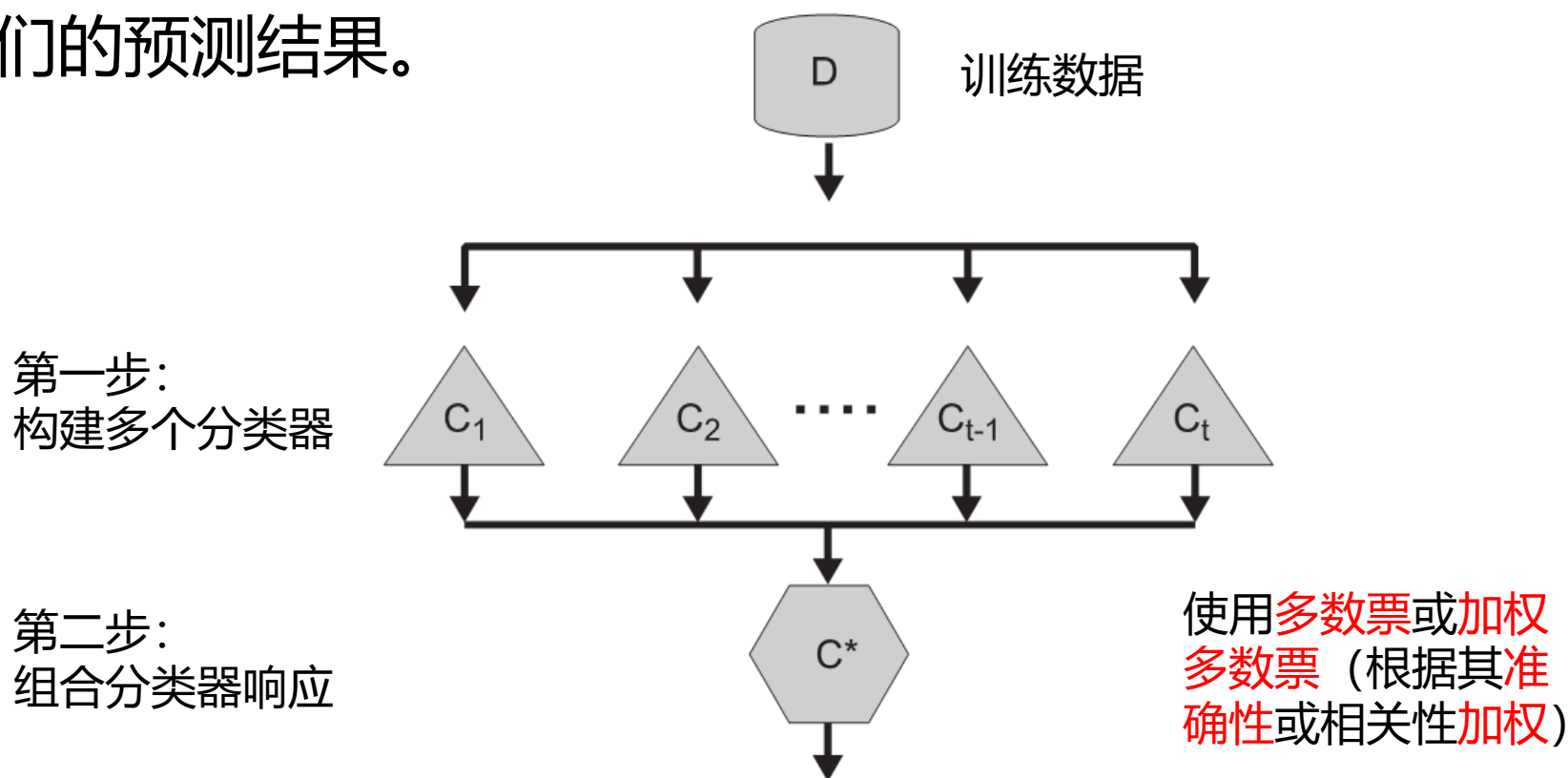
■ (2) 基分类器应该好于随机猜测分类器。 $(\epsilon < 0.5)$

■ 在基分类器轻微相关的情况下，组合方法可以提高分类的准确率

6. 组合方法

2. 构建组合分类器的方法

- 基本思想是在原始数据上构建多个分类器，然后在分类未知样本时聚集它们的预测结果。



6. 组合方法

2. 构建组合分类器的方法

- 1) **通过处理训练数据集**。这种方法根据**某种抽样分布**，通过对**原始数据**进行**再抽样**来得到**多个训练集**，并用**每个训练集**构建分类器。
- 抽样分布决定一个样本被选作训练样本的可能性大小，并且这个可能性因实验而异。
- **装袋**(bagging)和**提升**(boosting)是两种**处理训练集**的组合方法。

6. 组合方法

2. 构建组合分类器的方法

- 2) **通过处理输入特征**。在这种方法中，通过**选择输入特征**的子集来形成每个训练集。子集可以**随机选择**，也可以根据领域专家的建议选择。一些研究表明，对于那些含有大量冗余特征的数据集，这种方法的性能非常好。
- **随机森林**(random forest)就是一种**处理输入特征**的组合方法，它使用**决策树**作为基分类器。

6. 组合方法

2. 构建组合分类器的方法

分类器 f_2 将 C_1 类和 C_3 类的样例作为正例, C_2 类和 C_4 类的样例作为反例

	f_1	f_2	f_3	f_4	f_5	票数
	↓	↓	↓	↓	↓	
$C_1 \rightarrow$	-1	+1	-1	+1	+1	2
$C_2 \rightarrow$	+1	-1	-1	+1	-1	1
$C_3 \rightarrow$	-1	+1	+1	-1	+1	4
$C_4 \rightarrow$	-1	-1	+1	+1	-1	3
测试示例 $\rightarrow$	-1	-1	+1	-1	+1	

- 3)通过处理类标签。这种方法适用于类数足够多的情况。
- 通过将类标签随机划分成两个不相交的子集 A_0 和 A_1 , 把训练数据变换为二分类问题。
- 类标签属于子集 A_0 的训练样本指派到类+1, 而那些类标签属于子集 A_1 的训练样本指派到类-1。然后使用重新标记过的数据来训练一个基分类器。通过多次重复该过程, 就得到一组基分类器。
- 当遇到一个测试样本时, 使用每个基分类器 f_i 预测它的类标签。如果测试样本被预测为类+1, 则所有属于 A_0 的类都将得到一票。相反, 如果它被预测为类-1, 则所有属于 A_1 的类都得到一票。最后统计选票, 将测试样本指派到得票最高的类。
- 错误-纠错输出编码(error-correcting output coding, ECOC)方法就是这种方法的一个例子。

6. 组合方法

2. 构建组合分类器的方法

- 4) **通过处理学习算法**。许多学习算法都可以按如下方式来处理：在**相同的训练数据上多次执行算法**可能构建不同的分类器。例如，人工神经网络可以改变它的网络拓扑结构或各神经元之间连接的初始权重。
- 同样，通过在树生成过程中**注入随机性**可以得到决策树的组合分类器。例如，在每一个结点上，可以**随机地从最好的 k 个属性中选择一个属性**，而不是选择该结点上**最好的属性**来进行划分。

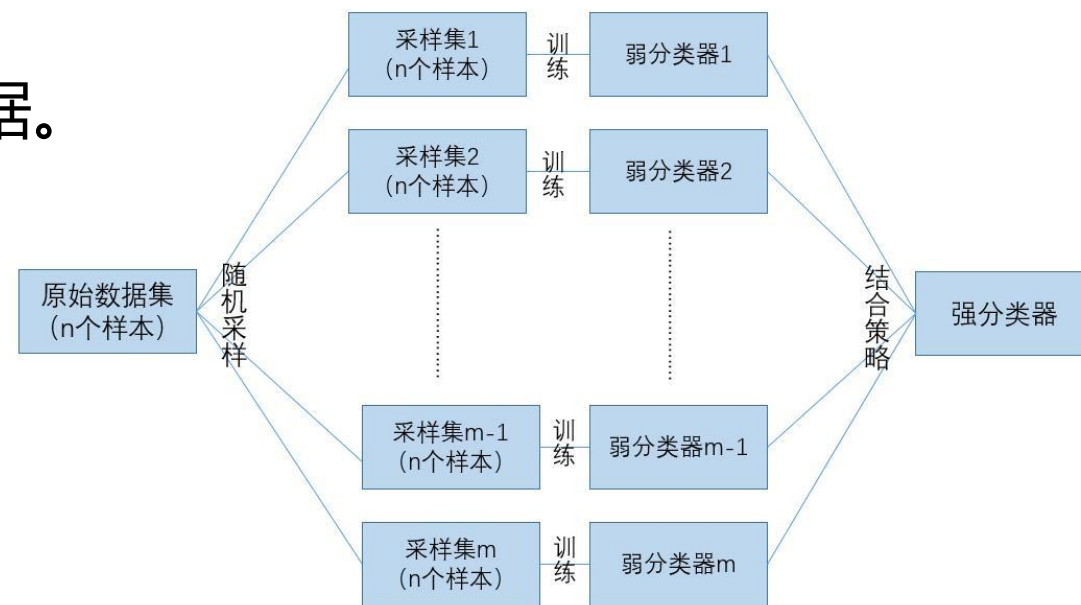
6. 组合方法

约有**36.8%**的样本不出现

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n \mapsto \frac{1}{e} \approx 0.368$$

3. 装袋

- 装袋(bagging): 也称为自助聚集(bootstrap aggregating), 是一种根据均匀概率分布从数据集中**有放回的重重复抽样**技术。每个**自助样本集**都和**原始数据集一样大**。
- 自助样本 D_i 大约包含 63% 的原始训练数据。
- 训练过 k 个分类器后, 测试实例被指派到**得票最高**的类。



6. 组合方法

3. 装袋

算法4.5 装袋算法

1:	设 k 为自助样本集的数目
2:	for $i = 1$ to k do
3:	生成一个大小为 N 的自助样本集 D_i
4:	在自助样本集 D_i 上训练一个基分类器 C_i
5:	end for
6:	$C^*(x) = \operatorname{argmax}_y \sum_i \delta(C_i(x) = y)$
	{如果参数为真, 则 $\delta(\cdot)=1$, 否则 $\delta(\cdot)=0$ }

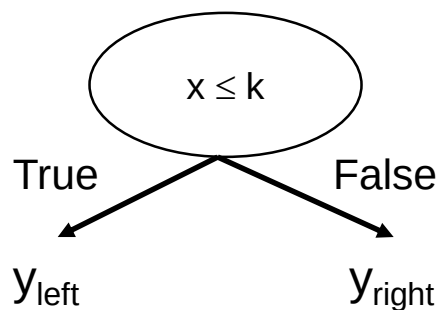
6. 组合方法

3. 装袋

- 设 x 表示一维属性， y 表示标签。假设使用这样一个分类器，它是**仅包含一层的二叉决策树**，具有一个测试条件 $x \leq k$ ， k 是使得叶结点熵最小的划分点。这样的树也称为**决策树桩**(decision stump)。
- 不进行装袋，能产生的最好的决策树桩的分裂点为 $x \leq 0.35$ 或 0.75 。无论选择哪一个，树的准确率最多为 70%

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1



6. 组合方法

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$

$x > 0.7 \rightarrow y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$

$x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

6. 组合方法

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = -1$

$x > 0.05 \rightarrow y = 1$

6. 组合方法

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

3. 装袋

■ 使用多数投票（预测和的符号）来确定组合分类器的类别

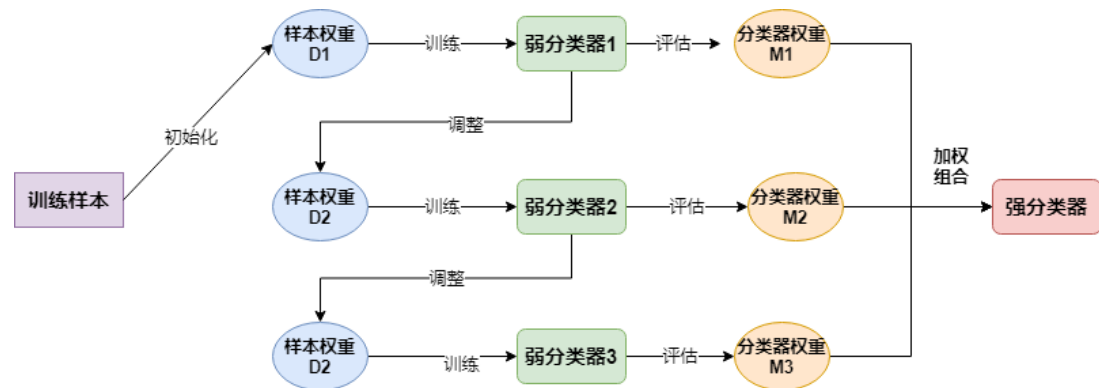
Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1

预测的类



100%准确!

6. 组合方法



4. 提升

- 提升(boosting)是一个迭代的过程，用于自适应地改变训练样本的分布，使得基分类器聚焦那些难以分类的样本。
- 不同于装袋，提升给每个训练样本赋一个权重，而且可以在每一轮提升过程结束时自动地调整权重。
- 训练样本的权重可以用于以下方面：
 - 1) 它们可用于指导从原始数据中抽取自助样本集时所用的抽样分布;
 - 2) 基本分类器可以使用它们来学习偏向于更高权重样本的模型。

6. 组合方法

4. 提升

- 初始时，所有样本都被赋予相同的权重 $\frac{1}{N}$ ，从而使得它们被选择用于训练的可能性都一样。
- 每一轮提升结束时更新训练样本的权重。增加被错误分类的样本的权重，而减小被正确分类的样本的权重。

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

- 样本4很难分类。随着它被重复地错误分类，该样本的权重在后面的迭代中将会增加。
- 同时，前一轮没有被选中的样本(如样本1和5)也有更好的机会在下一轮被选中。

6. 组合方法

4. 提升 --- AdaBoost

- 令 $\{(x_j, y_j) \mid j = 1, 2, \dots, N\}$ 表示包含 N 个训练样本的集合。在AdaBoost 算法中，基分类器 C_i 的**重要性**取决于其**错误率**，该错误率定义为：

$$\epsilon_i = \frac{1}{N}[\sum_j w_j \delta(C_i(x_j) \neq y_j)]$$

- 如果谓词 p 为真，则 $\delta(p)=1$ ，否则 $\delta(p)=0$ 。
- $w_j^{(i)}$ 表示在第 i 轮提升迭代中分配给样本 (x_j, y_j) 的权重，并按如下方式更新：

$$w_j^{(i+1)} = \frac{w_j^{(i)}}{Z_i} \times \begin{cases} e^{-\alpha_i} & \text{if } C_i(x_j) = y_j \\ e^{\alpha_i} & \text{if } C_i(x_j) \neq y_j \end{cases} \quad (4.103)$$

e^{α_i} 肯定是 ≥ 1 的数，因为 $\alpha_i \geq 0$

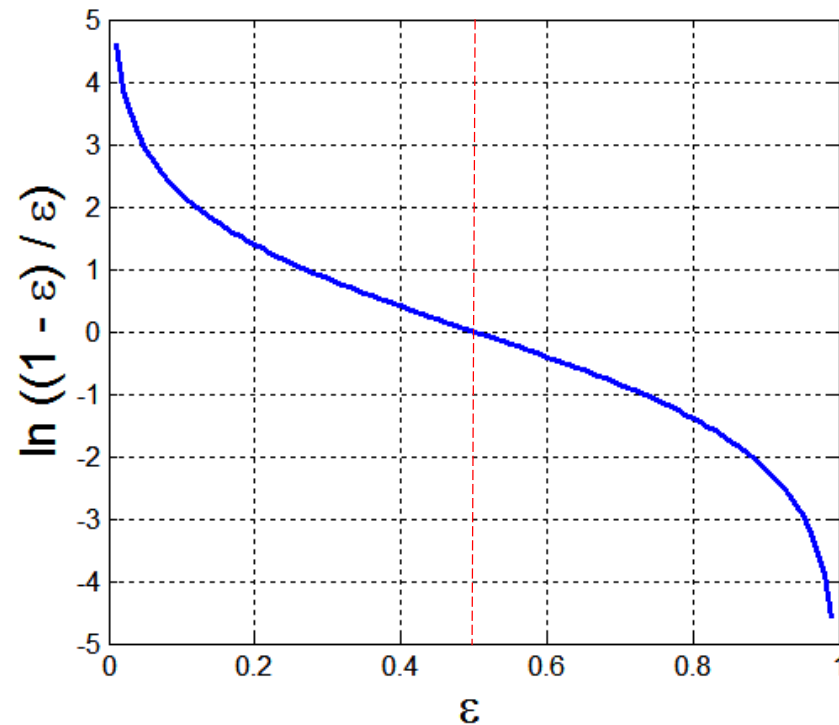
6. 组合方法

4. 提升 --- AdaBoost

- α_i 称为基分类器 C_i 的重要性，定义为：

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$

- 如果 ϵ_i 接近0，则 α_i 是一个很大的正值；而当 ϵ_i 接近1， α_i 是一个很大的负值
- AdaBoost 算法将每一个分类器 C_i 的预测值根据 α_i 进行加权，而不是使用多数表决的方案。
- 如果任何中间轮产生高于50%的错误率，则将权重恢复为开始的一致值 $w_j = \frac{1}{N}$ ，并重新进行抽样。



6. 组合方法

4. 提升 --- AdaBoost

$$w_j^{(i+1)} = \frac{w_j^{(i)}}{Z_i} \times \begin{cases} e^{-\alpha_i} & \text{if } C_i(x_j) = y_j \\ e^{\alpha_i} & \text{if } C_i(x_j) \neq y_j \end{cases}$$

算法4.6 AdaBoost算法

```
1:  $w = \{w_j = \frac{1}{N} | j = 1, 2, \dots, N\}$  /*初始化N个样本的权重*/
2: 令 $k$ 表示提升的轮数
3: for  $i=1$  to  $k$  do
4:   根据 $w$ , 通过对 $D$ 进行有放回地抽样产生训练集 $D_i$ 
5:   在 $D_i$ 上训练基分类器 $C_i$ 
6:   用 $C_i$ 对原训练集 $D$ 中的所有样本分类
7:    $\epsilon_i = \frac{1}{N} [\sum_j w_j \delta(C_i(x_j) \neq y_j)]$  /*计算加权误差*/
8:   if  $\epsilon_i > 0.5$  then
9:      $w = \{w_j = \frac{1}{N} | j = 1, 2, \dots, N\}$  /*重置N个样本的权重*/
10:    goto Step 4
11:   end if
12:    $\alpha_i = \frac{1}{2} \ln \left( \frac{1-\epsilon_i}{\epsilon_i} \right)$ 
13:   根据式(4.103)更新每个样本的权重
14: end for
15:  $C^*(x) = \underset{y}{argmax} \sum_{j=1}^T \alpha_j \delta(C_i(x) = y_j)$ 
```

6. 组合方法

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

4. 提升 --- AdaBoost

Boosting Round 1:

x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Round	Split Point	Left Class	Right Class	alpha
1	0.75	-1	1	1.738

因为0.1~0.3都分错, $\epsilon_1 = \frac{1}{N}[\sum_j w_j \delta(C_i(x_j) \neq y_j)] = \frac{1}{10}[\frac{1}{10} \times 3] = \frac{3}{100}$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_1}{\epsilon_1} \right) = \frac{1}{2} \ln \left(\frac{97}{3} \right) = 1.738 \quad e^{\alpha_1} = \left(\frac{97}{3} \right)^{\frac{1}{2}} = 5.686 \quad e^{-\alpha_1} = 1/e^{\alpha_1} = 0.176$$

$$Z_1 = \underbrace{\frac{1}{10} \times e^{\alpha_1} \times 3}_{x = 0.1 \sim 0.3} + \underbrace{\frac{1}{10} \times e^{-\alpha_1} \times 7}_{x = 0.4 \sim 1} = 1.829$$
$$w_j^{(2)} = \begin{cases} \frac{\frac{1}{10} \times e^{\alpha_1}}{1.829} = 0.311 & \text{对于 } x = 0.1 \sim 0.3 \uparrow \\ \frac{\frac{1}{10} \times e^{-\alpha_1}}{1.829} = 0.0096 & \text{对于 } x = 0.4 \sim 1.0 \downarrow \end{cases}$$

6. 组合方法

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

4. 提升 --- AdaBoost

Boosting Round 2:

x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Round	Split Point	Left Class	Right Class	alpha
2	0.05	1	1	2.7784

因为0.4~0.7都分错, $\epsilon_2 = \frac{1}{N}[\sum_j w_j \delta(C_i(x_j) \neq y_j)] = \frac{1}{10}[0.0096 \times 4] = 0.00384$

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_2}{\epsilon_2} \right) = 2.7784 \quad e^{\alpha_2} = \left(\frac{1 - \epsilon_2}{\epsilon_2} \right)^{\frac{1}{2}} = 16.0933 \quad e^{-\alpha_2} = 1/e^{\alpha_2} = 0.06214$$

$$Z_2 = \underbrace{0.311 \times e^{-\alpha_2} \times 3}_{x=0.1 \sim 0.3} + \underbrace{0.0096 \times e^{\alpha_2} \times 4}_{x=0.4 \sim 0.7} + \underbrace{0.0096 \times e^{-\alpha_2} \times 3}_{x=0.8 \sim 1} = 0.67778$$
$$w_j^{(3)} = \begin{cases} \frac{0.311 \times e^{-\alpha_2}}{0.67778} = 0.0285 & \text{对于 } x = 0.1 \sim 0.3 \downarrow \\ \frac{0.0096 \times e^{\alpha_2}}{0.67778} = 0.2279 & \text{对于 } x = 0.4 \sim 0.7 \uparrow \\ \frac{0.0096 \times e^{-\alpha_2}}{0.67778} = 0.00088 & \text{对于 } x = 0.8 \sim 1.0 \downarrow \end{cases}$$

6. 组合方法

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

4. 提升 --- AdaBoost

训练记录的
权重

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.311	0.311	0.311	0.01	0.01	0.01	0.01	0.01	0.01	0.01
3	0.029	0.029	0.029	0.228	0.228	0.228	0.228	0.0009	0.0009	0.0009

alpha
1.738
2.7784
4.1195

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	5.16	5.16	5.16	-3.08	-3.08	-3.08	-3.08	0.397	0.397	0.397
Sign	1	1	1	-1	-1	-1	-1	1	1	1

预测的类 →

100%准确!

6. 组合方法

5. 随机森林

- 随机森林(Random Forest, RF)试图通过构建去相关(decorrelated)决策树组合分类器来提高泛化性能。
- 随机森林基于装袋的思想，使用不同训练数据集的自助样本来学习决策树。
- 随机森林与装袋的一个关键区别在于，在树的每个内部结点处，在一小组随机选择的属性中选择划分标准。
- 通过这种方式，随机森林不仅通过操纵训练实例（通过使用类似于装袋的自助样本），还通过操纵输入属性（通过在每个内部节点使用不同的属性子集）来构建决策树的集合。

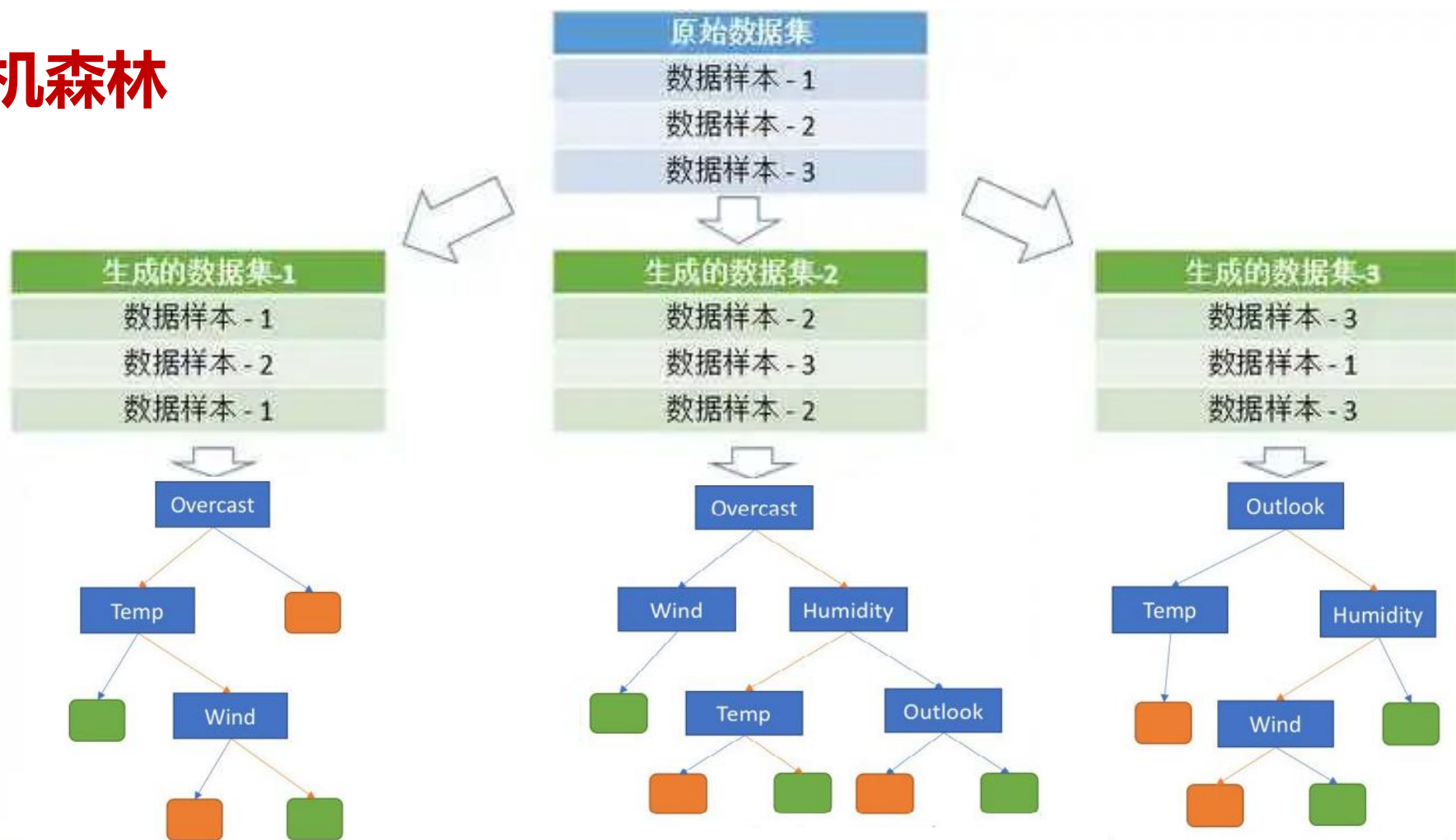
6. 组合方法

5. 随机森林

- 给定一个由 n 个实例和 d 个属性组成的训练集 D ，以下步骤为训练一个随机森林分类器的基本过程：
 - 1) 通过从 D 中**有放回地随机抽样** n 个实例来构建训练集的自助样本 D_i 。
 - 2) 使用 D_i 来学习决策树 T_i 。在 T_i 的每个内部结点处，**随机抽样一组 p 个属性**，并从该子集中**选择一个属性**，该属性可以**获得**用于划分的不纯度度量的**最大减少**。重复此过程直到每个叶子都是纯的，即包含来自同一类别的实例。

6. 组合方法

5. 随机森林





目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

10

类不均衡问题

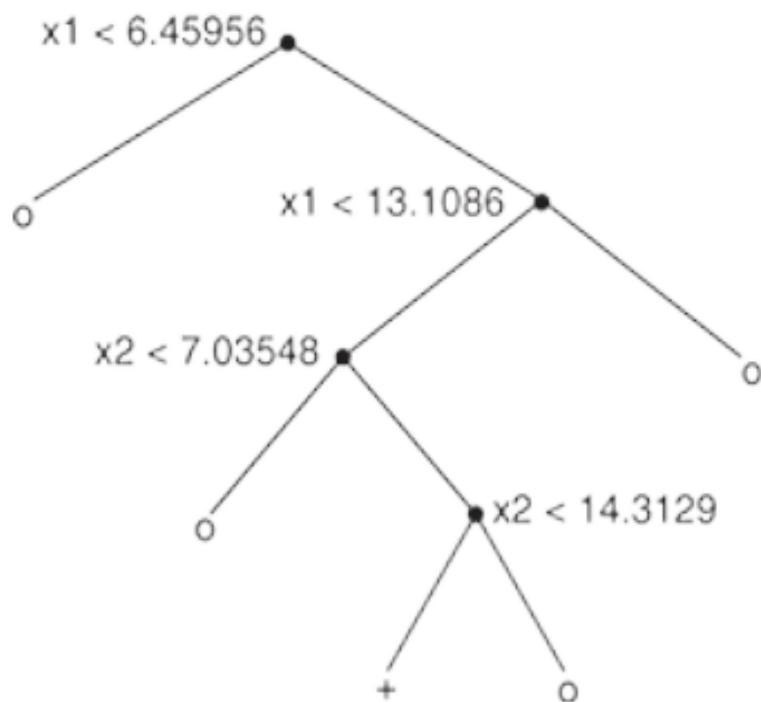
7. 模型的过拟合

1. 基本概念

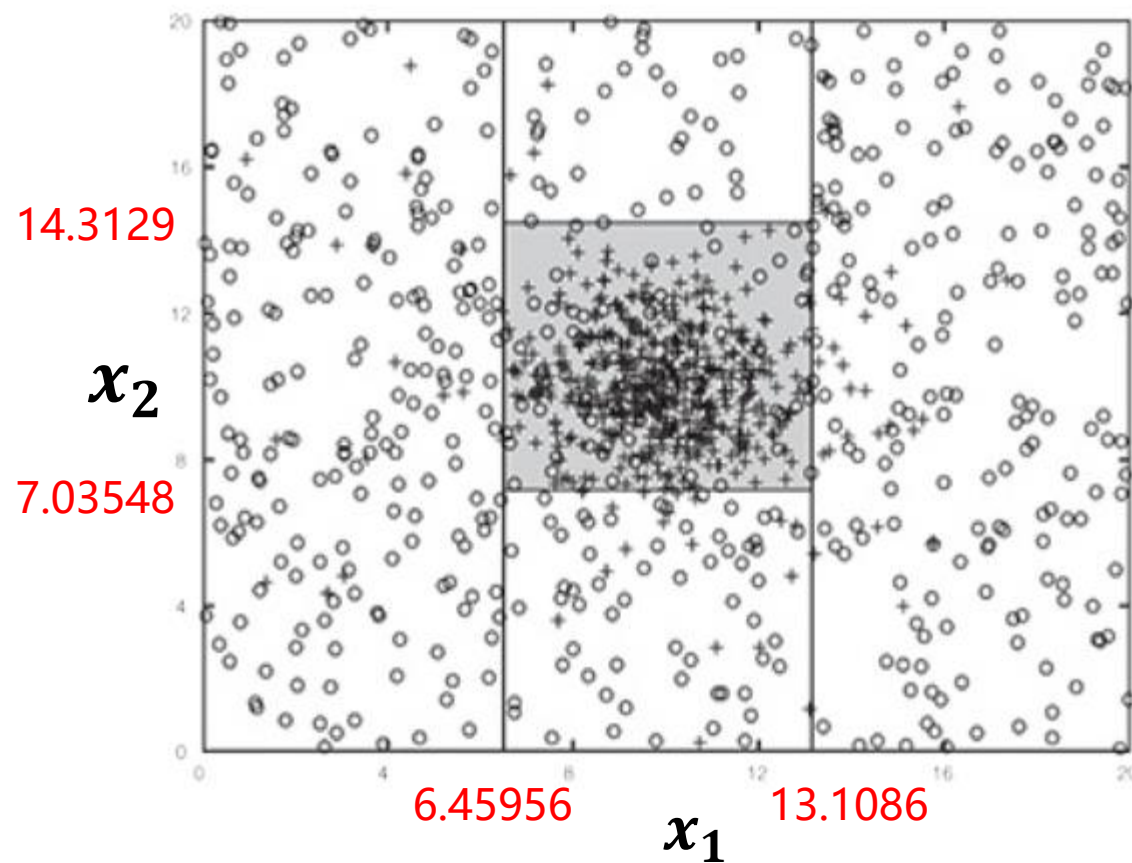
- 模型没有很好地捕捉到数据特征，不能够很好地拟合数据，这种情况称作模型欠拟合(model underfitting)
- 有的模型即使能够很好地覆盖训练数据，但它仍然可能表现出较差的泛化性能，这种现象称为模型的过拟合(model overfitting)
- 模型过拟合指的是在追求训练错误率最小化的过程中，选择了一种过于复杂的模型，这种模型捕捉到了训练数据中的特定模式，但没能获取到整体数据中属性和类别标签之间的本质关系

7. 模型的过拟合

1. 基本概念

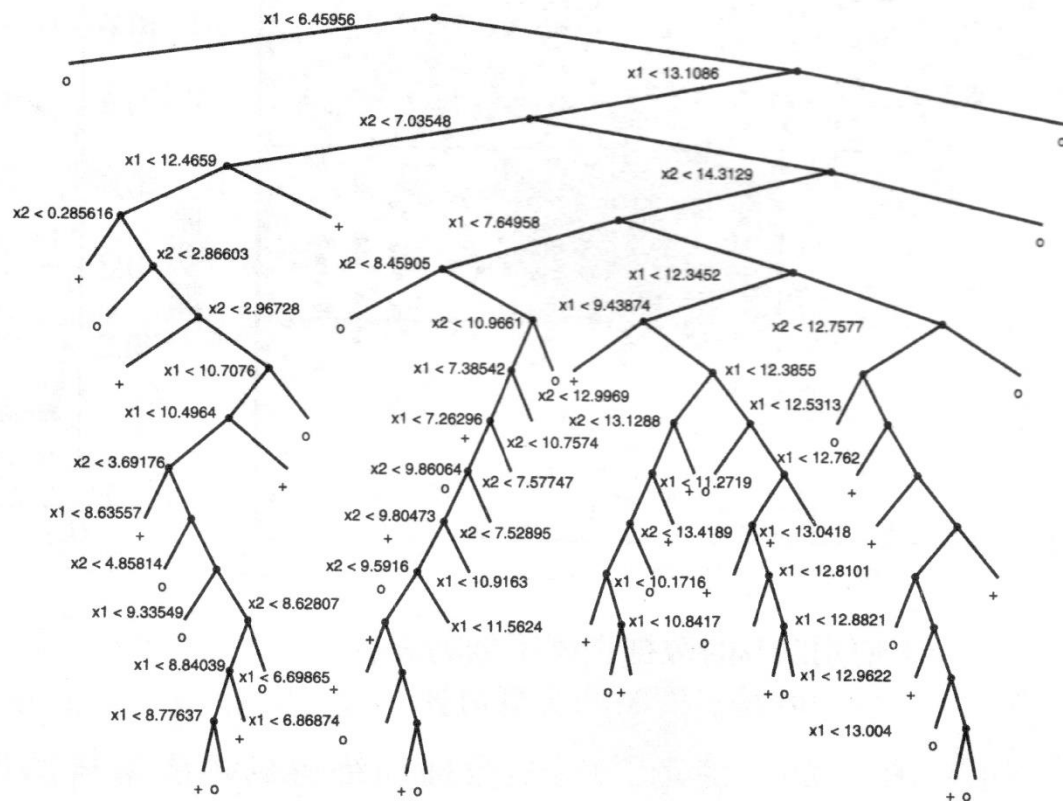


具有5个叶结点的决策树

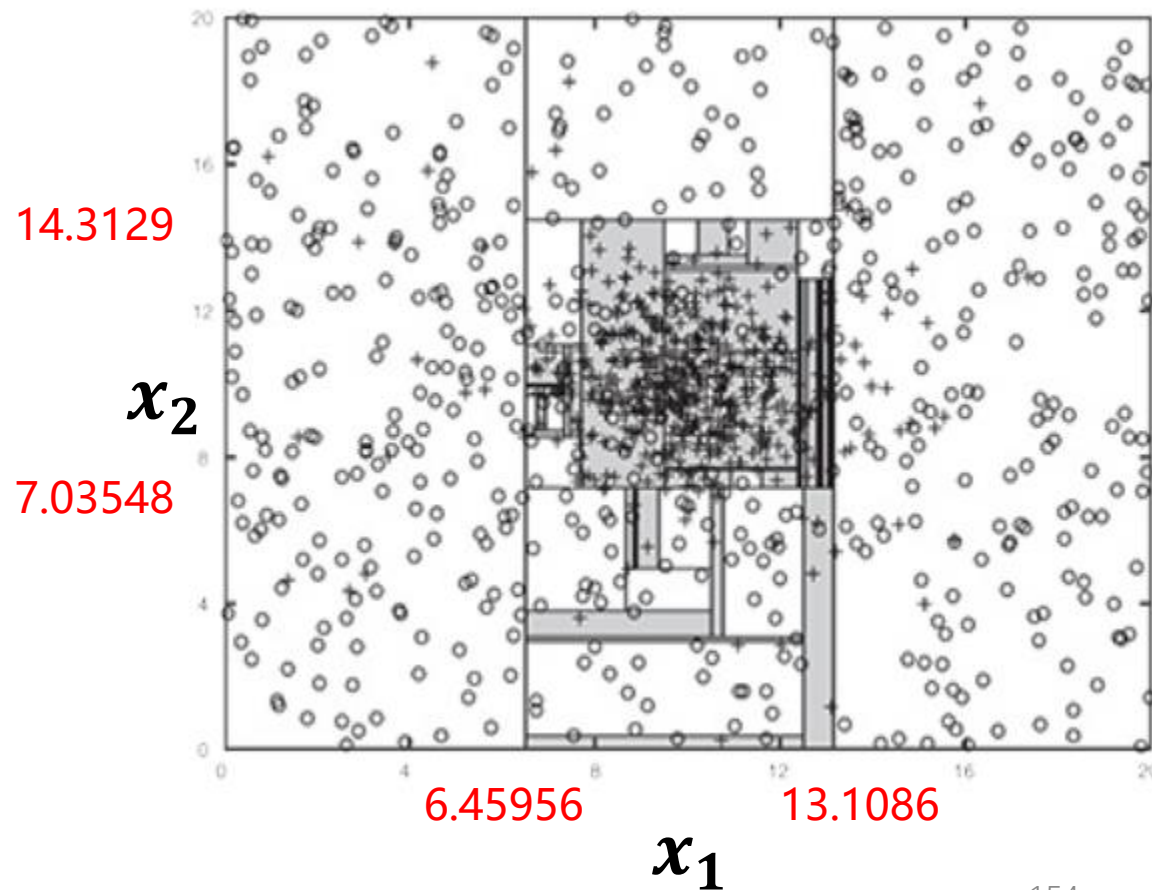


7. 模型的过拟合

1. 基本概念



具有50个叶结点的决策树



7. 模型的过拟合

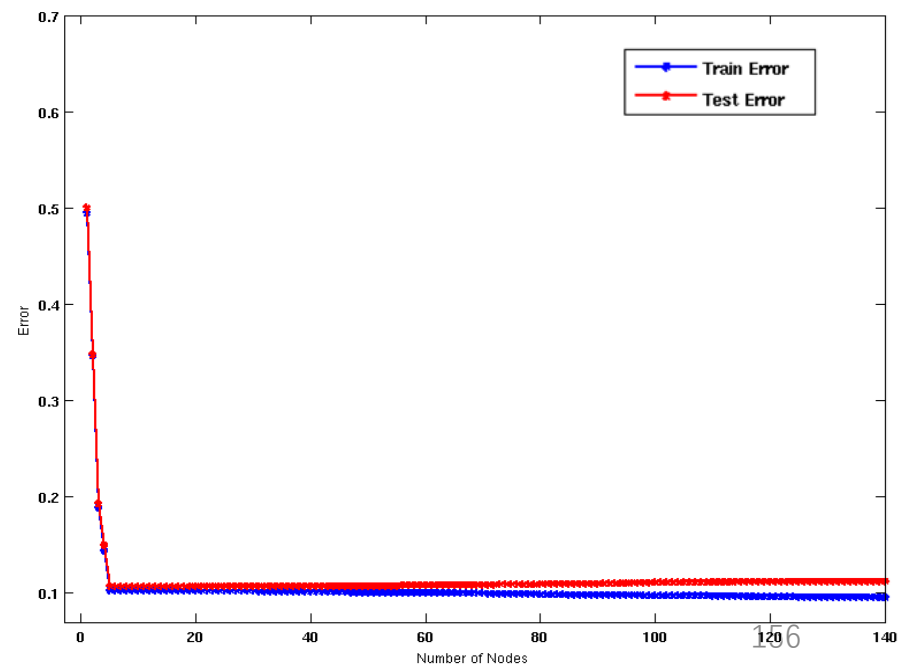
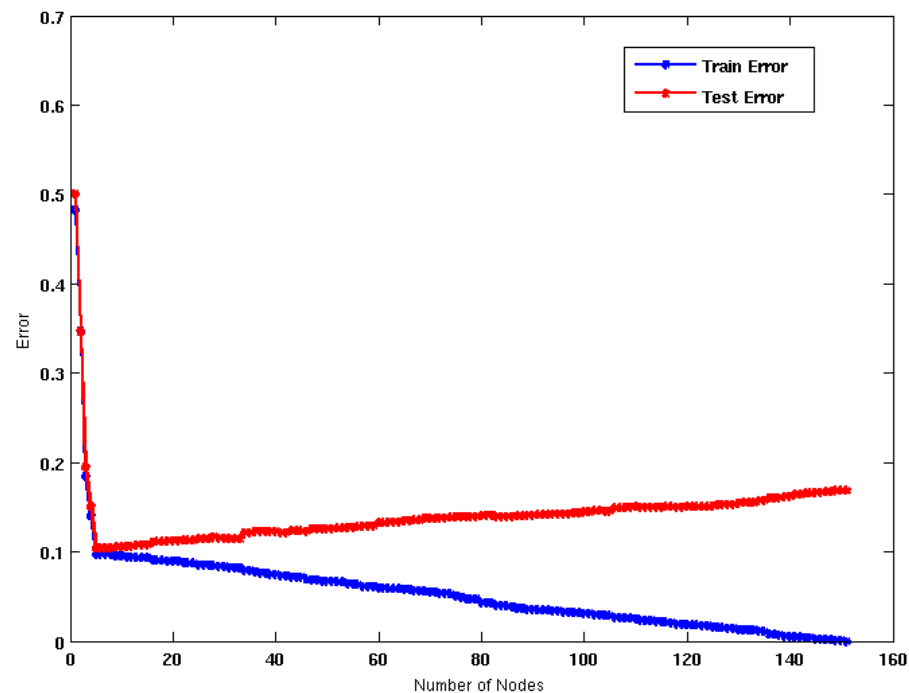
1. 基本概念

- 大小为5的决策树显得非常简单，它的决策边界为最佳决策边界提供了一个合理的近似值，在这种情况下，它对应于以(10, 10)处的高斯分布为中心的圆。虽然它的训练和测试错误率非零，但是它们十分接近，这表明在训练集中学习到的模式能很好地泛化到测试集。
- 规模为50的决策树比规模为5的决策树看上去更复杂，同时具有繁复的决策边界。为了完美拟合训练数据，规模为50的决策树开始根据训练数据中的特定模式进行微调，导致在独立选择的测试集上性能较差。

7. 模型的过拟合

2. 过拟合的原因 --- 有限的训练规模

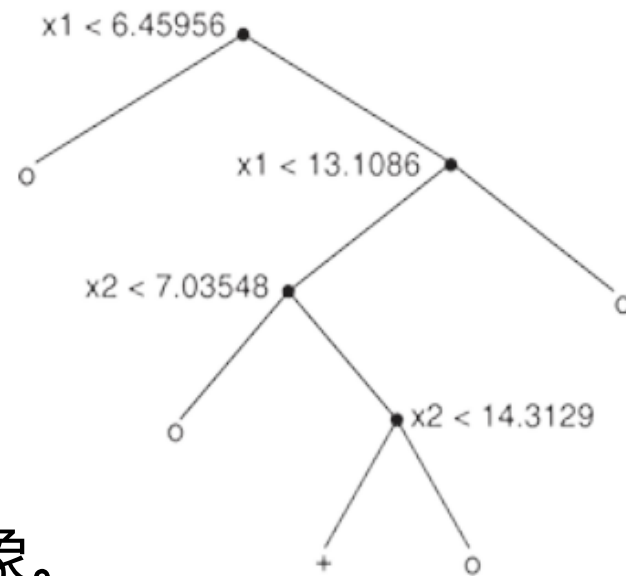
- 由有限个实例组成的训练集仅能对整体数据进行有限表示。
- 从训练集中学习得到的模式不能完全代表整体数据的实际模式。
- 通过增大训练规模可以减少过拟合的影响。
- 增加训练数据的大小(比如: 两倍数量的数据实例, 如下图)可以减少给定模型大小下训练和测试误差之间的差异。



7. 模型的过拟合

2. 过拟合的原因 --- 高模型复杂度

- 较为复杂的模型能够更好地表示数据中的复杂模式。
- 对于高度复杂的模型，应谨慎使用，避免出现过拟合现象。
- 从训练集中需要推导出的参数个数是模型复杂度的一种度量方式。例如，在决策树构造过程中，内部结点的属性测试条件数量与从训练集中推导出的模型参数数量一致。
- 具有大量属性测试条件(导致更多叶结点)的决策树具有更多的“参数”，因此更为复杂。



7. 模型的过拟合

2. 过拟合的原因 --- 高模型复杂度

- 给出一类模型以及一定数量的参数，一个学习算法试图筛选出令训练集的评价指标（比如：准确率）最大化的**最佳参数值组合**。
- 如果**参数值的组合数很多**(因此复杂度更高)，学习算法需要通过**有限的训练集**，从**大量可能的组合中筛选出最佳组合**。
- 在这种情况下，学习算法很有可能筛选出一个**虚假的参数组合**，仅通过**随机因素**使评价指标最大化。这类似于统计学中的**多重比较问题** (multiple comparisons problem)，也称**多重测试问题**。

7. 模型的过拟合

2. 过拟合的原因 --- 高模型复杂度

- 考虑对未来 10 个交易日股市涨跌进行预测的任务。如果一位股票分析师的预测只是随机猜测，那么对于任意交易日的预测，其准确率：

$$P(\text{correct}) = 0.5$$

- 然而，在10次预测中至少正确9次的概率为：

$$P(\#correct \geq 9) = \frac{\binom{10}{9} + \binom{10}{10}}{2^{10}} = 0.0107$$

↑
小概率事件

Day 1	Up
Day 2	Down
Day 3	Down
Day 4	Up
Day 5	Down
Day 6	Down
Day 7	Up
Day 8	Up
Day 9	Up
Day 10	Down

7. 模型的过拟合

2. 过拟合的原因 --- 高模型复杂度

- 假设我们要从 200位股票分析师中挑选出一位投资顾问。策略是挑选出在未来 10个交易日中，能做出最多正确预测的分析师。假设所有分析师的预测都是随机产生的，那么至少有一人正确预测不少于9次的概率为：

$$1 - (1 - 0.0107)^{200} = 0.8847$$

- 尽管对于每个分析师，正确预测不小于9次的概率都很低，但综合考虑后，至少找到一个这样的分析师的概率是很高的。然而，我们无法确保这样一个分析师在未来能通过随机猜测的方法一直做出正确的预测。

7. 模型的过拟合

2. 过拟合的原因 --- 高模型复杂度

- 在学习分类模型的过程中，每个参数值组合都相当于一个分析师(比如200)，而训练实例的数量相当于天数(比如10)。
- 类似于选择在连续几天内做出最准确预测的最佳分析师的任务，学习算法的任务是挑选出最适配训练集的参数值组合。
- 如果参数组合很多，但训练规模很小，学习算法极有可能随机选中一个虚假的参数组合，它仅凭偶然机会就能提供较高的训练准确度。

目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

10

类不均衡问题

8. 模型评估

1. 基本概念

- 估计模型泛化性能的方法被称为模型评估(model evaluation)
- 模型评估的正确方法是在有标签的测试集上，对学习模型的性能进行评估，且这些测试集未在模型选择的任何阶段被使用。
- 这可以通过将整个有标签的实例集合 D 划分为两个不相交的子集来实现。
- $D.train$ 用于模型选择，而 $D.test$ 用于计算测试错误率 err_{test} 。

8. 模型评估

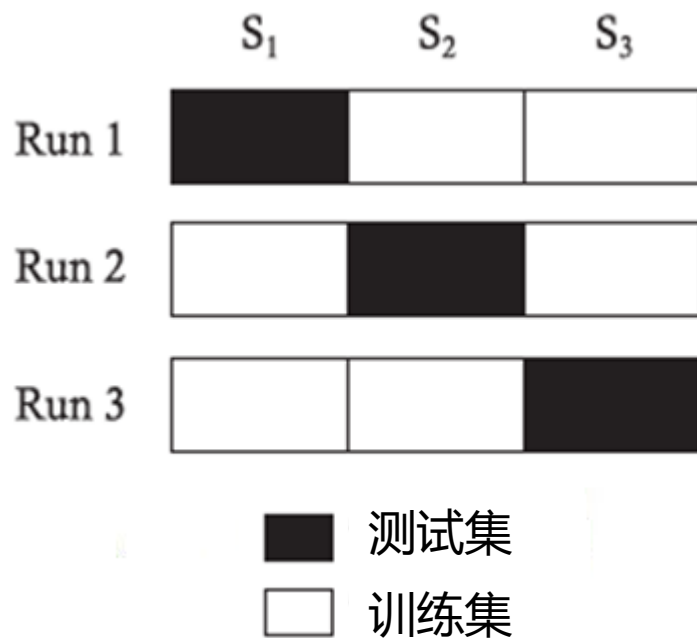
2. 留出法(Holdout)

- 在有标签数据集中，选择用于训练和测试的**正确比例**十分重要
- 如果 $D.train$ 的规模很小，使用**数量不足**的训练实例可能学习到**错误的分类模型**，从而导致对泛化性能的估计有偏差。
- 如果 $D.test$ 的规模很小，那么 err_{test} 可能**不太可靠**，因为它是基于少量测试实例计算出的。
- **留出法**可以多次重复后获得测试错误率的分布，这种方法称为**随机子采样**(random subsampling)或**重复留出法**。该方法生成的错误率分布有助于理解 err_{test} 的方差。

8. 模型评估

3. 交叉验证(Cross-Validation)

- 假设给定一个有标记的集合，将其随机地划分为三个相同大小的子集 S_1 , S_2 和 S_3 。第一次运行时，使用 S_2 和 S_3 训练模型，并在子集 S_1 上测试模型。
- 因此，在第一次运行计算中， S_1 上的测试错误率表示为 $err(S_1)$ 。
- 第二次， S_1 和 S_3 作为训练集， S_2 作为测试集，测试错误率 $err(S_2)$ 。
- 第二次， S_1 和 S_2 作为训练集， S_3 作为测试集，测试错误率 $err(S_3)$ 。
- 总的测试错误率由所有运行中的子集测试错误数之和除以总的实例数。



三折交叉验证

8. 模型评估

3. 交叉验证(Cross-Validation)

- **k 折交叉验证**方法一般指将标记数据 D (大小为 N) 分成个 **k 相等大小**的分区 (或子类)的方法。
- 在第 i 次运行中, D 的一个分区作为 $D.test(i)$ 用于测试, 其余的分区作为 $D.train(i)$ 用于训练。
- 使用 $D.train(i)$ 学习得到模型 $m(i)$, 并在 $D.test(i)$ 上获得测试错误数之和 $err_{sum}(i)$ 。该过程**重复 k 次**。总测试错误率 err_{test} 的计算如下:

$$err_{test} = \frac{\sum_{i=1}^k err_{sum}(i)}{N}$$

8. 模型评估

3. 交叉验证(Cross-Validation)

- 数据集中的每个实例恰好用于一次测试、 $(k - 1)$ 次训练。每次运行使用 $\frac{k-1}{k}$ 部分的数据进行训练，用 $\frac{1}{k}$ 部分的数据进行测试。
- 在极端情况下，当 $k = N$ 时，每次运行只使用一个数据实例进行测试，其余的数据用于训练。这种 k 折交叉验证的特殊情况称为留一法(leave-one-out)。
- 对于大多数实际应用， k 在5到10之间进行选择，可提供估计泛化错误率的合理方法。

目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

10

类不均衡问题

9. 超参数的使用

1. 基本概念

- 超参数是学习算法的参数，需要在学习分类模型之前确定。
- 与常规模型参数（如决策树内部节点中的测试条件）不同，超参数（如决策树最大深度）并不会出现在用于分类未标记实例的最终模型中。
- 超参数的取值需要在模型选择期间被确定，该过程称为超参数选择 (hyper-parameter selection)，同时模型评估期间也需要考虑超参数。
- 稍微修改前一节中描述的交叉验证方法，就可以有效地完成这两项任务。

9. 超参数的使用

第3次运行

$D.tr$	$D.tr$	$D.val$
--------	--------	---------

第2次运行

$D.tr$	$D.val$	$D.tr$
--------	---------	--------

第1次运行

$D.val$	$D.tr$	$D.tr$
---------	--------	--------

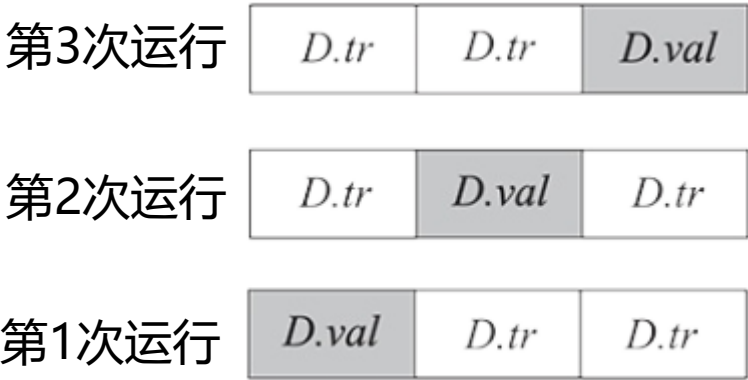
2. 超参数选择

- 设 p 是从有限值域 $P = \{p_1, p_2, \dots, p_n\}$ 中选取的超参数
- 将 $D.train$ 划分为 $D.tr$ 和 $D.val$ 。对于每个超参数的取值 p_i , 可以在 $D.tr$ 上习得模型 m_i , 并在 $D.val$ 上获得模型的验证错误率 $err_{val}(p_i)$
- 选择验证错误率最低时的超参数 p^* , 并使用它在整个训练集 $D.train$ 上习得模型 m^*

9. 超参数的使用

算法3.2 模型选择步骤($k, P, D.train$)

1:	$N_{train} = D.train $	/* $D.train$ 的大小*/
2:	将 $D.train$ 分割成 k 份, 即 $D.train_1, D.train_2, \dots, D.train_k$	
3:	for $i = 1$ to k do	
4:	$D.val(i) = D.train_i$	/*用于验证的分区*/
5:	$D.tr(i) = D.train \setminus D.train_i$	/*用于训练的分区*/
6:	for each $p \in P$ do	
7:	$m = \mathbf{model-train}(p, D.tr(i))$	/*训练模型*/
8:	$err_{sum}(p, i) = \mathbf{model-test}(m, D.val(i))$	/*验证错误和*/
9:	end for	
10:	end for	
11:	$err_{val}(p) = \frac{\sum_{i=1}^k err_{sum}(p, i)}{N_{train}}$	/*计算验证错误率*/
12:	$p^* = \underset{p}{\operatorname{argmin}} err_{val}(p)$	/*选择最佳的超参数值*/
13:	$m^* = \mathbf{model-train}(p^*, D.train)$	/*在 $D.train$ 上学习最终模型*/
14:	return (p^*, m^*)	



目 录

06

组合方法

07

模型的过拟合

08

模型评估

09

超参数的使用

10

类不平衡问题

10. 类不平衡问题

1. 基本概念

- 在许多数据集中，不同类别的实例数量是不接近的，这种性质称为**倾斜**(skew)或**类不平衡**(class imbalance)
 - 信用卡诈骗
 - 入侵检测
 - 制造装配线上的缺陷产品
- 类之间的不平衡度在**不同的应用之间**，甚至在同一个应用的**不同数据集之间**都会不同

10. 类不平衡问题

1. 基本概念

- 类不平衡给分类带来了两个挑战：
- 1) 很难找到充足的稀有类别的样本。在不平衡数据集上进行训练的分类器显示出偏向于改善多数类（majority class）的性能，这往往不是所期望的行为。因此，现有的许多分类模型在不平衡数据集上进行训练时，可能无法有效检测稀有类（rare class）的实例。

10. 类不平衡问题

1. 基本概念

- 类不平衡给分类带来了两个挑战：
- 2) **准确率**是**评估**分类性能的传统方法，**不适合**在**类不平衡**的测试数据中评估模型。例如，如果1%的信用卡交易是欺诈性的，那么一个预测每笔交易合法的普通模型即使未能检测到任何欺诈活动，也会有 99%的准确性。因此，需要使用**对倾斜敏感**的替代评估指标，并且可以捕获**与准确率不同**的性能指标。

10. 类不平衡问题

2. 类不平衡的分类器构建

- 在训练集中出现类不平衡时，建立分类器主要要考虑两个因素：
- 首先，需要确保在一个数据集上进行训练的学习算法既能够充分表示多数类，也能够表达少数类。确保这一点的一些常见方法包括对训练数据集的过采样和欠采样。
- 其次，在学习分类模型之后，需要一种方法来调整其分类决策(从而创建适当调整的分类器)，以最佳地匹配不平衡测试集的要求。这通常将分类模型的输出转换为实数分数，然后在分类分数上选择合适的阈值以匹配测试集的需求。

10. 类不平衡问题

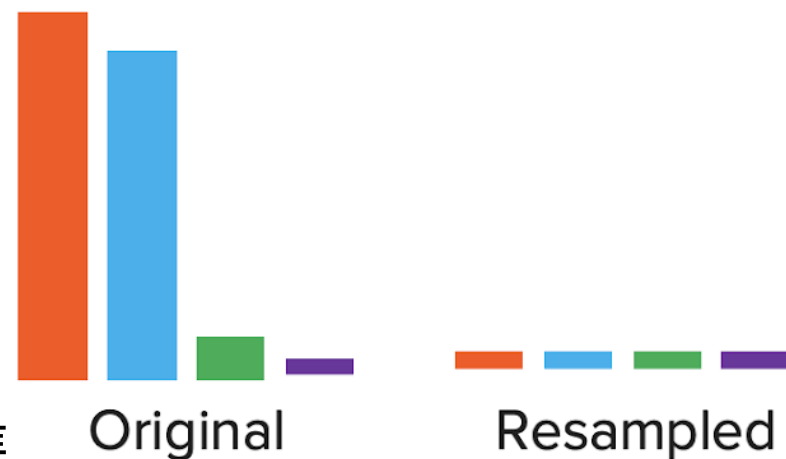
2. 类不平衡的分类器构建 --- 过采样和欠采样

- 学习不平衡数据的第一步是将训练集转换为平衡训练集。
- 平衡训练集可以使用现有的任何分类技术(不需要对学习算法进行任何修改)来学习一个模型, 该模型同等地对待两个类。
- 创建平衡训练集的一个基本方法是生成一个训练实例样本, 其中罕见类具有足够的代表性。有两种采样方法可用于增强少数类的代表性:
 - (a) 欠采样(undersampling), 降低多数类的频率来匹配少数类的频率;
 - (b) 过采样(oversampling), 生成少数类的人造实例, 使它们与负实例的数量成比例。

10. 类不平衡问题

2. 类不平衡的分类器构建 --- 欠采样

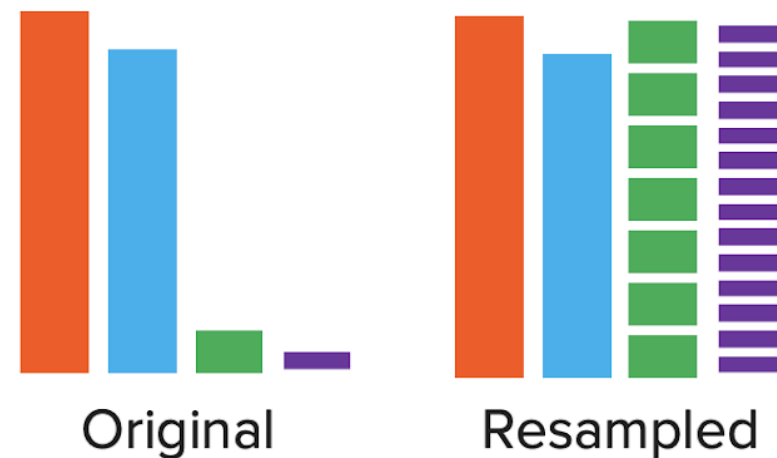
- 一个包含 100 个正实例和 1000 个负实例的训练集。
- 为了克服这些类之间的倾斜，可以从负类中选择 100 个实例的随机样本，并将它们与 100 个正实例一起构成一个平衡的训练集。
- 欠采样的一个局限是一些有意义的数据不被选来训练(例如，更接近实际决策边界的实例)，因此导致较差的分类模型。
- 另一个局限是 100 个负实例的较小样本可能比 1000 个较大数据集具有更高的方差。



10. 类不平衡问题

2. 类不平衡的分类器构建 --- 过采样

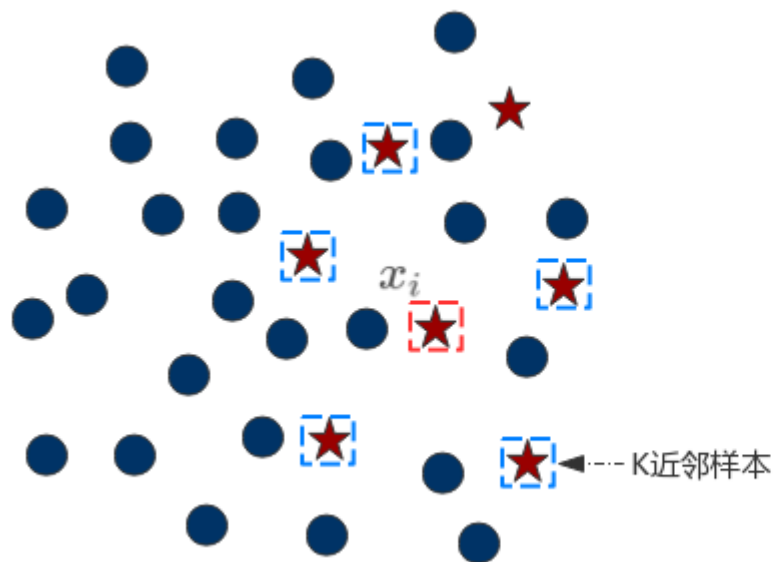
- 过采样的一个简单方法是复制每个正例 $\frac{n_-}{n_+}$ 次, 其中 n_+ 和 n_- 分别是正和负训练实例的数目。
- 过采样的一种方法是在现有正实例附近生成合成正实例。在这种被称为合成少数过采样技术(Synthetic Minority Oversampling Technique, SMOTE)的方法中, 首先确定每个正实例 x 最近的 k 个近邻正实例, 然后在连接 x 到随机选择的 k 近邻的线段上的某个中间点处生成一个合成正实例。重复该过程, 直到达到所需数量的正实例。



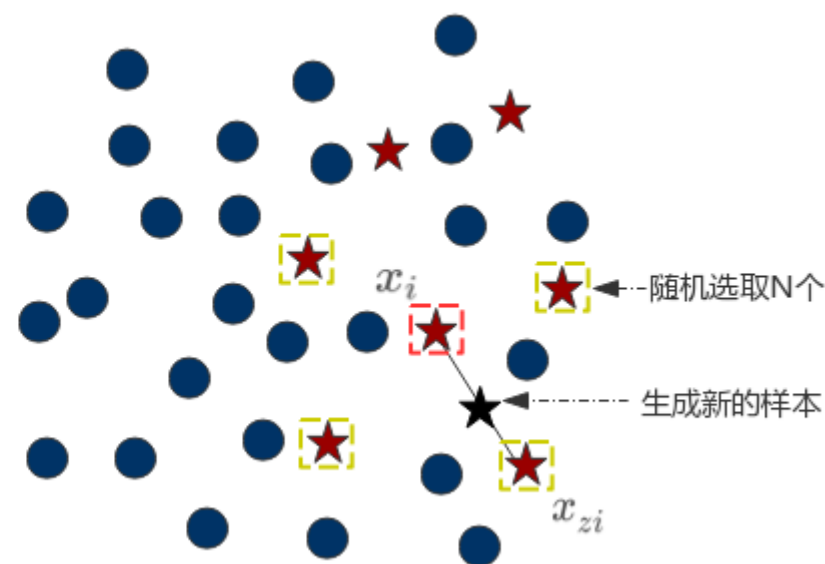
10. 类不平衡问题

2. 类不平衡的分类器构建 --- 过采样

- 过采样的一种方法是合成少数过采样技术(Synthetic Minority Oversampling Technique, **SMOTE**)的方法。



图(1). 计算 x_i 的 k 近邻样本



图(2). 合成新样本

10. 类不平衡问题

二分类混淆矩阵

红色: 预测正确
蓝色: 预测错误

		预测类别	
		类 = +	类 = -
实际类别	类 = +	f_{++}	f_{+-}
	类 = -	f_{-+}	f_{--}

3. 带类不平衡的性能评估

- 真正(True Positive, TP)或 f_{++} , 对应于被分类模型**正确预测**的**正样本**数。
- 假负(False Negative, FN)或 f_{+-} (也称为第一类错误), 对应于被分类模型**错误预测**为负类的**正样本**数。
- 假正(False Positive, FP)或 f_{-+} (也称为第二类错误), 对应于被分类模型**错误预测**为正类的**负样本**数。
- 真负(True Negative, TN)或 f_{--} , 对应于被分类模型**正确预测**的**负样本**数。
- 混淆矩阵中的计数通常使用若干评估方法(evaluation measure)进行概括。

10. 类不平衡问题

$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$

		预测类别	
		类 = +	类 = -
实际类别	类 = +	f_{++}	f_{+-}
	类 = -	f_{-+}	f_{--}

3. 带类不平衡的性能评估

- 准确率(Accuracy, Acc)将这4种计数结合成一个单一值，这种方法在类平衡时被广泛使用。
- 准确率度量不适合处理不平衡类分布的数据集，因为它往往倾向于正确分类多数类的分类器。

A	ACTUAL	PREDICTED	
		Class=Yes	Class=No
		Class=Yes	Class=No
	Class=Yes	0	10
	Class=No	0	990

Accuracy: 99%

B	ACTUAL	PREDICTED	
		Class=Yes	Class=No
		Class=Yes	Class=No
	Class=Yes	10	0
	Class=No	500	490

Accuracy: 50%

虽然模型A的Acc要比模型B的高很多，但是模型A真的好吗？

10. 类不平衡问题

		预测类别	
		类 = +	类 = -
实际类别	类 = +	f_{++}	f_{+-}
	类 = -	f_{-+}	f_{--}

3. 带类不平衡的性能评估

■ 对倾斜敏感的评估度量是精度(precision)，其可以定义为预测为正类的总数中被正确预测为正类的比例，即 $precision = \frac{TP}{TP+FP}$

■ 真正率(True Positive Rate, TPR)，它定义为由分类器正确预测的正测试实例的比例，也称为召回率 (recall)，即 $recall = \frac{TP}{TP+FN}$

■ F1 度量是基于精度与召回率的调和平均(harmonic mean)

$$F1 = \frac{2rp}{r+p} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

■ F度量的值较高确保精度和召回率都较高。

10. 类不平衡问题

3. 带类不平衡的性能评估



A

	PREDICTED CLASS		
		Class=Yes	Class=No
	ACTUAL CLASS		
	Class=Yes	10	0
	Class=No	10	980

B

	PREDICTED CLASS		
		Class=Yes	Class=No
	ACTUAL CLASS		
	Class=Yes	1	9
	Class=No	0	990

$$precision = \frac{TP}{TP+FP}$$

$$recall = \frac{TP}{TP+FN}$$

$$F1 = \frac{2rp}{r+p}$$

$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$

$$Precision(p) = \frac{10}{10+10} = 0.5$$

$$Recall(r) = \frac{10}{10+0} = 1$$

$$F\text{-measure}(F) = \frac{2*1*0.5}{1+0.5} = 0.67$$

$$Accuracy = \frac{990}{1000} = 0.99$$

$$Precision(p) = \frac{1}{1+0} = 1$$

$$Recall(r) = \frac{1}{1+9} = 0.1$$

$$F\text{-measure}(F) = \frac{2*0.1*1}{1+0.1} = 0.18$$

$$Accuracy = \frac{991}{1000} = 0.991$$